



كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

الجامعة السورية الخاصة
SPU SYRIAN PRIVATE UNIVERSITY

AI Student Assistance (Moein)

A junior project report - submitted to complete the requirements for obtaining a bachelor's degree in the Department of Artificial Intelligence Engineering and Data Science

Authors

MHD Zaher Krayem

MHD Yahia Abo Samra

MHD Awab Kheir

Othman Darkal

Supervised by

Dr. Riad Sonbol

Eng. Raghad Al Hossny

1447 – 1448 هـ

2025 – 2026 م

Damascus

SUPERVISION CERTIFICATION

Acknowledgements

The project team would like to express their sincere appreciation and gratitude to Dr. Riad Sonbol for his continuous guidance, valuable insights, and academic supervision throughout all stages of this project. His expertise, constructive feedback, and commitment to high academic standards played a crucial role in shaping the technical direction of the project and ensuring the quality of the final outcomes.

We also extend our sincere thanks to Eng. Raghad Al Hossny for her dedicated support, technical assistance, and thoughtful recommendations during the development process. Her guidance in system design, implementation practices, and problem-solving strategies significantly contributed to improving the robustness and reliability of the system.

In addition, we would like to acknowledge the efforts of the faculty members and academic staff for providing a supportive learning environment and the necessary resources required to carry out this project. Their encouragement and administrative support helped facilitate the smooth progress of the work.

Finally, we are grateful for the opportunity to apply theoretical knowledge in a practical setting through this project. The experience gained throughout the development process has been invaluable in strengthening our technical skills, teamwork abilities, and understanding of software engineering principles.

ABSTRACT

In modern higher education environments, students face increasing volumes of academic content, making efficient content organization and revision essential for effective learning outcomes. To address this challenge, educational software systems must not only manage learning materials but also enhance student understanding through intelligent processing and continuous improvement. Such systems play a critical role in reducing cognitive load and supporting students' evolving academic needs.

In response to this need, this report presents an intelligent Student Course Management and AI Summarization System designed to assist university students in organizing courses and comprehending lecture materials more efficiently. The system enables students to create and manage courses, upload lecture files in supported formats (PDF, DOCX, PPTX), and automatically generate structured text summaries using artificial intelligence techniques. By analyzing lecture content, the system produces concise and organized summaries that can be stored, reviewed, and managed by students for future use.

Through the integration of modern web technologies, secure authentication mechanisms, and AI-based text processing, the proposed system enhances the learning experience by minimizing revision time, improving content accessibility, and providing a scalable foundation for future educational features.

ملخص

في بيئات التعليم العالي الحديثة، يواجه الطلاب كميات متزايدة من المحتوى الأكاديمي، مما يجعل تنظيم المحتوى ومراجعته بكفاءة أمرًا ضروريًا لتحقيق نتائج تعليمية فعالة. ولمواجهة هذا التحدي، يجب ألا تقتصر أنظمة البرمجيات التعليمية على إدارة المواد التعليمية فحسب، بل يجب أن تُعزز أيضًا فهم الطلاب من خلال المعالجة الذكية والتحسين المستمر. وتلعب هذه الأنظمة دورًا حاسمًا في تخفيف العبء المعرفي ودعم الاحتياجات الأكاديمية المتطورة للطلاب.

استجابةً لهذه الحاجة، يقدم هذا التقرير نظامًا ذكيًا لإدارة المقررات الدراسية وتلخيصها باستخدام الذكاء الاصطناعي، مصممًا لمساعدة طلاب الجامعات على تنظيم مقرراتهم الدراسية وفهم مواد المحاضرات بكفاءة أكبر. يُمكن النظام الطلاب من إنشاء المقررات الدراسية وإدارتها، وتحميل ملفات المحاضرات بالصيغ المدعومة (PDF، DOCX، PPTX)، وإنشاء ملخصات نصية منظمة تلقائيًا باستخدام تقنيات الذكاء الاصطناعي. من خلال تحليل محتوى المحاضرات، يُنتج النظام ملخصات موجزة ومنظمة يمكن للطلاب حفظها ومراجعتها وإدارتها لاستخدامها لاحقًا.

من خلال دمج تقنيات الويب الحديثة وآليات المصادقة الآمنة ومعالجة النصوص القائمة على الذكاء الاصطناعي، يعزز النظام المقترح تجربة التعلم من خلال تقليل وقت المراجعة وتحسين إمكانية الوصول إلى المحتوى وتوفير أساس قابل للتطوير للميزات التعليمية المستقبلية.

Index of content

SUPERVISION CERTIFICATION	II
Acknowledgements	III
ABSTRACT	IV
ملخص.....	V
Index of content	VI
Index of tables.....	X
Index of figures.....	XII
List of abbreviations.....	XIV
Chapter 1 Introduction	15
1.1 Introduction	15
1.2 Project Overview	15
1.3 Problem Definition.....	15
1.4 Project objectives	16
1.5 Project Scope, Constraints, and Assumptions	17
1.6 Research Contribution.....	17
1.7 Research Beneficiaries.....	18
1.8 Research Methodology and Scientific Methods Used	18
1.9 Report organization:	19
1.10 Basic Definitions and Concepts	19
Chapter 2 Basic Concepts and Literature Review	15
2.1 Introduction	22
2.2 Basic concepts	22
2.3 Literature review	24
2.4 Comparative Analysis of AI-Based Educational Platforms.....	27
2.5 Embedding Models Evaluation for Retrieval-Augmented Generation	28
2.6 Review of Selected Research Papers	30
Chapter 3 Project Management.....	22
3.1 Introduction	30
3.2 Project charter	30

3.3 The SOW document	31
3.4 Project plan – Gantt chart	34
3.5 Risk Management	34
3.6 Version Control and Configuration Management	36
Chapter 4 System Analysis	46
4.1 Introduction	47
4.2 Requirements Elicitation	47
4.3 SRS Document	49
Chapter 5 System Design	92
5.1 Introduction	93
5.2 System Architecture	93
5.3 System decomposition	94
5.4 System Architecture	97
5.5 Used design patterns.....	98
5.6 Database Design	100
5.7 Updating the RTM.....	102
Chapter 6 Account & User Service	104
6.1 Account & User Service Overview	105
6.2 Purpose of the Account & User Service.....	105
6.3 Responsibilities of the Account & User Service	106
6.4 Functional Requirements	106
6.5 API Endpoints	107
6.6 Security Considerations	108
6.7 Technology Stack	108
6.8 Service screenshots.....	109
6.9 Conclusion	112
Chapter 7 Course & Lecture Service	113
7.1 Course & Lecture Service Overview	114
7.2 Purpose of the Course & Lecture Service	114
7.3 Responsibilities of the Course & Lecture Service	115
7.4 Functional Requirements	115
7.5 API Endpoints	116
7.6 Security Considerations	117
7.7 Technology Stack	117

7.8 Service screenshots.....	118
7.9 Conclusion	122
Chapter 8 AI Service.....	123
8.1 Introduction	124
8.2 Basic Concepts and literature review.....	125
8.3 Services Design and implementation.....	129
Chapter 9 Conclusion	140
9.1 Introduction	141
9.2 Lessons Learnt.....	141
9.3 Future Recommendations.....	142
9.4 Appendices	142
9.5 References	143

Index of tables

Table 1 List of abbreviations.....	XIV
Table 2 Basic Terms and Definitions	19
Table 3 Comparative analysis of AI-powered educational platforms	28
Table 4 Risk Management.....	34
Table 5 Requirement database	47
Table 6 Use case specification for signup	56
Table 7 Use case specification for login	57
Table 8 Use case specification for account management	58
Table 9 Use case specification for generate quiz	59
Table 10 Use case specification for upload & parse lecture	61
Table 11 edit lecture info	65
Table 12 search lecture content	66
Table 13 Use case specification for delete lecture	67
Table 14 Use case specification for create course	68
Table 15 Use case specification for delete course	68
Table 16 Use case specification for edit course.....	69
Table 17 Use case specification for view lecture	69
Table 18 Use case specification for analyze uploaded file	70
Table 19 Use case specification for generate summary using AI	71
Table 20 Use case specification for export summary as PDF.....	72
Table 21 Test case specification for Signup.	76
Table 22 Test case specification for Login	76
Table 23 Test case specification for Manage Account.....	77
Table 24 Test case specification for Generate quiz.....	78
Table 25 Test case specification for Auto evaluate answers	80
Table 26 Test case specification for Analyze Uploaded File	81
Table 27 Test case specification for Generate Summary using AI	82
Table 28 Test case specification for Export Summary as pdf.....	82
Table 29 Test case specification fo Create Course	83
Table 30 Test case specification for Edit Course Info	84
Table 31 Test case specification for Delete Course.....	84
Table 32 Test case specification for View Lecture	85
Table 33 Test case specification for Edit lecture info	85
Table 34 Test case specification for Delete lecture.....	86
Table 35 Test case specification for Search lecture content	87
Table 36 Test case specification for Upload Lecture	88
Table 37 Test case specification for Parse Lecture File.....	89
Table 38 Initial RTM	90
Table 39 Updated RTM.....	102
Table 40 Main API Endpoints.....	107
Table 41 Main API Endpoints.....	116

Table 42 Model Comparison for Lecture Summarization	128
Table 43 Summarization Service API Endpoints	131
Table 44 Chatbot Module Service API Endpoints	133
Table 45 Quiz Generator Service API Endpoints	135
Table 46 Transcription Service API Endpoints	137

Index of figures

Figure 1 StudyFetch interface showcasing AI-powered study tools.	25
Figure2 Humata interface showcasing AI-powered study tools.	26
Figure3 Genei interface showcasing AI-powered study tools.	27
Figure4 Gantt chart.	34
Figure 5 Backend GitHub repository for the Moein project.	37
Figure 6 frontend GitHub repository for the Moein project.	38
Figure 7 Branching and merging workflow from feature development to production release.	41
Figure 8 Pull request used to merge the summary service feature branch into the development branch.	41
Figure 9 Git tags used to mark stable backend releases, highlighting versions v1.1 and v1.2.	43
Figure 10 GitHub release illustrating stable system version v1.1.	44
Figure 11 GitHub release illustrating stable system version v1.2.	44
Figure 12 Use case diagram.	55
Figure13 Sequence diagram for signup.	56
Figure14 Sequence diagram for login.	57
Figure15 Sequence diagram for accountmanagement.	58
Figure16 Sequence diagram for generate quiz.	60
Figure17 Sequence diagram for upload lecture.	63
Figure18 Sequence diagram for parse lecture.	64
Figure19 Sequence diagram for search lecture content.	66
Figure20 Sequence diagram for analys uploaded file, generate summary using AI & export summary as PDF.	73
Figure21 class diagram.	74
Figure22 Entity Relationship Diagram (ERD).	75
Figure 23 Component diagram.	94
Figure 24 System architecture.	97
Figure 25 Facade pattern applied using the API Gateway.	99
Figure 26 Template Method pattern for lecture file parsing.	100
Figure27 MongoDB document structure.	100
Figure 28 MySQL database modeling.	101
Figure29 Signup screenshot.	109
Figure30 Login screenshot.	109
Figure31 Edit account screenshot.	110
Figure32 Delete account screenshot.	110
Figure33 user info screenshot.	111
Figure34 side bar screenshot.	111
Figure35 Courses list screenshot.	118
Figure36 Create course screenshot.	118
Figure37 Edit course info screenshot.	119

Figure38 Delete course screenshot	119
Figure39 upload lecture screenshot	120
Figure40 Lectures list screenshot.....	120
Figure41 Edit lecture name screenshot.....	121
Figure42 Delete lecture screenshot.....	121
Figure 43 Summarization Service Flow	130
Figure44 Generated summary screenshot	131
Figure 45 Chatbot Module Service Flow.....	132
Figure 46 Transcription Service Flow	136
Figure 47 Notification service flow	139

List of abbreviations

Table 1 List of abbreviations

Term	Definition
AI	Artificial Intelligence
API	Application Programming Interface
DRF	Django REST Framework
JWT	JSON Web Token
NLP	Natural Language Processing
ML	Machine Learning
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
STT	Speech-to-Text
OTP	One-Time Password
SQL	Structured Query Language
NoSQL	Not Only SQL
DB	Database
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
HTTP	Hypertext Transfer Protocol
REST	Representational State Transfer
JSON	JavaScript Object Notation
PDF	Portable Document Format
DOCX	Microsoft Word Open XML Document
PPTX	Microsoft PowerPoint Open XML Presentation
OCR	Optical Character Recognition
ERD	Entity Relationship Diagram
UML	Unified Modeling Language
SRS	Software Requirements Specification
SOW	Statement of Work
RTM	Requirements Traceability Matrix
MCQ	Multiple Choice Question
URL	Uniform Resource Locator
HTTPS	Hypertext Transfer Protocol Secure
CORS	Cross-Origin Resource Sharing
MQ	Message Queue

Chapter 1 Introduction

1.1 Introduction

In this chapter we introduce the Student Course Management & AI Summarization System, describe the motivating problems that led to its development, and state the main objectives we intend to achieve. The system addresses the increasing volume of course materials and the time students spend reading and reviewing lectures. By combining course and lecture management with automated AI-based text summarization and assessment, the proposed system reduces revision time and improves learning effectiveness.

We will also give an overview of the report structure so the reader can easily follow the subsequent analysis, design, and implementation chapters.

1.2 Project Overview

Moein is an AI-powered student assistance system that helps students organize and study lecture materials more efficiently. It allows students to manage courses, upload lectures, generate summaries and quizzes, transcribe audio, receive notifications, and ask questions through an AI Chatbot.

1.3 Problem Definition

University students often face large quantities of lecture materials (slides, handouts, notes) across many courses. Manually organizing, reading, and extracting key ideas from these documents is time-consuming and inefficient. Students commonly struggle to:

- Quickly grasp the core concepts of long lecture documents.

- Find and revisit essential points across multiple courses and lecture files.

- Self-assess understanding with rapid, relevant practice questions derived from course content.

From a system-administration perspective, institutions and course owners also need a reliable way to host and manage course content, control access, and support future enhancements (scalability, security, maintainability).

Existing learning management workflows may lack integrated, automated support for summarization and formative assessment. Without automated summarization and question generation, students spend excessive time on review; instructors and admins lack tools for scalable content processing. Furthermore, manual processing is inconsistent and does not scale when course material volume grows.

Therefore, there is a clear need for a robust web-based system that simultaneously:
Provides course and lecture management (create/delete courses, upload lecture files).
Automatically analyzes uploaded lecture files (PDF, DOCX, PPTX) and produces concise, structured text summaries.
Generates multiple-choice questions from lecture text and automatically grades student attempts.
Secures user accounts and course data, and supports a maintainable, scalable architecture for future extensions.
Such a system will enhance student learning efficiency, improve content accessibility, and offer a practical platform demonstrating modern web and AI techniques.

1.4 Project objectives

The principal objective of this project is to design and implement a secure, scalable, and user-friendly web application that helps students organize course materials and study more effectively through AI-generated text summaries and automated multiple-choice assessment.

To fulfil this objective, the system will:

Allow users (Students and Admin) to register, authenticate, and manage accounts using JWT-based security.

Enable Students to create courses and upload lecture files in supported formats (PDF, DOCX and PPTX).

Process uploaded lecture files asynchronously using NLP models (via Hugging Face) to generate concise, well-structured text summaries (headings, bullet points, key concepts).

Persist original files and generated summaries for fast retrieval and searching.
Automatically generate multiple-choice questions from lecture content (MCQs only) and perform automatic grading after student submission, providing immediate feedback.

Use microservices architecture with an API Gateway to centralize authentication and route requests, improving modularity and scalability.

Employ the chosen technology stack (React + Vite frontend, Python/Django backend services, MongoDB for document/AI artifacts, MySQL for relational data, Docker for containerized deployment, and GitHub for source/version control).

Provide a foundation for future features: full-text search, PDF export, sharing summaries, and support for additional media types (audio/video) via speech-to-text.

These objectives aim to reduce students' revision time, increase comprehension and retention, and demonstrate a production-oriented integration of web engineering practices with AI-driven text processing.

1.5 Project Scope, Constraints, and Assumptions

The scope of Moein includes the main features required to support students in managing and studying their academic materials. The system allows students to create accounts, manage courses, upload lecture files, generate AI-based summaries, create quizzes, transcribe audio lectures, receive notifications, and use a chatbot to ask questions based on uploaded content.

The project also has some constraints. Moein depends on a stable internet connection, supported file formats, available AI models, and processing time for long-running tasks such as summarization, quiz generation, and transcription. Large files or unclear lecture content may affect the quality or speed of the generated results.

The system assumes that users access the platform through a modern web browser, upload valid academic materials, and have permission to use the uploaded content. It also assumes that backend services, databases, RabbitMQ, and AI services are available during system operation.

1.6 Research Contribution

The main research contribution of Moein is providing an integrated AI-powered educational assistant that combines course management, lecture upload, automatic summarization, quiz generation, and chatbot support in one system.

Unlike many existing tools that mainly focus on English document processing or general study assistance, Moein targets Arabic-speaking students and local educational workflows. It helps students transform uploaded lecture materials into structured summaries, questions, and learning support outputs.

Moein also contributes a scalable microservices-based architecture that separates account management, course and lecture management, AI processing, and API Gateway routing. This makes the system easier to maintain, extend, and improve in future versions.

Overall, Moein provides a practical foundation for future AI-assisted learning features such as semantic search, retrieval-based question answering, speech-to-text, and personalized study support.

1.7 Research Beneficiaries

The main beneficiaries of this research are university students, especially those who need support in organizing lecture materials, understanding academic content, and reviewing efficiently using AI-generated summaries and quizzes.

Instructors and academic institutions can also benefit from the system by using it as a supportive educational tool that improves access to learning materials and encourages self-study.

In addition, future researchers and developers in educational technology can benefit from Moein as a practical model for building AI-assisted learning systems using microservices, NLP, and modern web technologies.

1.8 Research Methodology and Scientific Methods Used

The research methodology followed in Moein is based on analyzing the educational problem, studying similar AI-based learning systems, collecting system requirements, and designing a suitable software architecture. The project applies software engineering methods such as requirements analysis, UML modeling, system decomposition, and service-based design.

The scientific methods used include studying related platforms, comparing AI models, testing different system workflows, and evaluating the system through functional test cases. The project also follows an incremental development approach, where each service is designed, implemented, tested, and integrated gradually.

This methodology helped ensure that Moein was developed in an organized way, with clear requirements, structured design, reliable implementation, and practical AI-based features that support student learning.

1.9 Report organization:

The report will be organized in the following chapters:

- Chapter 1: introduction.
- Chapter 2: Basic concepts and literature review.
- Chapter 3: project management.
- Chapter 4: high level System analysis and Design
- Chapter 5: Account Service
- Chapter 6: Course and Lecture Service
- Chapter 7: AI Service
- Chapter 8: Conclusion.

1.10 Basic Definitions and Concepts

Table 2 Basic Terms and Definitions

Term	Definition
Artificial Intelligence (AI)	Technology that enables systems to perform tasks such as summarization, question answering, and quiz generation.
API	A method that allows software services to communicate with each other.
API Gateway	A central entry point that routes frontend requests to the correct backend service.
Microservices	An architecture where the system is divided into independent services with separate responsibilities.
JWT	A secure token used to authenticate users and protect API access.
NLP	AI techniques used to process and understand human language.
LLM	A large AI model used to understand and generate text.

RAG	A technique that retrieves relevant content before generating an answer.
Embeddings	Numeric representations of text used for semantic search.
Vector Database	A database used to store and search embeddings.
Milvus	The vector database used in Moein for semantic retrieval.
MongoDB	A NoSQL database used to store AI outputs and flexible documents.
MySQL	A relational database used to store structured system data.
RabbitMQ	A message broker used for asynchronous processing between services.
Summarization	The process of producing a shorter version of lecture content.
Quiz Generation	The process of creating questions from lecture materials.
Speech-to-Text (STT)	Technology that converts audio lectures into written text.
OTP	A temporary code used to verify user identity.

Chapter 2 Basic Concepts and Literature Review

2.1 Introduction

In this chapter we present the technical and theoretical building blocks used in the **Student Course Management & AI Summarization System**, and survey related systems and research that informed our design choices. The goal is to give the reader the background necessary to understand why particular algorithms, architectures, and technologies were chosen, and how past work in learning platforms, text summarization, and question generation relates to this project.

2.2 Basic concepts

Maintenance in Software Engineering:

Maintenance in software engineering is essential to keep the system functional, secure, and efficient over time. For the Student Course Management & AI Summarization System, maintenance includes updating AI models, fixing bugs in upload/extraction pipelines, adapting to new file-format edge cases, and improving service interfaces. Regular maintenance prolongs system life, ensures reliable summaries and assessments, and preserves user trust.

- Requirements engineering:

Requirements engineering is the disciplined process of eliciting, documenting, and maintaining system requirements. For this project, it involves gathering needs from students and admins (features like course creation, file upload, summary generation, and automatic MCQ grading) through interviews, surveys, and prototyping. Clear elicitation ensures the delivered application matches user expectations and supports future extension (search, export, multimedia).

- Student feedback and usage telemetry:

Student feedback and usage telemetry provide direct insight into how the system is used and where it fails to meet learning needs. Collecting feedback (ratings on summaries, report of poor-quality MCQs) and telemetry (upload patterns, processing times) helps prioritize improvements, tune summarization models, and enhance UX for faster revision and higher satisfaction.

- Text summarization (extractive vs. abstractive):

Text summarization creates a concise representation of source content. Extractive summarization selects and concatenates key sentences from the lecture, preserving

factual phrasing; abstractive summarization generates new, concise text that may rephrase concepts for clarity. The system targets high-quality, text-only summaries that balance faithfulness and readability for student review.

- Question generation and automatic grading (MCQs):

Question generation produces multiple-choice questions from lecture text, including the correct answer and plausible distractors. Limiting output to MCQs simplifies automatic grading: once the student submits answers, the system compares them to stored keys and returns an immediate score and feedback, supporting rapid formative assessment.

- Semantic similarity and embeddings:

Semantic similarity measures how close two pieces of text are in meaning. Embedding models convert sentences or paragraphs into vectors so the system can detect duplicate points, cluster related concepts, and select distractors for MCQs. Semantic techniques improve summary coherence and reduce redundancy across generated outputs.

- File formats and text extraction:

Lecture materials are accepted in PDF, DOCX, and PPTX formats. Reliable text extraction is a prerequisite: parsing slide text, body paragraphs, and handling layout artifacts ensures the NLP pipeline receives clean input. Robust extraction also prepares the system for future OCR or multimedia processing.

- Asynchronous processing and job queues:

Summarization and question generation can be time-consuming; therefore, processing runs asynchronously. The system enqueues jobs when a student uploads a lecture, processes them in background workers, and stores results for later retrieval. This pattern preserves frontend responsiveness and enables scalable AI workloads.

- Storage choices: MongoDB vs. MySQL:

A hybrid storage strategy fits the project needs: MySQL stores relational data (users, courses, enrollments), while MongoDB holds variable AI artifacts (extracted text, summaries, generated MCQs, embeddings). This separation optimizes query performance for structured operations and flexible retrieval of document-like AI outputs.

2.3 Literature review

StudyFetch

StudyFetch is one of the prominent educational platforms that utilizes artificial intelligence to enhance the learning experience for university students. The platform aims to convert study materials into interactive tools such as flashcards, quizzes, and summaries, helping improve comprehension and reduce study time.

Core Features:

- Converts study files (PDFs, presentations, notes) into automatic study tools.
- Summarizes lectures using AI techniques.
- Includes a smart assistant named **Spark.E** that provides instant answers, essay reviews, and progress tracking.
- Supports audio and video content by allowing lecture recordings to be transformed into study-ready summaries.

Relevance to the Project:

StudyFetch demonstrates how AI technologies can be used to improve learning, especially through automatic summarization and personalized study tools. However,

this project aims to offer a more tailored experience for Arabic-speaking students by supporting Arabic audio content and providing a simplified interface suited to local educational environments.

Research Gap:

Although StudyFetch supports Arabic in its interface and some content, its core AI features—such as summarization and the Spark.E assistant—are still primarily designed for English-language input, which limits their effectiveness in Arabic educational contexts. This project seeks to bridge that gap by developing a smart system specifically for Arabic-speaking students, with expanded support for Arabic audio and text content, and a fully integrated, user-friendly learning experience.

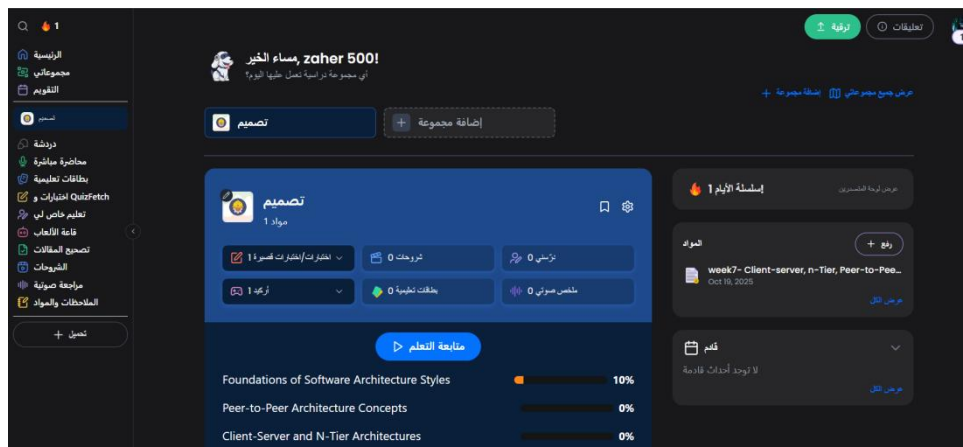


Figure 1 StudyFetch interface showcasing AI-powered study tools.

Humata AI

Humata AI is an advanced document assistant powered by artificial intelligence, designed to help users interact with and extract insights from complex documents—especially PDFs. It enables real-time question answering, summarization, and comparison across multiple files, making it a valuable tool for researchers, students, and professionals dealing with technical or lengthy content.

Core Features:

- **AI-powered PDF interaction:** Users can upload unlimited documents and ask questions directly to the system.
- **Summarization and comparison:** The platform can summarize findings, compare documents, and extract key insights.
- **Citation-based answers:** Responses include citations linked to the original source files for transparency.
- **Security and scalability:** Offers encrypted storage, role-based access, and single sign-on for enterprise use.

Relevance to the Project:

Humata AI showcases how artificial intelligence can streamline document analysis and enhance productivity. Its ability to summarize and answer questions from uploaded files aligns with the goals of this project, especially in automating content extraction. However, Humata primarily focuses on written documents, while this project emphasizes **audio-based summarization** and **Arabic language support**, which are not core strengths of Humata.

Research Gap:

Although Humata AI excels in processing technical documents and offers multilingual interface support, its core AI functionalities are optimized for English-language text and PDF formats. It does not natively support Arabic audio input or educational workflows tailored to Arabic-speaking students. This project addresses that gap by developing a system that supports **Arabic audio summarization**, **student-focused interaction**, and **localized educational content**, offering a more inclusive and accessible solution.

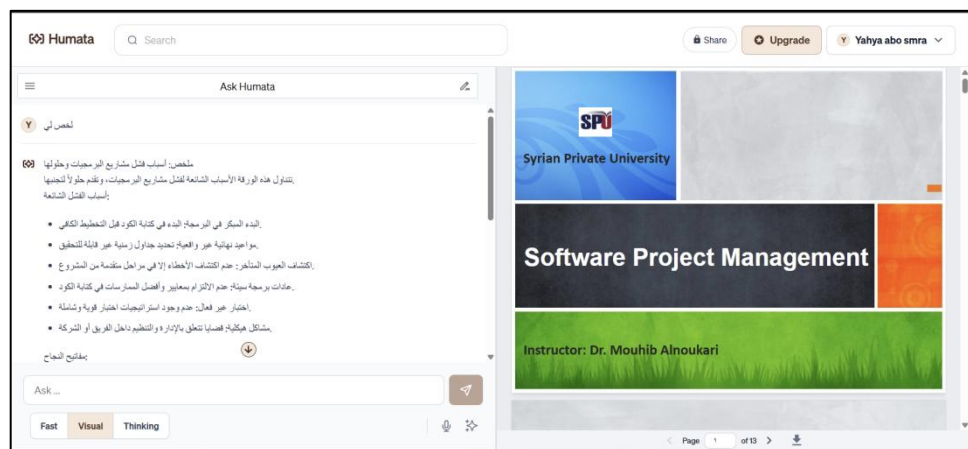


Figure2 Humata interface showcasing AI-powered study tools.

Genei

Genei is an AI research tool that helps students and researchers summarize articles, PDFs, and web pages. It generates concise summaries, extracts keywords, and organizes notes to speed up academic work.

Core Features:

- Summarizes long documents automatically
- Highlights key concepts
- Central workspace for notes and files
- Improves reading efficiency

Relevance to Moein:

Genei streamlines academic reading, but it focuses on English text and static documents. Moein, by contrast, supports Arabic lecture summarization and offers tools tailored to Arabic-speaking students.

Research Gap:

Genei lacks Arabic language support and audio interaction. Moein fills this gap with Arabic audio summarization, academic structuring, and a localized student assistant.

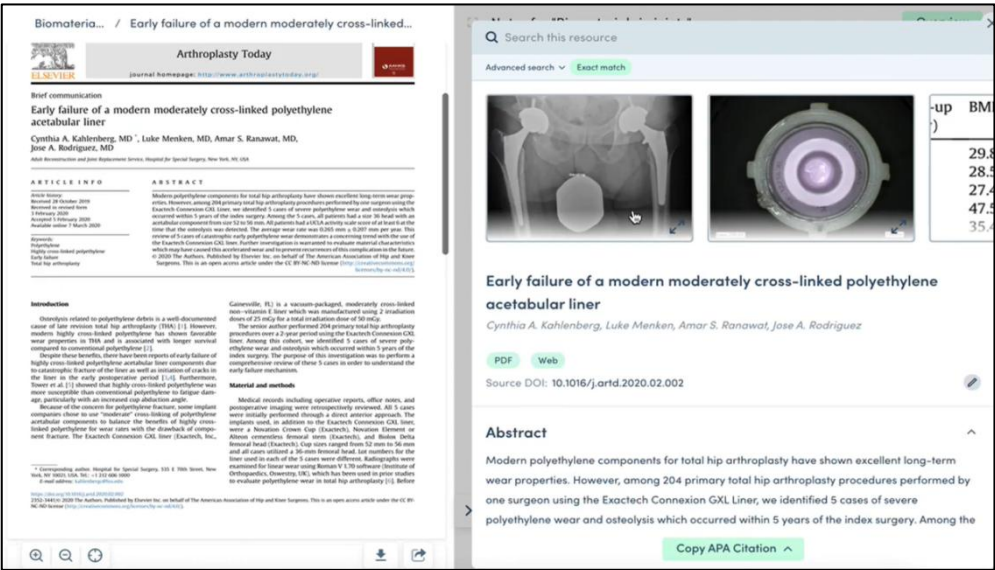


Figure3 Genei interface showcasing AI-powered study tools.

2.4 Comparative Analysis of AI-Based Educational Platforms

To highlight the unique value of **Moein**, the proposed system, the following table presents a comparative analysis between three existing AI-powered platforms — **StudyFetch**, **Humata AI**, and **AskYourPDF**— and **Moein**. The comparison focuses on key features relevant to Arabic-speaking students and educational workflows.

Table 3 Comparative analysis of AI-powered educational platforms

Feature	StudyFetch	Humata AI	Genei	Moein (Proposed System)
Text summarization	✓	✓	✓	✓
speech to text	✓	✗	✗	✓
Arabic interface support	✓	✗	✗	✓
AI support for Arabic content	✓	✓	✗	✓
Auto-generated quizzes	✓	✗	✗	✓
Simplified UI for local educational context	✗	✓	✓	✓
Responses include citations linked to the original	✗	✓	✓	✓
Student-focused smart assistant (Chat Bot)	✓	✓	✗	✓
Document and content library management	✓	✓	✓	✓

2.5 Embedding Models Evaluation for Retrieval-Augmented

Generation

2.5.1 Overview

In Retrieval-Augmented Generation (RAG) systems, the retrieval component plays a critical role in grounding generated outputs in relevant source documents. This work evaluates multiple sentence embedding models to identify the most suitable one for a **multilingual educational RAG system**, where both English and Arabic lecture content must be handled effectively.

The evaluation focuses on the ability of each model to **discriminate between relevant and non-relevant passages**, rather than on absolute similarity scores alone.

2.5.2 Evaluated Models

The following embedding models were considered due to their widespread use in modern RAG pipelines:

- **E5-large**: A dense embedding model trained using a query–passage contrastive objective.
- **bge-m3**: A multilingual embedding model designed to support dense retrieval, sparse retrieval, and reranking within a single architecture.
- **LaBSE**: A language-agnostic sentence embedding model optimized for cross-lingual semantic similarity.

2.5.3 Experimental Setup

To ensure a fair comparison, all models were evaluated using the same sentence similarity task on Hugging Face.

Queries

- **English:**
explain software design patterns
- **Arabic:**
اشرح أنماط التصميم البرمجية

Passages

For each language, two passages were used:

1. A **relevant passage** describing software design patterns.
2. A **non-relevant passage** discussing REST APIs.

2.5.4 Experimental Results

Model	Language	Relevant Passage	Non-Relevant Passage	Separation Gap
E5-large	English	0.880	0.766	0.114
E5-large	Arabic	0.846	0.831	0.015
bge-m3	English	0.678	0.497	0.181
bge-m3	Arabic	0.672	0.552	0.120
LaBSE	English	0.515	0.384	0.131
LaBSE	Arabic	0.556	0.395	0.161

2.5.5 Model Selection

Based on the experimental evaluation, **bge-m3** was selected as the embedding model for the Moein RAG system.

Although **E5-large** showed strong performance in English, it demonstrated weak discrimination in Arabic. **LaBSE** provided stable multilingual results but lower retrieval specialization. In contrast, **bge-m3** achieved balanced and consistent separation between relevant and non-relevant passages in both English and Arabic.

Additionally, **bge-m3** is explicitly optimized for retrieval tasks and supports dense retrieval, sparse retrieval, and reranking within a single architecture. This makes it well-suited for a multilingual educational RAG.

2.6 Review of Selected Research Papers

Table 4 A summary of selected research papers on AI-based educational systems and Retrieval-Augmented Generation (RAG) approaches.

Paper ID	Paper Title	Publisher	Year	Used Methodology	Dataset	Findings
LR-P-01	Retrieval-Augmented Generation for Educational Application: A Systematic Survey	ScienceDirect	2025	Systematic literature review analyzing 51 studies on RAG in education. The paper examines the RAG pipeline (indexing, retrieval, generation) and reviews retrieval techniques such as sparse (BM25) and dense retrieval (DPR).	-	RAG improves factual accuracy and reliability of LLM-based educational systems. It is widely used in applications such as educational chatbots and intelligent tutoring systems, but challenges remain including hallucinations, outdated knowledge bases, and computational cost.
LR-P-02	A Survey on	ACM	2023	Comprehensive	-	The study shows that

	RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models			survey analyzing architectures that integrate retrieval mechanisms with large language models. The paper categorizes RAG systems, reviews retriever-generator interaction methods, and discusses retrieval techniques, training strategies, and evaluation metrics for knowledge-grounded generation.		integrating retrieval with LLMs significantly improves factual accuracy and reduces hallucinations. It highlights key challenges such as retrieval quality, system latency, and knowledge updating, and identifies future research directions including adaptive and agent-based RAG systems.
--	---	--	--	---	--	---

Chapter 3 Project Management

3.1 Introduction

In this chapter, we will dive into the management phase of the project, which is a critical aspect of ensuring the project's success. We will examine the project charter, project plan, Statement of Work (SOW) document, stakeholder analysis, and risk management strategies to effectively guide and control the project from initiation to completion.

3.2 Project charter

A project charter is a formal document that serves as an official authorization for the start of a project. It acts as a reference point throughout the project, providing a clear understanding of the project's purpose and establishing a foundation for decision-making and project governance.

<i>Project title</i>	<i>AI Student Assistant</i>
<i>Project start date</i>	<i>21 – 2 – 2026</i>
<i>Project finish date</i>	<i>3 – 6 - 2026</i>
<i>Project manager</i>	<i>Dr. Riad Sonbol & Eng. Raghad Al Hossny</i>
<i>Project objectives</i>	<i>Develop a web-based educational system that assists students in managing their university courses and lecture materials in an efficient and organized manner that supports effective learning. By using artificial intelligence algorithms and natural language processing techniques to analyze uploaded lecture files (PDF, DOCX, PPTX), the system automatically generates concise text summaries and multiple-choice questions to help students understand and review course content.</i> <i>The system aims to reduce the time and effort required for manual revision by providing structured summaries and instant assessment through automatic question generation and grading. Through secure user authentication, course management features, and intelligent text processing, the system enhances the overall learning experience and provides a scalable foundation for future improvements and extensions.</i>
<i>Approach</i>	<i>1. Determining the scope of the project.</i> <i>2. Perform literature review for similar tools or software.</i> <i>3. perform analysis for the requirements of the project.</i>

Roles and responsibilities:

4. *perform system decomposition and design.*
5. *Start the first increment and develop a subsystem that is responsible for account and project management functionalities.*
6. *Testing the functionalities of the first subsystem*
7. *Start the second increment and develop the AI components that are responsible for the reviews analysis process.*
8. *Testing the functionalities of the other subsystem.*
9. *Document every step of the work.*

<i>Name</i>	<i>Role</i>	<i>Responsibility</i>
<i>Dr.Riad Sonbol</i>	<i>Supervisor</i>	<i>Highly Project management and Work monitoring</i>
<i>Eng.Raghad Al Hossny</i>	<i>Supervisor</i>	<i>Work monitoring</i>
<i>Zaher Kraym</i>	<i>Engineer</i>	<i>Full-Stack development</i>
<i>yahia Abou Samra</i>	<i>Engineer</i>	<i>Backend development</i>
<i>Awab Kheir</i>	<i>Engineer</i>	<i>Backend development</i>
<i>Othman Darkal</i>	<i>Engineer</i>	<i>Backend development</i>

3.3 The SOW document

A Statement of Work (SOW) is a comprehensive document that defines the scope of work for a project. It outlines the specific tasks, deliverables, timeline, and responsibilities. The SOW document provides a clear understanding of what needs to be accomplished, the project's objectives, and the criteria for success.

<i>Project Title:</i>	<i>Moein – AI-Powered Student</i>
<i>Project Description and Objectives</i>	The Moein project aims to develop a web-based AI-powered educational system that helps university students manage their courses and lecture materials more efficiently. The system uses artificial intelligence to generate lecture summaries, create quizzes, transcribe audio lectures, and provide chatbot-based academic assistance. The main objective is to deliver a secure, scalable, and maintainable student assistance platform based on a microservices architecture.
<i>Project Scope</i>	This project involves developing a full-stack software system that

	integrates backend microservices, a frontend web application, databases, message queues, and AI-based services. The system enables students to create accounts, manage personal information, create and manage courses, upload lecture files, generate AI-based summaries and quizzes, transcribe audio content, receive notifications, and interact with an AI chatbot based on uploaded academic materials. The architecture follows a microservices approach with an API Gateway to improve scalability, maintainability, security, and communication between services.
Project Goals	Develop a scalable backend system using Django REST Framework and microservices architecture. Implement secure user authentication and account management. Enable efficient management of courses, lectures, and uploaded materials. Integrate AI services for lecture summarization, quiz generation, chatbot support, and speech-to-text transcription. Use databases suitable for different data types, including MySQL, MongoDB, and Milvus. Apply software engineering practices such as version control, configuration management, testing, documentation, and incremental development.
Project Deliverables	Project plan. Working web-based software system. Backend microservices. Frontend web application. AI services integration. Final project report.
Technology and Tools	Programming Languages: Python, JavaScript, HTML, CSS. Frameworks: Django REST Framework for backend services, React for the frontend. Databases: MySQL, MongoDB, Milvus. AI Models / Services: Hugging Face models for summarization, quiz generation, chatbot support, embeddings, and speech-to-text processing.

	Tools: Git, GitHub, Docker, RabbitMQ, Ngrok.
Assumptions	Project team members will remain available throughout the development lifecycle. Regular supervision and feedback will be provided by the project supervisors. A stable development environment and internet connection will be available. AI models, external services, and required APIs will be available during development and testing. Users will access the system through modern web browsers and upload supported file formats.
Project Resources	<i>Human Resources:</i> Dr. Raid Sonbol - Project Manager Eng. Raghad al Hossny - Supervisor Zaher Krayem – Full-Stack & AI contributor Yahia Abo Samra – Backend & AI contributor Othman Darkal – Backend & AI contributor Awab Kheir – Backend & AI contributor
Schedules	Project Start Date: February 21, 2026 First Project Seminar: March 14, 2026 Second Project Seminar: April 25, 2026 Technical Committee Review: May 9, 2026 Final Project Seminar: June 2, 2026 Project Finish Date: June 2, 2026

3.4 Project plan – Gantt chart

A document outlining tasks, deadlines, and resources needed to achieve project objectives, serving as a roadmap for successful project execution.

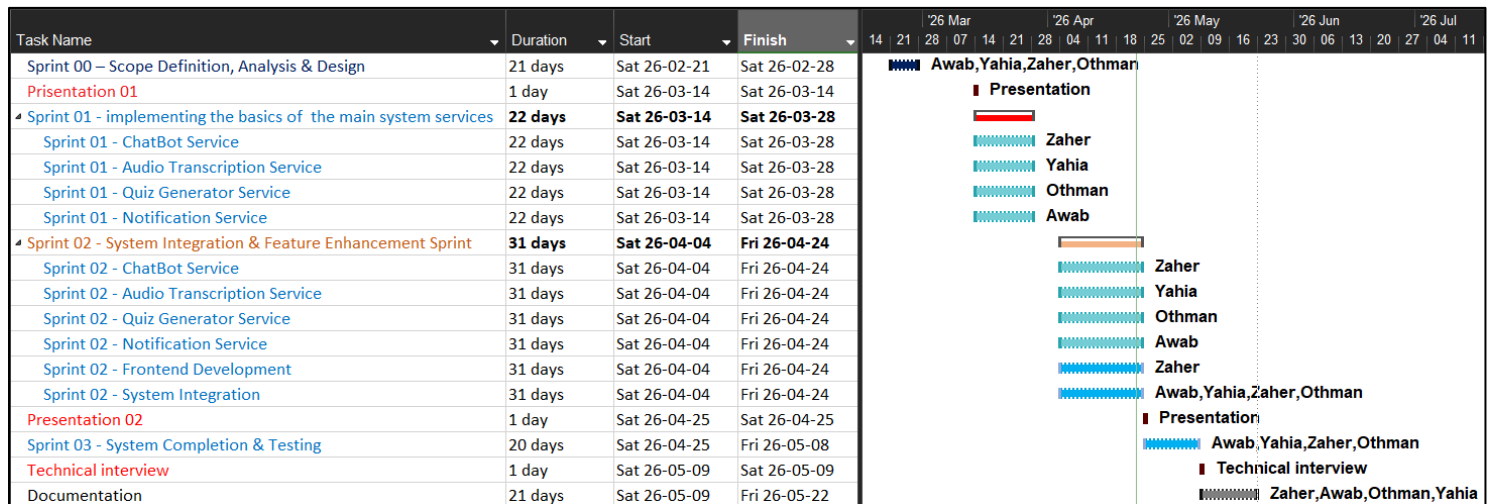


Figure4 Gantt chart

3.5 Risk Management

The process of identifying, evaluating, and mitigating potential risks that could impact the success of a project or a team.

Table 5 Risk Management

Risk ID	Risk Title	Risk Description	Impact	Probability	Mitigation Plan
RK-01	AI Model Unavailability	AI models or external AI services may become unavailable or slow during summarization, quiz generation, chatbot responses, or transcription.	High	Medium	Use fallback models when possible, handle timeout errors, store task status, and notify users when processing fails.
RK-02	Poor AI Output Quality	Generated summaries, quizzes, or chatbot answers	High	Medium	Improve prompts, test different models, validate outputs, and

		may be inaccurate, incomplete, or not well-structured.			allow future evaluation using user feedback.
RK-03	Microservices Integration Issues	Communication between services may fail due to incorrect URLs, headers, authentication tokens, or service downtime.	High	Medium	Use API Gateway logging, clear API documentation, consistent headers, service health checks, and error handling.
RK-04	Database Connection Failure	MySQL, MongoDB, or Milvus may become unavailable, causing failure in storing or retrieving system data.	High	Low	Configure database connections correctly, use retry mechanisms, backup important data, and monitor service availability.
RK-05	Message Queue Failure	RabbitMQ failure may stop background tasks such as summarization, quiz generation, transcription, or notifications.	High	Medium	Monitor RabbitMQ, persist task status, retry failed jobs, and separate long-running tasks from direct user requests.
RK-06	Large File Processing Delay	Large lecture files may take a long time to parse, summarize, or process.	Medium	High	Process files asynchronously, show processing status, limit maximum file size, and optimize text extraction.
RK-07	Unsupported or Corrupted Files	Students may upload unsupported, empty, scanned, or corrupted lecture files.	Medium	Medium	Validate file type and size, return clear error messages, and support only reliable formats such as PDF, DOCX, and PPTX.
RK-08	Security and Unauthorized	Unauthorized users may try to access	High	Medium	Apply JWT authentication,

	Access	private courses, lectures, summaries, or quizzes.			access control checks, secure password handling, and authorization validation in each service.
RK-09	Team Coordination Risk	Since the project is divided into several services, poor coordination may cause duplicated work or integration delays.	Medium	Medium	Use GitHub branches, pull requests, regular meetings, clear task distribution, and shared API documentation.
RK-10	Deployment and Environment Issues	Differences between local and deployment environments may cause service errors, CORS issues, or configuration problems.	Medium	Medium	Use environment variables, Docker when possible, document setup steps, and test services before final deployment.

3.6 Version Control and Configuration Management

3.6.1 Project Description

The Moein project is a full-stack AI-powered student assistance system developed using a microservices architecture. The backend services are implemented using Django REST Framework, while the frontend web application is developed using React. The main objective of the project is to provide a scalable, modular, and maintainable platform that helps students manage courses, upload lecture materials, generate AI-based summaries and quizzes, transcribe audio lectures, receive notifications, and interact with an intelligent chatbot.

The system uses an API Gateway as a centralized entry point for routing requests between the frontend and backend microservices. Each service is responsible for a specific function, such as account management, course and lecture management, summarization, quiz generation, chatbot support, transcription, and notifications. This architecture improves separation of concerns, supports independent development, and allows future extension of AI features and asynchronous processing using tools such as RabbitMQ.

3.6.2 Git Repository Structure

Version control for the project is managed using Git and hosted on GitHub. The backend source code is maintained in a dedicated repository that contains all backend microservices.

3.6.2.1 Backend Repository:

- **Repository Name:** Moein-Senior1
- **Purpose:** Contains the backend implementation of the Moein system, including all backend microservices, API Gateway routing, database integrations, authentication logic, AI-service communication, and asynchronous processing support.
- **Structure:** The repository is organized using a microservices architecture, where each service is kept in a separate module or project. This structure allows services such as Account & User Service, Course & Lecture Service, Chatbot Service, Quiz Generator Service, Summarization Service, Transcription Service, Notification Service, and API Gateway to be developed, tested, and maintained independently.
- **Benefit:** This separation improves scalability, maintainability, and flexibility, and makes it easier to extend the system with new AI-based educational features in future versions.

This separation improves maintainability and supports incremental system growth.

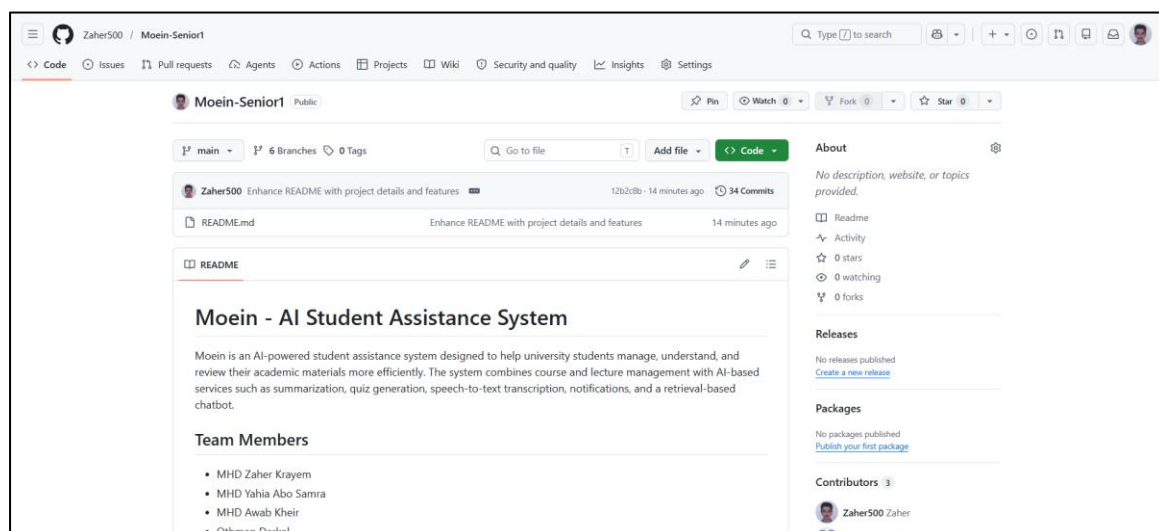


Figure 5 Backend GitHub repository for the Moein project.

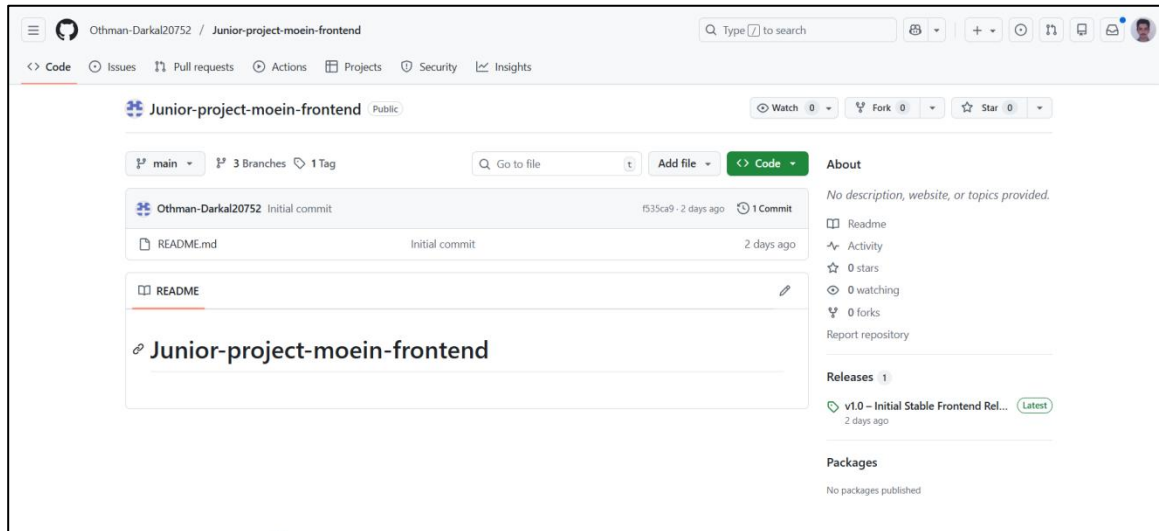


Figure 6 frontend GitHub repository for the Moein project.

3.6.2.2 Frontend Repository:

- **Repository Name:** Junior-project-moein-frontend
- **Purpose:** Contains the complete frontend source code of the Moein platform, including UI components, pages, services, styles, and configuration files.
- The repository is structured to separate components, pages, services, and assets, improving code organization and maintainability.
- This separation from the backend enhances development flexibility and supports incremental system growth.

3.6.2.3 Frontend–Backend Integration

Although the frontend and backend are maintained in separate repositories, both follow a consistent version control strategy, including branch management, pull requests, and release tagging. This approach enables coordinated integration while preserving the ability to develop and release each part of the system independently.

3.6.3 Branching and Merging Strategy

A simplified branching strategy was adopted across both the backend and frontend to support agile and incremental development while maintaining consistency throughout the project.

The following branches are used in **both repositories**:

- **production:** Contains the stable and production-ready versions of the system.
- **development:** Used for ongoing development and integration of new features.
- **feature/*:** Temporary branches created for implementing specific features or services.
-

Branch Flow

- Feature branches are created from the `development` branch
- Completed features are merged back into `development` after testing
- Stable milestones are merged from `development` into `production`

For the **backend**, this strategy ensures that new microservices or backend enhancements do not affect the production-ready system until they are fully integrated and tested.

For the **frontend**, the same strategy ensures that user interface changes and new features (such as authentication interfaces or lecture workflows) are safely developed and validated before being released to production.

This unified branching strategy promotes consistency, controlled integration, and system stability across the entire project.

3.6.4 Team Members and Responsibilities

The project was developed by a team of four members. All team members collaboratively participated in the system analysis and architectural design phases, including requirements analysis, system modeling, and defining the overall system architecture.

The implementation responsibilities were divided as follows:

- **Backend Development:**

Responsible for implementing the backend microservices using Django REST Framework. This includes Account & User Service, Course & Lecture Service, API Gateway, Notification Service, Quiz Generator Service, Chatbot

Service, Transcription Service, database integration, authentication handling, inter-service communication, and backend version control management.

- **Frontend Development:**

Responsible for implementing the frontend web application using React. This includes designing user interface components, page layouts, routing, authentication screens, course and lecture pages, chatbot interface, quiz pages, transcription pages, notification UI, and integration with backend APIs through the API Gateway.

- **AI Service Development:**

Responsible for implementing and integrating the AI-based services used in the system. This includes lecture summarization, quiz generation, chatbot retrieval and response generation, speech-to-text transcription, embedding generation, vector database integration using Milvus, and communication between AI services and other backend components.

- **Integration and Testing:**

Responsible for connecting frontend and backend services, testing API workflows, verifying service communication, handling errors, and ensuring that the system works correctly across different modules before release.

All team members contributed to version control activities. Backend and frontend branches, pull requests, merges, tags, and releases were managed by the responsible developers. This ensured controlled integration, clear change tracking, and stable releases for the Moein system.

3.6.5 Development Workflow

The development workflow for both the backend and frontend follows an incremental and task-based approach to ensure consistency, stability, and controlled integration across the entire system.

The workflow is summarized as follows:

1. A development task or feature is identified
2. A feature branch is created from the `development` branch

3. The feature is implemented and tested locally
4. A pull request is created to merge the feature branch into `development`
5. After integration and validation, the `development` branch is merged into `production` for release

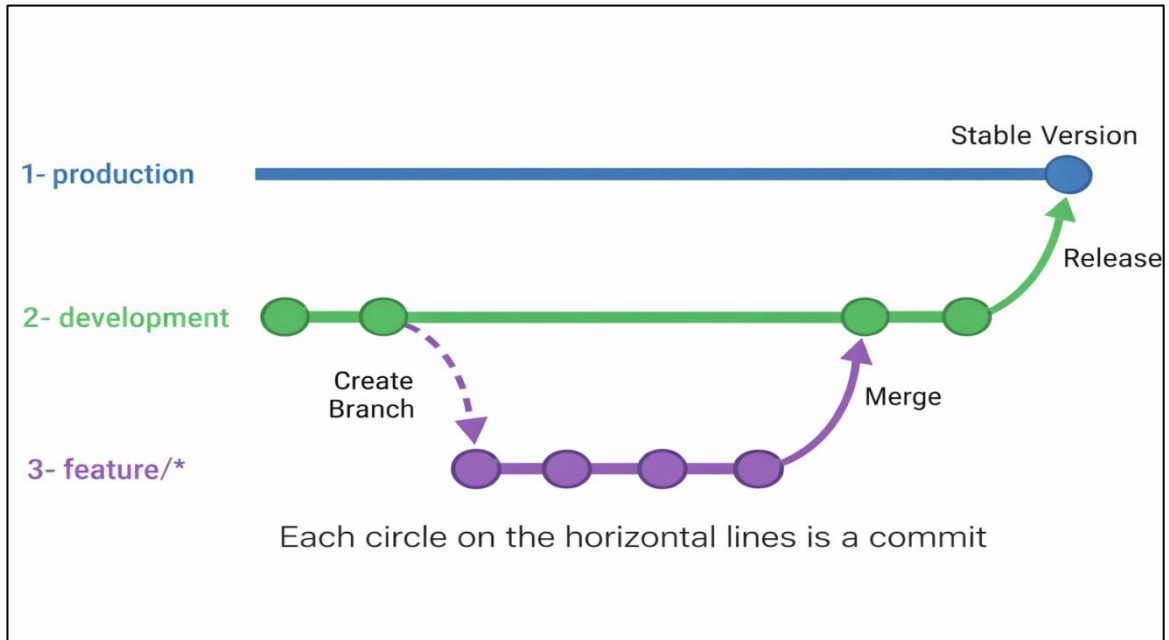


Figure 7 Branching and merging workflow from feature development to production release.

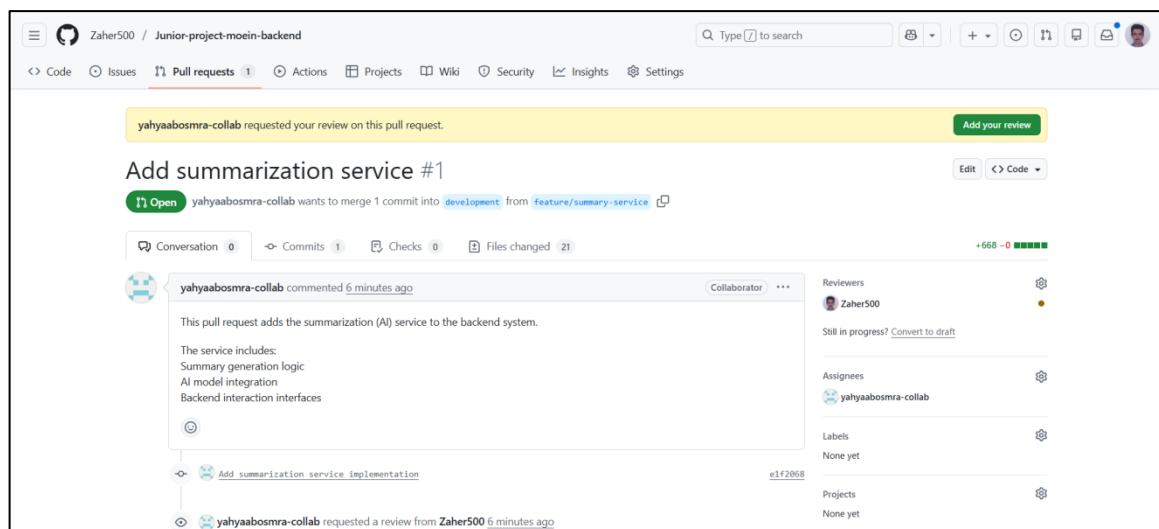


Figure 8 Pull request used to merge the summary service feature branch into the development branch.

This workflow is applied consistently to both backend and frontend development. For the backend, it ensures that new microservices or system enhancements are safely integrated before release. For the frontend, it guarantees that user interface changes and new client-side features are validated before being deployed.

Overall, this approach ensures code stability, clear change tracking, controlled integration, and organized progress throughout the project lifecycle.

3.6.6 Tags and Versioning

Git tags are used to mark important project milestones and stable releases across both the backend and frontend repositories. Tagging enables clear identification of release points, improves traceability, and supports controlled deployment.

v1.0 – Internal Development Version

Version **v1.0** was used as an internal development milestone during early stages of the project. This version was not considered stable due to missing core infrastructure components and incomplete system integration, and therefore was not treated as an official release.

v1.1 – Initial Stable Release

Version v1.1 represents the first fully functional and stable release of the system. For the backend, this release includes the Account User Service, Course and Lecture Service, and the API Gateway. For the frontend, it includes the stable user interface and full integration with backend APIs.

v1.2 – Summary Service Integration

Version v1.2 extends the system by integrating the summarization (AI) service. This release includes the AI-based summary generation service developed by a team member and its integration with the existing backend microservices. The release demonstrates collaborative development through feature branching, pull requests, and controlled merging into the production branch.

Only validated and production-ready versions were tagged as official releases.

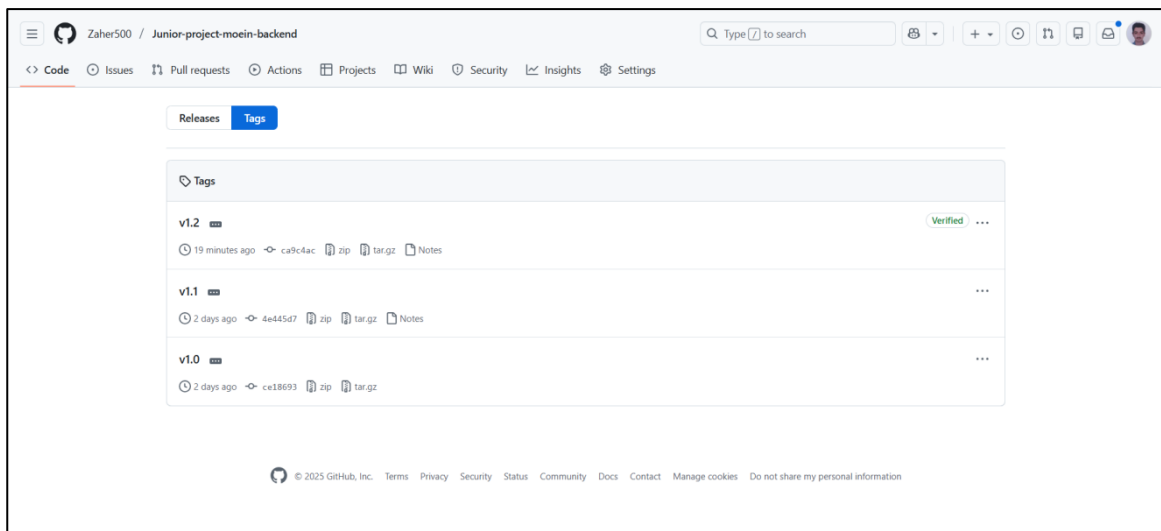


Figure 9 Git tags used to mark stable backend releases, highlighting versions v1.1 and v1.2.

3.6.7 Releases Management

The project follows a release-based development approach to deliver stable and incremental system functionality. Each release represents a validated milestone and is created only after successful integration, testing, and merging into the production branch.

GitHub Releases are used in combination with Git tags to document stable versions of the system, describe included features, and provide traceability throughout the project lifecycle.

Release v1.1 – Initial Stable Release

Release **v1.1** represents the first stable and fully functional version of the system. This release established the core backend and frontend infrastructure required for system operation.

Included components:

- Account User Service
- Course and Lecture Service
- API Gateway for request routing
- Stable frontend user interface with backend integration

This release served as the foundation for subsequent system extensions and enhancements.

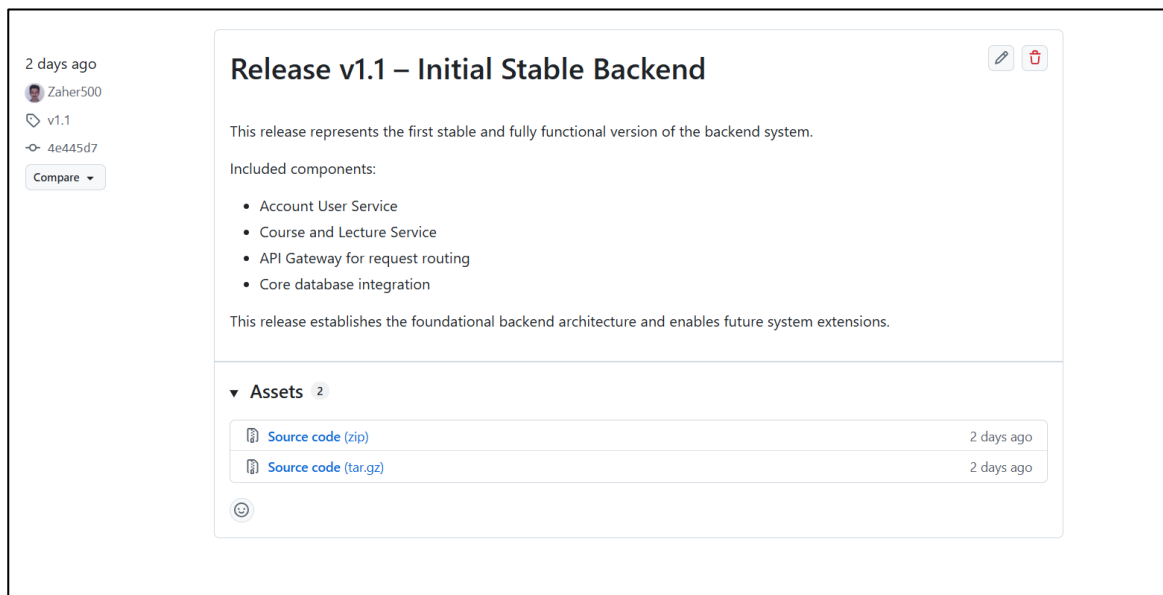


Figure 10 GitHub release illustrating stable system version v1.1

Release v1.2 – Summary Service Integration

Release **v1.2** extends the system by integrating the summarization (AI) service. This release demonstrates collaborative development and incremental feature delivery.

Key additions:

- AI-based summarization service implementation
- Integration of the summary service with existing backend microservices
- Controlled feature integration using pull requests and branch merging

This release reflects the evolution of the system beyond core functionality and highlights the use of version control practices to manage collaboration and feature growth.

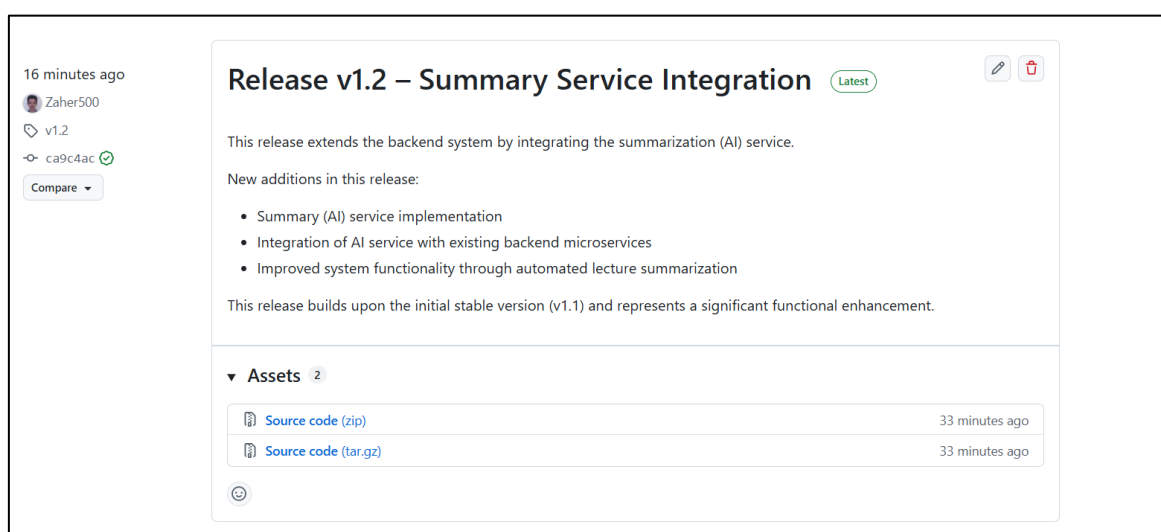


Figure 11 GitHub release illustrating stable system version v1.2

Release Strategy Summary

Only production-ready versions of the system are published as official releases. Earlier internal development versions were not released due to incomplete system components. This strategy ensures that each release represents a stable and deployable system state.

3.6.8 Summary:

In conclusion, good project management is vital for successful software system development. It provides structure, ensuring projects are completed within scope and schedule. Overall, strong project management enhances the delivery of high quality software systems that meet the goals of the project.

Chapter 4 System Analysis

4.1 Introduction

In this chapter we will introduce the analytical study of the system using the needed UML diagrams for system requirements modeling.

4.2 Requirements Elicitation

During the requirements elicitation process, we have taken several steps to ensure a comprehensive understanding of the project. We conducted a study and analysis of similar systems, also analyzed, and anticipated the needs of the application's users.

The resulted Requirement database:

Table 6 Requirement database

req-id	Requirement title	Actor	priority
RE-FR-AM-01	The system shall allow users to create new accounts.	User	high
RE-FR-AM-02	The system shall allow users to log in.	User	high
RE-FR-AM-03	The system shall allow users to manage their accounts	User	medium
RE-FR-AM-04	The system shall allow users to create a personal storage space for their uploaded files.	User	high
RE-FR-AM-05	The system shall allow users to upload files in PDF, DOCX, or PPTX formats.	User	High
RE-FR-AM-06	The system shall analyze the uploaded files using AI.	System	High

RE-FR-AM-07	The system shall generate summaries for the uploaded files using AI.	System	high
RE-FR-AM-08	The system shall allow users to export the generated summary as a PDF file.	System	medium
RE-FR-AM-09	The system shall store the generated summary along with the related PDF data.	System	medium
RE-FR-AM-10	The system shall generate exams based on the uploaded lecture files.	System	high
RE-FR-AM-11	The system shall generate various types of questions based on the uploaded files using AI.	System	high
RE-FR-AM-12	The system shall link each generated question to the file or lecture it was derived from.	System	medium
RE-FR-AM-13	The system shall automatically evaluate student answers based on the file content.	System	high
RE-FR-AM-14	The system shall provide explanations for answers based on the file content.	System	medium
RE-FR-AM-15	The system shall allow text-based search within all uploaded files.	User	medium
RE-FR-AM-16	The system shall provide an AI-powered chatbot that answers user questions based on uploaded files content.	User	high
RE-FR-AM-17	The system shall allow users to log out.	User	medium

4.3 SRS Document

4.3.1 Purpose:

The purpose of this Software Requirements Specification (SRS) is to outline the functional and non-functional requirements for our system. This document will serve as a foundational reference to ensure alignment between stakeholders and the development team throughout the project lifecycle.

4.3.2 Project scope:

Helps students organize and manage their study process using artificial intelligence, Allows uploading of study materials like lectures or audio, Automatically summarizes the uploaded content for quick understanding.

Creates smart questions and short quizzes to support learning, Builds a personalized study plan that adapts to the student's progress.

Offers an AI tutor (chatbot) that answers questions and explains topics using the student's own materials

4.3.3 Document overview:

The software requirements specification SRS document will be structured as follows:

1. Purpose.
2. Project scope.
3. Document overview.
4. Overall description and product features.
5. Nonfunctional requirements.
6. Conclusion.
7. Requirements modeling.

4.3.4 Overall description and product features:

4.3.4.1 Product features:

The product features derived from the systems use cases are:

- Sign Up (UC-01) — *Primary actor: Student.*
This feature allows a new student to create an account on the system by providing required details (email, password, name) and completing email verification.
- Log In (UC-02) — *Primary actor: Student.*
This feature enables registered users to authenticate (email/password) and receive a JWT for accessing protected APIs and services.
- Manage Account (UC-03) — *Primary actor: Student.*
This feature provides students the ability to view and update their profile information, change password, and manage personal settings.
- Upload Lecture (UC-04) — *Primary actor: Student.*
This feature allows a student to upload lecture files in supported formats (PDF, DOCX, PPTX) to a selected course.
- Parse Lecture File (UC-05) — *Primary actor: System (triggered by Upload).*
This feature extracts plain text and structure (titles, bullet points, slide text) from uploaded files to prepare content for AI processing.
- Analyze Uploaded File (UC-06) — *Primary actor: System*
This feature runs NLP preprocessing on the extracted text (cleaning, segmentation, embedding) as a preparation step for summarization and question generation.
- Generate Summary using AI (UC-07) — *Primary actor: System*
This feature automatically produces a concise, well-structured text summary of the lecture using Hugging Face models and stores the result for student access.
- Export Summary as PDF (UC-08) — *Primary actor: Student.*
This feature enables students to export any generated summary into a downloadable PDF document for offline review.
- Generate Quiz (UC-09) — *Primary actor: Student / System.*
This feature creates a quiz based on the lecture content; quizzes consist only of multiple-choice questions (MCQs) derived automatically from the lecture text.
- Generate MCQs (UC-10) — *Primary actor: System.*
This feature generates multiple-choice questions with one correct answer and plausible distractors from lecture text using transformer-based QG techniques.

- Auto-Evaluate Answers (UC-11) — *Primary actor: System.*
This feature automatically grades student quiz submissions by comparing answers to stored keys and records the score and result.
- Provide Answer Explanation (UC-12) — *Primary actor: System / Student.*
This feature supplies an explanation for each question's correct answer (generated or retrieved) after grading to support learning and feedback.
- AI Chatbot Q&A (UC-13) — *Primary actor: Student.*
This feature allows students to ask free-text questions about course content or summaries and receive AI-driven answers and clarifications in natural language.
- Lecture Management — View Lecture (UC-14) — *Primary actor: Student.*
This feature enables students to browse and open lecture files and their generated summaries for any course they own or are enrolled in.
- Lecture Management — Edit Lecture Info (UC-15) — *Primary actor: Student.*
This feature allows students to edit lecture metadata (title, description, tags, associated module/course) and trigger reprocessing if needed.
- Lecture Management — Search Lecture Content (UC-16) — *Primary actor: Student.*
This feature provides full-text search over lecture texts and summaries so students can quickly find concepts, keywords, or specific sections.
- Lecture Management — Delete Lecture (UC-17) — *Primary actor: Student.*
This feature allows students to remove a lecture file and its associated AI artifacts (extracted text, summary, generated MCQs) from a course.
- Course Management — Create Course (UC-18) — *Primary actor: Student.*
This feature enables creation of a new course container (title, code, description) to which lectures can be added.
- Course Management — Edit Course Info (UC-19) — *Primary actor: Student*
This feature permits updating course metadata, enrollment settings, and visibility options.
- Course Management — Delete Course (UC-20) — *Primary actor: Student*
This feature allows the removal of an entire course and optionally all contained lectures and artifacts.
- Manage Accounts (UC-21) — *Primary actor: Student .*
This feature enables the administrator to create, update, suspend, or delete user accounts and to manage role assignments.

4.3.4.2 User classes and characteristics:

Our system has two main types of users:

- **Admin (System Administrator):**

The Admin is a high-privilege user responsible for overseeing the overall operation and maintenance of the application. Duties include managing user accounts (create, update, suspend, delete), creating or deleting courses when needed, monitoring system usage and processing performance, moderating content, and configuring system-wide settings such as upload limits or retention policies. Admins are expected to have above-average technical proficiency (comfortable with dashboards, logs, and basic troubleshooting) and require reliable, secure access (strong authentication and audit trails). They typically use the system less frequently than students but need powerful tools for management, and emergency intervention. Security, data privacy, and the ability to view system analytics are key characteristics of this role.

- **Student (Primary user):**

The Student is the main consumer of the system's learning and content-management features. Students register and manage their accounts, create and organize courses, upload lecture files (PDF, DOCX, PPTX), request AI summaries and quizzes, view/export summaries, take auto-generated MCQ quizzes, and review their graded results and explanations. They are assumed to have basic computer literacy (familiarity with web apps, file uploads, and reading documents) and will interact with the system frequently—often on mobile devices or laptops—primarily for studying and course organization. Students expect quick, accurate summaries, straightforward UX for uploads and quizzes, and clear privacy controls over their uploaded materials and generated artifacts.

4.3.4.3 Assumptions:

Network Connection: It is assumed that users will have a stable and reliable network connection. The system's performance and accessibility depend on a consistent internet connection, as the application is web-based.

Browser Requirements: Users are expected to access the system through a modern web browser that supports HTML5, CSS3, and JavaScript. The system will be optimized for browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge.

Project in Production: It is assumed that the Product Owner has their project in the production stage. This is necessary for them to gather and analyze user reviews, which are critical for leveraging the system's insights and analytical capabilities.

4.3.5 Nonfunctional requirements:

- **RE-NF-01 (Usability – User-Friendly Interface):**

The system shall provide simple and intuitive user interfaces for both Students and Admins. A new user should be able to understand the main system functions (course creation, lecture upload, summary viewing, and quiz usage) within **15 minutes** of initial use without external assistance.

- **RE-NF-02 (Security – Authentication and Data Protection):**

The system shall ensure the security and privacy of all user data by implementing JWT-based authentication, role-based access control, and secure password policies (minimum 8 characters including at least one number). All data transmission must occur over secure HTTPS connections.

- **RE-NF-03 (Performance – Responsiveness and AI Processing):**

The system shall respond to standard user operations (navigation, search, viewing summaries) within **10 seconds**. AI-based operations such as lecture summarization and quiz generation shall be processed asynchronously and completed within an acceptable time frame, with progress status provided to the user.

▪ **RE-NF-04 (Scalability and Extensibility):**

The system shall be designed with a modular and scalable architecture that allows future expansion, including upgrading AI models, adding new features (search, export, multimedia support), and handling increased numbers of users and lecture files without major system redesign.

4.3.6 Conclusion:

This software Requirements Specification (SRS) document outlines the critical functionalities, user roles, and assumptions essential for developing a robust and

User-friendly system. By adhering to these requirements, the system will effectively support software product owners in enhancing their products to meet user needs and streamline project management processes.

4.3.7 Requirements modeling:

Use case diagram: use-case diagrams model the behavior of a system and help to capture the requirements of the system.

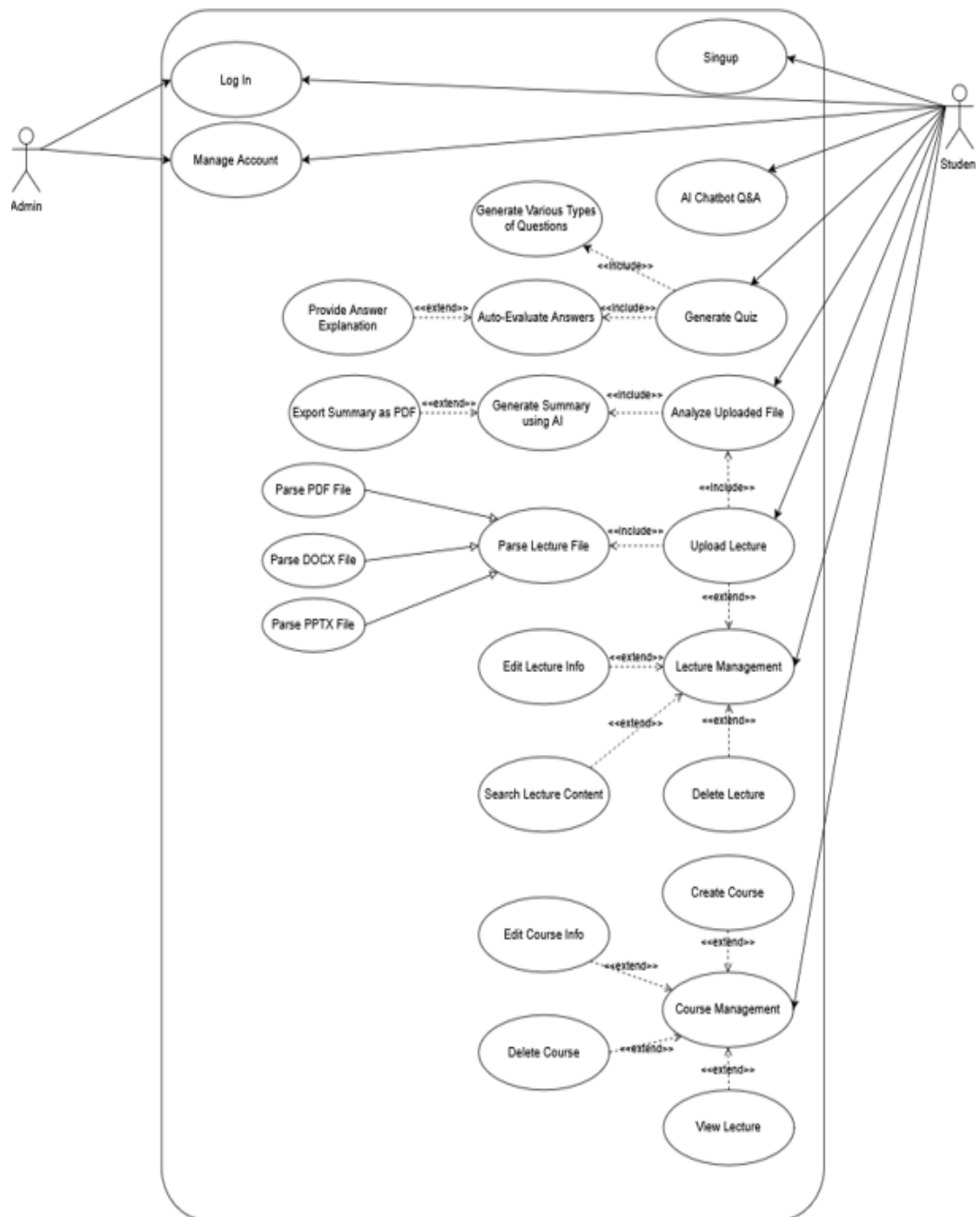


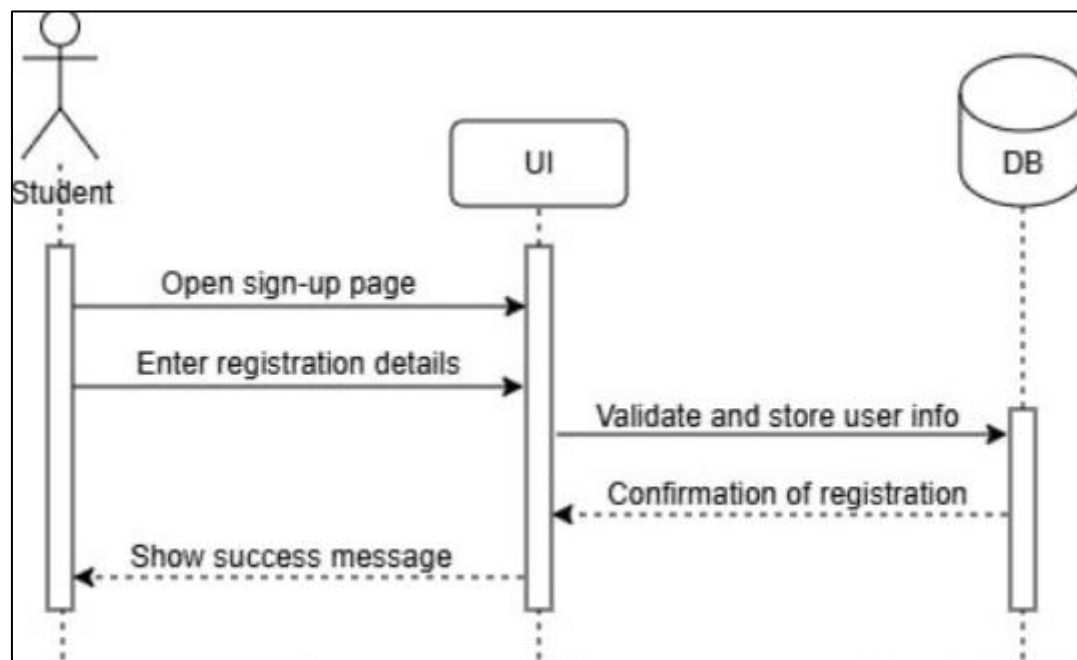
Figure 12 Use case diagram

System features – use cases:

SignUp (UC-01):

Table 7 Use case specification for signup

Use case name:	UC-01 Signup
Participating Actors:	Student
The flow of events:	1. Open sign-up page. 2. Enter name, email, phone, password. 3. System validates data. 4. System stores user. 5. System displays success message.
Alternative flows:	Alternative flow A1: starts at step 3 from the main flow: invalid or duplicate email is entered. 4. The system displays a validation error message and prevents account creation.
Entry condition:	• User is not registered
Exit conditions:	• Account created successfully



Sequence diagram for signup13 Figure

Login (UC-02):

Table 8 Use case specification for login

Use case name:	UC-02 Log In
Participating Actors:	Student, Administrator
The flow of events:	1. Open login page. 2. Enter email and password. 3. System verifies credentials. 4. System grants access.
Alternative flows:	Alternative flow A1: starts at step 3 from the main flow: incorrect email or password is entered. 4. The system displays an authentication failure message and denies access.
Entry condition:	• User has an existing account
Exit conditions:	• User authenticated successfully

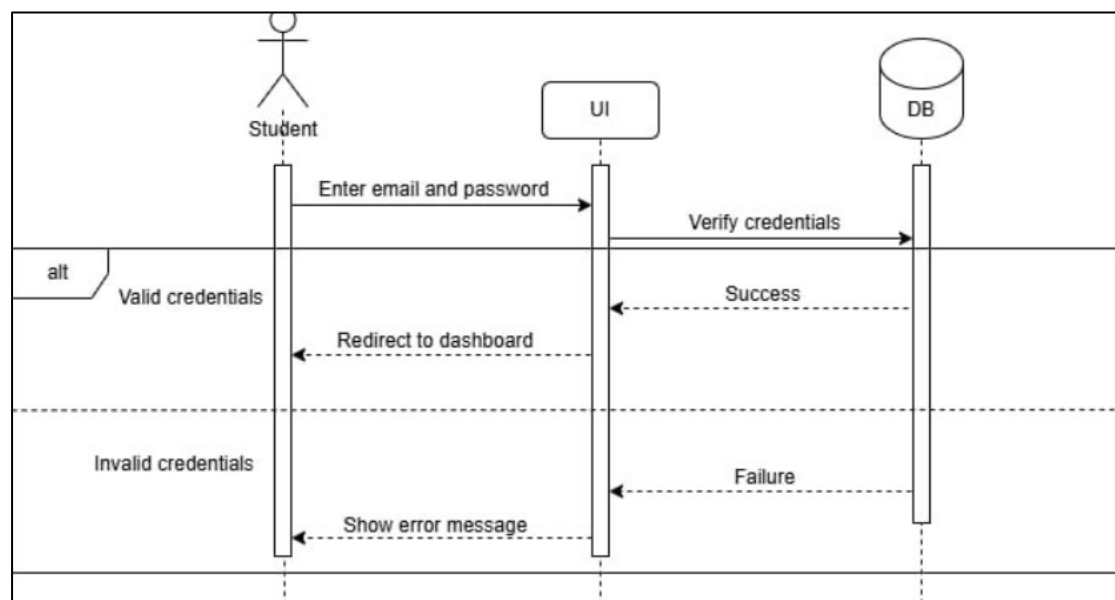
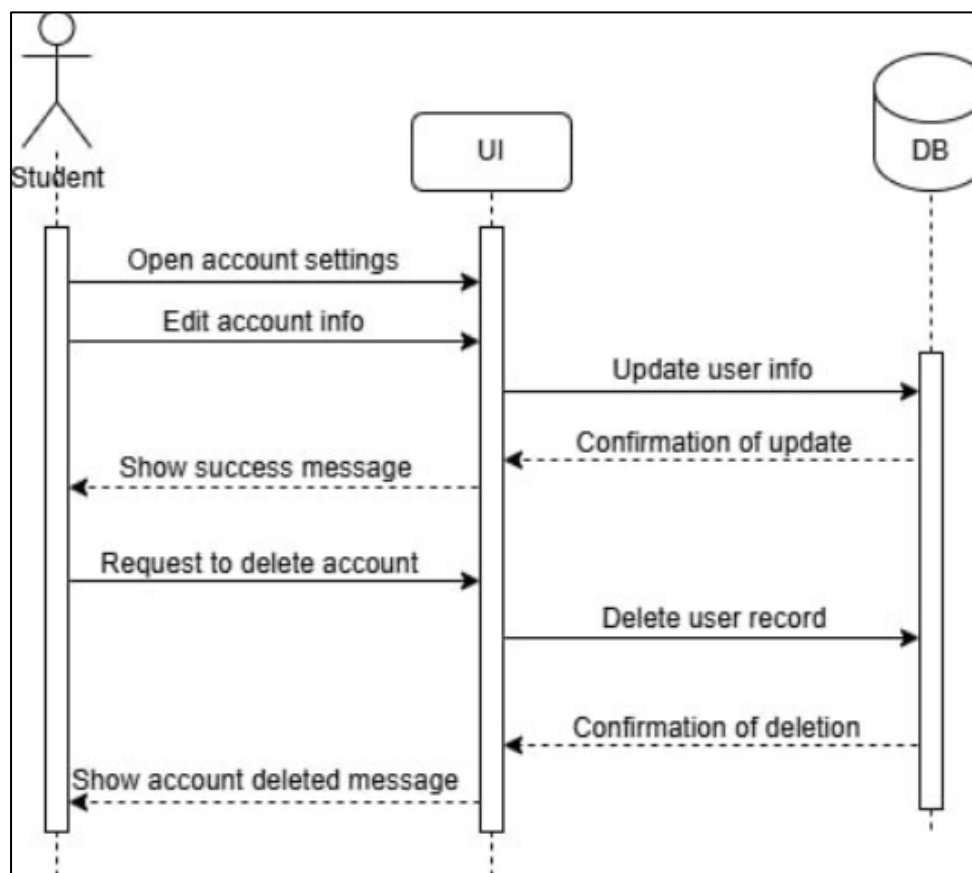


Figure14 Sequence diagram for login

Account Management (UC-03):

Table 9 Use case specification for account management

Use case name:	UC-03 Manage Account
Participating Actors:	Student, Administrator
The flow of events:	1. Open account settings. 2. Edit personal information. 3. System validates data. 4. System updates data. 5. System displays confirmation
Alternative flows:	Alternative flow A1: starts at step 3 from the main flow: updated data is invalid. 4. The system rejects the update and displays a validation error message.
Entry condition:	• User is logged in
Exit conditions:	• Account updated successfully



Sequence diagram for accountmanagement15 Figure

Generate Quiz (UC-04):

Table 10 Use case specification for generate quiz

Use case name:	UC-04 Generate Quiz
Participating Actors:	Student
The flow of events:	1. The student selects a lecture file or summary from the available course materials. 2. The student selects the desired exam type (e.g., topic-based, difficulty level). 3. The system retrieves the selected content. 4. The system analyzes the content and generates a set of questions (e.g., multiple choice, true/false, short answer). 5. The system displays the generated questions to the student. 6. The student begins answering the questions one by one. 7. For each answer, the system evaluates the response and provides immediate feedback (correct/incorrect + explanation). 8. After completing all questions, the student clicks "Finish Exam". 9. The system calculates the final score and displays it to the student.
Alternative flows:	<u>Alternative flow A1:</u> starts at step 6 from the main flow: the student submits a blank answer. 7. The system prompts the student to submit an answer before proceeding.
Entry condition:	• The student must be logged in. • A valid lecture file or summary must be available and selected
Exit conditions:	• All answers are automatically evaluated and feedback is provided. • The final grade is calculated and displayed to the student

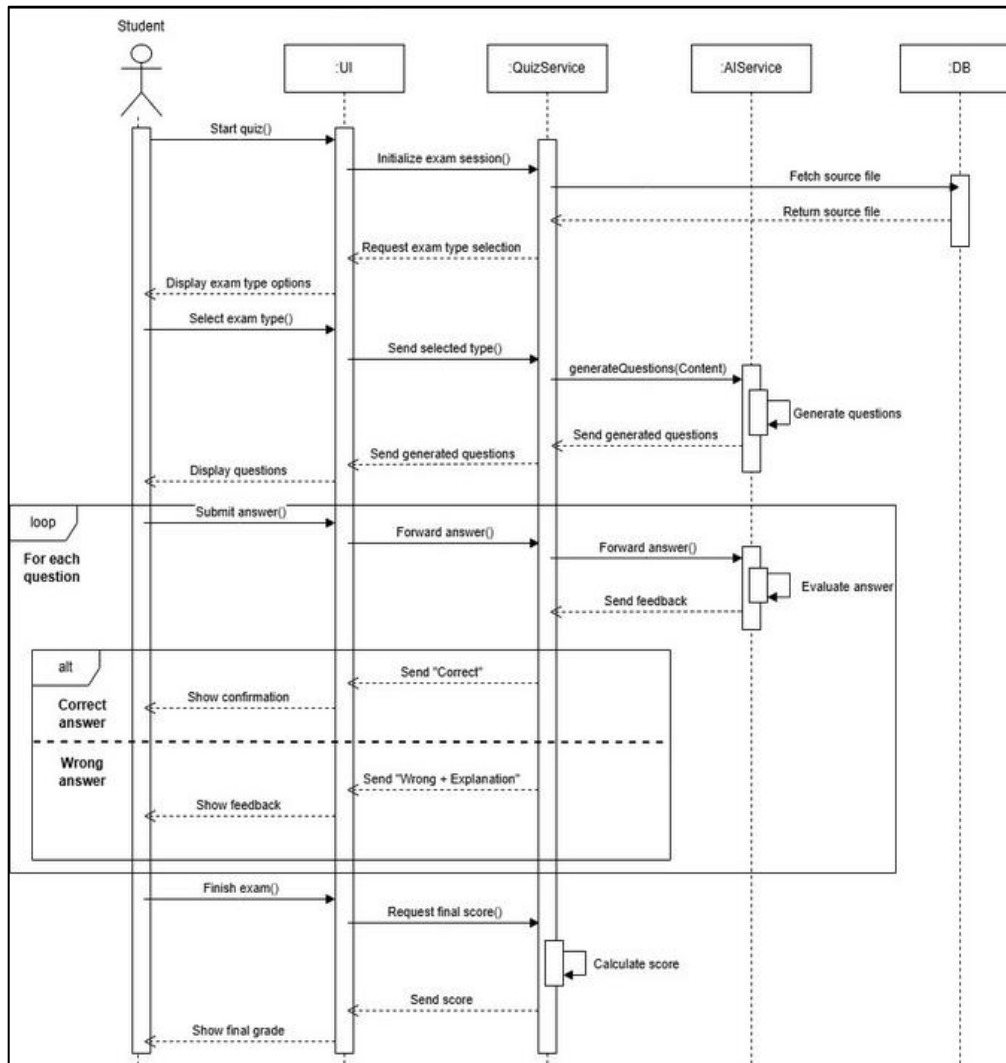


Figure16 Sequence diagram for generate quiz

Upload Lecture& Parse Lecture (UC-05):

Table 11 Use case specification for upload & parse lecture

Use case name:	UC-05 Upload & Parse Lecture
Participating Actors:	Student
The flow of events:	1. The student logs in and navigates to the selected course. 2. The student chooses Upload Lecture and selects a file in a supported format (PDF, DOCX, PPTX). 3. The student submits the upload request. 4. The system validates the file (file type, basic integrity / not empty). 5. The system stores the original file in persistent storage and returns an upload confirmation with a job-id for processing. 6. The system enqueues a parsing job and immediately returns control to the student (upload is accepted). 7. A background parser worker retrieves the stored file and extracts textual content and structure (titles, slide bullets, headings, paragraph text). 8. The parser normalizes and cleans extracted text (remove headers/footers, fix encoding, merge multi-line sentences). 9. The parser saves the extracted plain text and structured metadata (page/slide boundaries, detected headings) as an AI-ready artifact in the document store. 10. The system updates the job status to Completed and notifies the student (or the frontend reports completion on next status poll). 11. The student views the parsed text and can trigger downstream actions (Generate Summary, Generate Quiz, or Edit lecture metadata).
Alternative flows:	• Alternative A1 — Unsupported file type: • Occurs at step 4. The system rejects the upload and returns an error message explaining supported formats. The student may choose a supported file and retry. • Alternative A2 — File corrupted / unreadable: • Occurs during step 4 or 7. The system marks the job Failed, logs the error, and notifies the student with a suggested action (re-upload or contact admin). • Alternative A3 — Partial extraction (e.g., images-only slides): • Occurs at step 7. The parser extracts available textual content and flags sections requiring manual review or OCR. The system stores the partial text, marks the job Partial, and prompts the student to review or re-

	upload a text-native file. • Alternative A4 — Asynchronous timeout / queue delay: • If processing is delayed due to heavy load, the system keeps the job in In Progress and provides estimated time or position in queue; the student may continue other tasks and check back later.
Entry condition:	• The student must be authenticated (logged in). • The student must have permission to add lectures to the selected course (owner/enrolled as allowed). • The selected file must be locally available to the student for upload.
Exit conditions:	• Success: Extracted plain text and structured metadata are stored and associated with the lecture record; the lecture status is Parsed/Ready; the student is notified; downstream AI tasks can begin. • Failure / Partial: The lecture record reflects the failure/partial state with an explanatory message; the original file remains stored (unless the student deletes it); student may retry upload or request support.

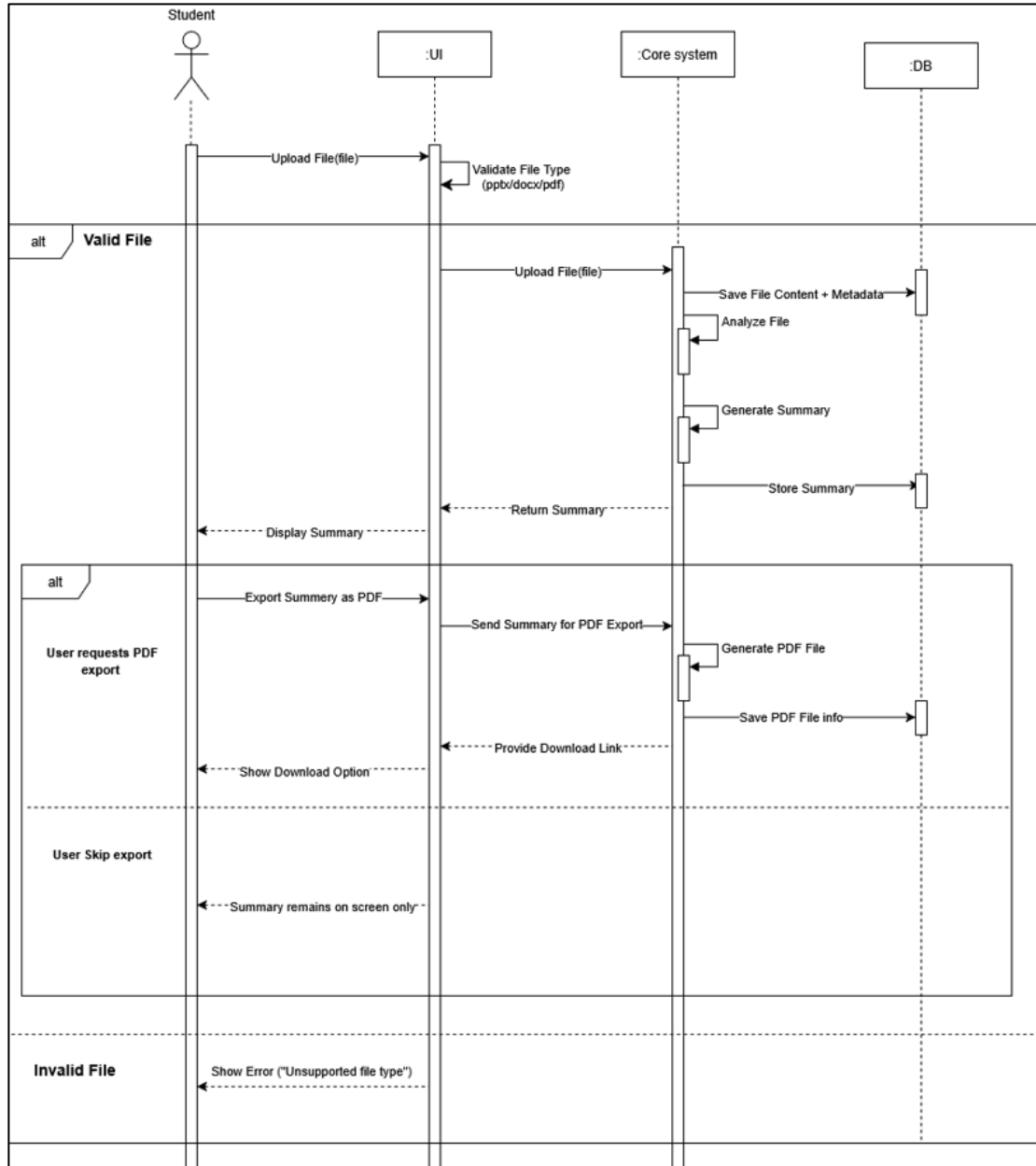


Figure17 Sequence diagram for upload lecture

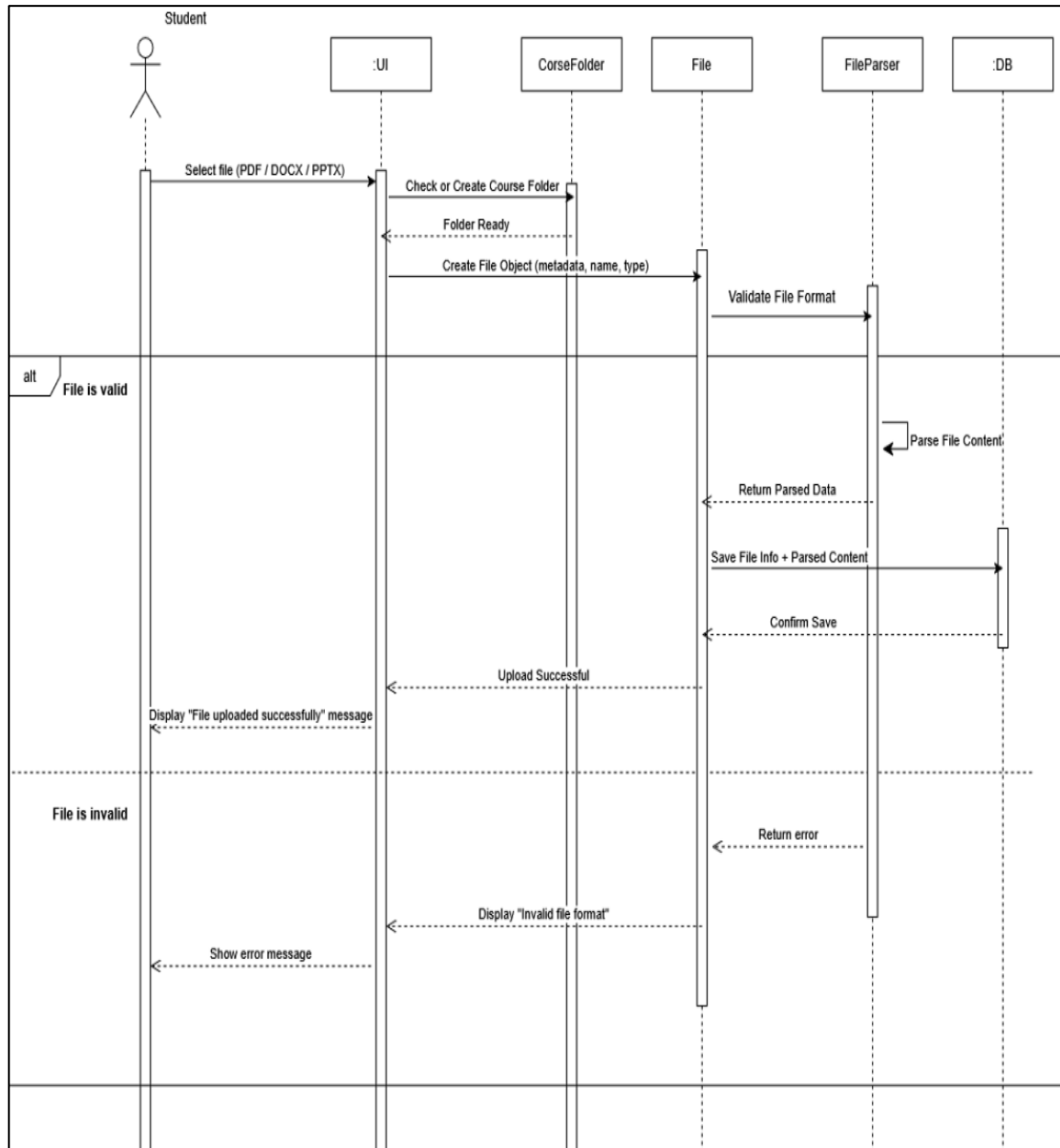


Figure18 Sequence diagram for parse lecture

Edit Lecture Info (UC-06):

Table 12 edit lecture info

Use case name:	UC-06 <i>Edit Lecture Info</i>
Participating Actors:	Student
The flow of events:	1. The student logs in and navigates to the selected course. 2. The student opens the lecture list and selects a lecture to edit. 3. The student clicks Edit Lecture and updates metadata fields (title, description, tags, module, visibility). 4. The student may optionally upload a replacement file or request re-processing. 5. The student submits the changes. 6. The system validates the input and, if a new file was provided, stores it and enqueues parsing as needed. 7. The system updates the lecture record with new metadata and timestamps, and returns a success confirmation. 8. The student views the updated lecture info.
Alternative flows:	• A1 — Validation error: occurs at step 6 if required fields are missing or invalid (e.g., title empty). The system rejects the change and displays validation messages; the student corrects input and resubmits. • A2 — File reprocessing fails: if a replacement file is uploaded and parsing fails, the system marks parsing Failed/Partial, retains the original file (if chosen), and notifies the student with remediation steps. • A3 — Insufficient permission: if the student is not authorized to edit this lecture, the system denies access and shows an authorization error.
Entry condition:	• Student is authenticated. • Student has permission to edit the selected lecture (owner or granted role). • Lecture exists in the selected course.
Exit conditions:	• Success: Lecture metadata is updated; if a new file was uploaded it is stored and parsing job is created. • Failure: No changes applied; system provides error details. All actions are logged.

Search Lecture Content (UC-07):

Table 13 search lecture content

Use case name:	UC-07 Search Lecture Content
Participating Actors:	Student
The flow of events:	1. The student logs in and navigates to a course or global search interface. 2. The student enters a search query (keywords, phrase, tag, or filter by course/module/date). 3. The system receives the query and queries the full-text index (lecture text, summaries, tags, metadata). 4. The system returns ranked search results showing matching lectures, snippets of matched text, and relevance scores. 5. The student selects a result to view the matching lecture and highlighted snippets within the summary or extracted text.
Alternative flows:	• A1 — No results found: system shows “no matches” with suggestions (broaden query, check spelling, remove filters). • A2 — Partial index / stale index: if content is recently uploaded and not yet indexed, the system indicates results may be incomplete and offers to retry after indexing completes. • A3 — Advanced filters produce zero matches: system suggests removing or changing filters..
Entry condition:	• Student is authenticated. • Indexing service is available (or system will transparently handle delayed indexing).
Exit conditions:	• Success: Ranked list of matching lectures returned; user can navigate to selected content. • Failure: Search service error displayed and logged; student may retry.

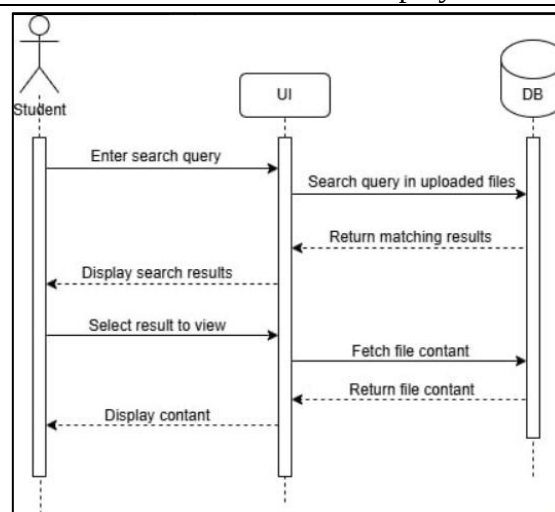


Figure19 Sequence diagram for search lecture content

Delete Lecture (UC-8):

Table 14 Use case specification for delete lecture

Use case name:	UC-8 Delete Lecture
Participating Actors:	System
The flow of events:	1. The student logs in and navigates to the course lecture list. 2. The student selects the lecture to delete and clicks Delete. 3. The system prompts for confirmation (shows consequences: removal of file, summaries, MCQs, scores). 4. The student confirms deletion. 5. The system marks the lecture as deleted (soft-delete) and removes or flags associated AI artifacts and generated quizzes. 6. The system returns success confirmation and updates the lecture list.
Alternative flows:	• A1 — Cancel deletion: student cancels at confirmation prompt; no change occurs. • A2 — Admin forced delete / policy restriction: if retention policy or admin settings prevent immediate removal, the system flags the lecture as Pending Deletion and schedules final purge after retention window; student is notified. • A3 — Deletion failure (storage error): system reports an error, retries or logs the failure, and notifies student to retry or contact admin.
Entry condition:	• Student is authenticated and authorized to delete the lecture (owner or permitted role). • Lecture exists.
Exit conditions:	• Success (soft-delete): Lecture marked deleted and hidden from normal views; associated artifacts flagged; deletion action logged with user/timestamp. • Success (hard-delete): If policy allows and retention expired, lecture and all related files/artifacts are permanently removed from storage. • Failure: Lecture remains; error returned and logged.

Create Course (UC-9):

Table 15 Use case specification for create course

Use case name:	UC-9: Create Course.
Participating Actors:	Student.
The flow of events:	1. The student selects the option to create a new course. 2. The system displays a form to enter course details. 3. The student submits the course information. 4. The system validates and saves the course data.
Alternative flows:	<u>Exceptional flow E1:</u> starts at step 3 from the main flow: The entered course information is incomplete or invalid. 4. The system shows an error message.
Entry condition:	• The student must be authenticated and logged into the system.
Exit conditions:	• A new course is successfully created and stored in the system.

Delete Course (UC-10):

Table 16 Use case specification for delete course

Use case name:	UC-10: Delete Course.
Participating Actors:	Student.
The flow of events:	1. The student selects the course to delete. 2. The system requests deletion confirmation. 3. The student confirms the deletion. 4. The system deletes the course and its related data.
Entry condition:	• The student must be authenticated and logged into the system.
Exit conditions:	• The selected course is permanently deleted.

Edit Course info (UC-11):

Table 17 Use case specification for edit course

Use case name:	UC-11: Edit Course Info.
Participating Actors:	Student.
The flow of events:	1. The student selects a course to edit. 2. The system displays current course information. 3. The student updates the desired fields. 4. The system saves the updated information.
Alternative flows:	<u>Exceptional flow E1:</u> starts at step 3 from the main flow: The updated course information fails validation. 4. The system rejects the update and displays an error message.
Entry condition:	• The student must be authenticated and logged into the system.
Exit conditions:	• Course information is successfully updated.

View Lecture (UC-12):

Table 18 Use case specification for view lecture

Use case name:	UC-12: View Lecture.
Participating Actors:	Student.
The flow of events:	1. The student selects a course. 2. The system displays a list of associated lectures. 3. The student selects a lecture to view its content.
Entry condition:	• The student must be authenticated and logged into the system.
Exit conditions:	• Lecture content is displayed to the student.

Analyze Uploaded File (UC-13):

Table 19 Use case specification for analyze uploaded file

Use case name:	UC-13 Analyze Uploaded Files.
Participating Actors:	Student.
The flow of events:	1. The student uploads a lecture file. 2. The system validates the file format and content. 3. The system prepares the file data for AI processing.
Alternative flows:	<u>Exceptional flow E1</u>: starts at step 2 from the main flow: The uploaded file format is not supported. 3. The system displays an error message indicating an invalid file type.
Entry condition:	• The student is authenticated and lecture file has been successfully uploaded.
Exit conditions:	• The uploaded file is successfully analyzed and ready for summarization.

Generate Summary Using AI (UC-14):

Table 20 Use case specification for generate summary using AI

Use case name:	UC-14 Generate Summary Using AI.
Participating Actors:	Student.
The flow of events:	1. The system sends the analyzed content to the AI service. 2. The AI model processes the text data. 3. The system generates a structured summary. 4. The summary is stored in the DB.
Entry condition:	• The student is authenticated and lecture file has been successfully uploaded.
Exit conditions:	• The lecture summary is successfully generated and saved.

Export Summary as PDF(UC-15):

Table 21 Use case specification for export summary as PDF

Use case name:	UC-15 Export Summary as PDF.
Participating Actors:	Student.
The flow of events:	5. The student selects the option to export the summary as PDF. 6. The system formats the summary content. 7. The system generates a PDF file. 8. The PDF file is provided for download.
Alternative flows:	<u>Exceptional flow E1:</u> starts at step 1 from the main flow: The summary does not exist or is not ready. 2. The system displays an error message indicating that summary is unavailable. <u>Exceptional flow E2:</u> starts at step 3 from the main flow: The system fails to generate the PDF file. 4. The system displays an error message: “Failed to export summary as PDF.”
Entry condition:	• The student is authenticated and summary has already been generated.
Exit conditions:	• The summary is successfully exported as a PDF file.

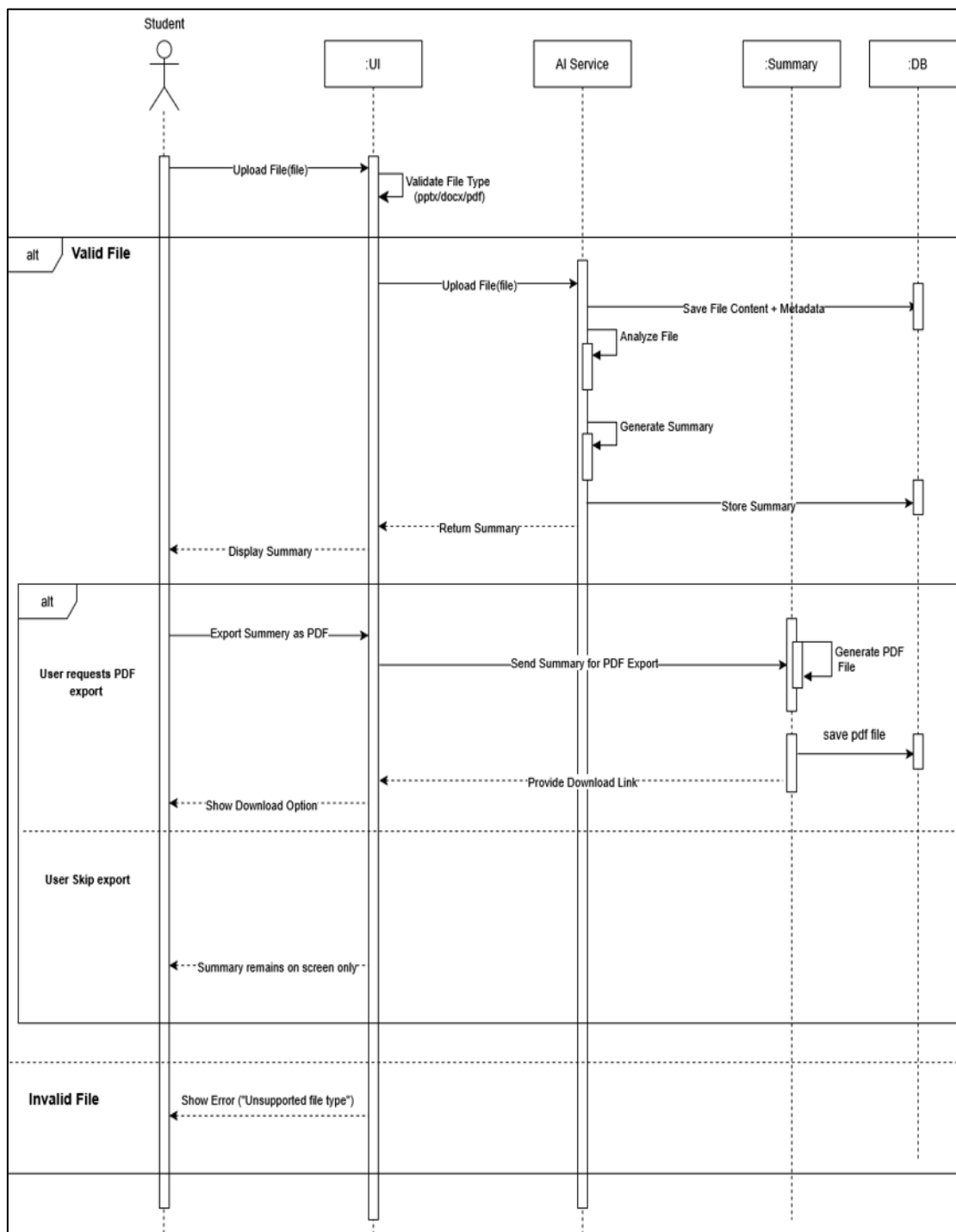


Figure20 Sequence diagram for analys uploaded file, generate summary using AI & export summary as PDF

Analysis class diagram:

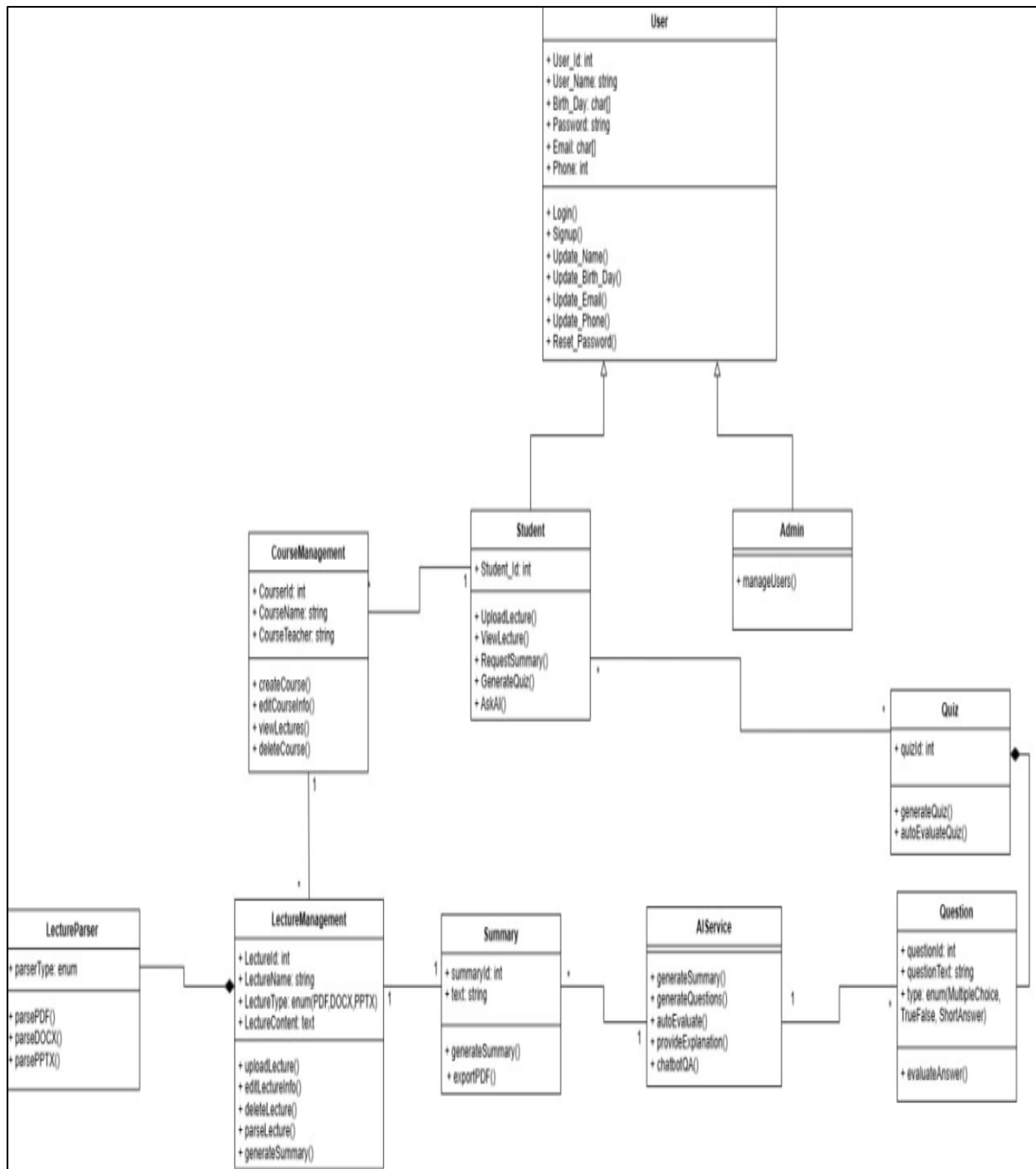


Figure21 class diagram

Entity Relationship Diagram ERD:

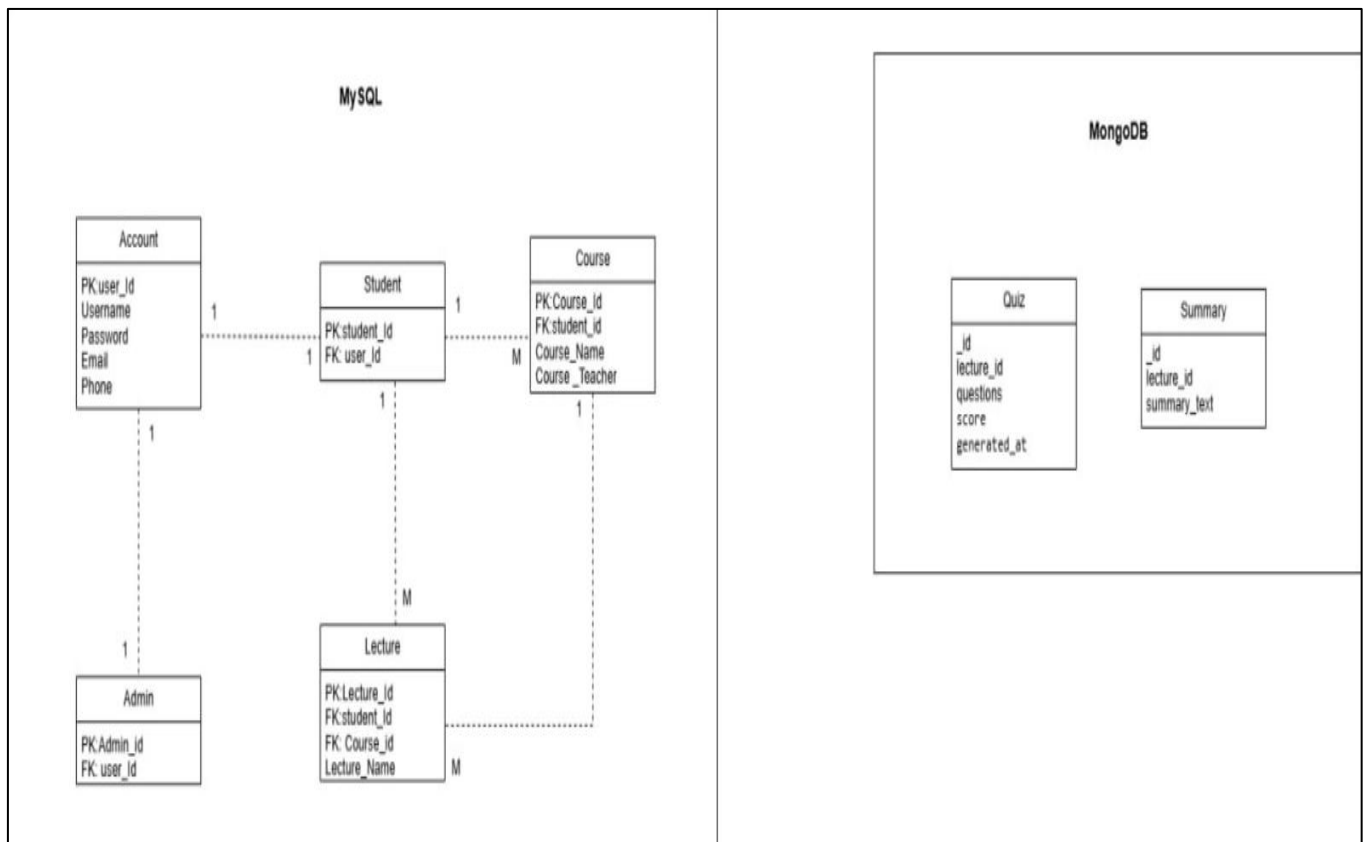


Figure22 Entity Relationship Diagram (ERD)

Initial Test Cases:

Table 22 Test case specification for Signup.

Test Case Scenario:		Case 1: Signup	
Test case id	Test case title	Test steps	Expected result
TC-01	Signup with valid data	1) Navigate to Signup page. 2) Enter valid username. 3) Enter valid email. 4) Enter strong password. 5) Click Sign Up.	Account is created successfully.
TC-02	Signup with email without @	1) Navigate to Signup page. 2) Enter username. 3) Enter email without @. 4) Enter valid password. 5) Click Sign Up.	Invalid email format error is displayed.

Table 23 Test case specification for Login

Test Case Scenario:		Case 2: Login	
Test case id	Test case title	Test steps	Expected result
TC-03	Login with valid credentials	1) Open system login page. 2) Enter valid email. 3) Enter valid password. 4) Click Login.	User is logged in successfully.
TC-04	Login with invalid email or password	1) Open system login page. 2) Enter invalid email or enter invalid password. 3) Click Login.	Error message is displayed.

Table 24 Test case specification for Manage Account

Test Case Scenario:		Case 3: Manage Account	
Test case id	Test case title	Test steps	Expected result
TC-05	Update account information	1) Login as student. 2) Navigate to Manage Account. 3) Edit profile information. 4) Click Save.	Account information updated successfully.
TC-06	Edit user data while leaving blank fields	1) Login as student. 2) Navigate to Manage Account. 3) Edit profile information. 4) Leave fields blank 5) Click Save.	Error message is displayed.
TC-07	Edit user data with an incorrect password entered	1) Login as student. 2) Navigate to Manage Account. 3) Edit profile information. 4) Entering a password that does not match 5) Click Save.	Error message is displayed.

Table 25 Test case specification for Generate quiz

Test Case Scenario:		Case 4: Generate quiz	
Test case id	Test case title	Test steps	Expected result
TC-08	Generate quiz successfully with valid input.	1) Login as student. 2) Navigate to "Generate Quiz". 3) Select ≥ 1 lecture. 4) Select question type. 5) Enter question count. 6) Click Generate.	Quiz is generated with requested number & type of questions.
TC-09	Generate quiz from multiple lectures.	1) Login as student. 2) Navigate to "Generate Quiz". 3) Select multiple lectures. 4) Select question type. 5) Enter question count. 6) Click Generate.	Quiz generated using all selected lectures.
TC-10	Generate quiz with mixed question types.	1) Login as student. 2) Navigate to "Generate Quiz". 3) Select lectures. 4) Select multiple types. 5) Enter counts. 6) Click Generate.	Quiz contains selected mixed types.
TC-11	Generate quiz with no lecture selected.	1) Login as student. 2) Navigate to "Generate Quiz". 3) Do not select any lecture. 4) Click Generate.	Error: Must select at least 1 lecture.

TC-12	No question type selected.	1) Login as student. 2) Navigate to "Generate Quiz". 3) Select lectures. 4) Do not select question type. 5) Click Generate.	Error: Must select question type.
TC-13	No question count entered.	1) Login as student. 2) Navigate to "Generate Quiz". 3) Select lectures. 4) Select type. 5) Leave count empty. 6) Click Generate.	Error: Must enter number of questions.
TC-14	Zero question count	1) Login as student. 2) Navigate to "Generate Quiz". 3) Select lectures. 4) Select type. 5) Enter 0. 6) Click Generate.	Error: Question count must be greater than 0.
TC-15	Count > available questions	1) Login as student. 2) Navigate to "Generate Quiz". 3) Select lectures. 4) Select type. 5) Enter > available content. 6) Click Generate.	Error: Requested questions exceed available content.

Table 26 Test case specification for Auto evaluate answers

Test Case Scenario:		Case 5: Auto evaluate answers	
Test case id	Test case title	Test steps	Expected result
TC-16	Submit a correct answer.	1) Login as student. 2) Navigate to Quiz. 3) Select an answer. 4) Click Submit.	Correct answer message shown.
TC-17	Submit an incorrect answer.	1) Login as student. 2) Navigate to Quiz. 3) Select an answer. 4) Click Submit.	Incorrect answer message shown + explanation .
TC-18	Submit an empty answer.	1) Login as student. 2) Navigate to Quiz. 3) Do not select answer. 4) Click Submit.	Must select an answer message shown.

Table 27 Test case specification for Analyze Uploaded File

Test Case Scenario:		Case 6: Analyze Uploaded File	
Test case id	Test case title	Test steps	Expected result
TC-19	Upload valid PDF.	1) Login as Student. 2) Open upload page. 3) Select a valid PDF file. 4) Click Upload.	The system accepts the PDF.
TC-20	Upload valid DOCX.	1) Login as student. 2) Open upload page. 3) Select a valid DOCX file. 4) Click Upload.	The system accepts the DOCX.
TC-21	Upload valid PPTX.	1) Login as student. 2) Open upload page. 3) Select a valid PPTX file . 4) Click Upload.	The system accepts the PPTX.
TC-22	Reject unsupported file type (TXT).	1) Login as student. 2) Open upload page . 3) Select a valid TXT file. 4) Click Upload.	Error: The file not supported.
TC-23	Empty file content.	1) Login as student. 2) Open upload page. 3) Select a PDF/DOCX/PPTX. 4) Click Upload.	Error: The file is Empty.

Table 28 Test case specification for Generate Summary using AI

Test Case Scenario:		Case 7: Generate Summary using AI	
Test case id	Test case title	Test steps	Expected result
TC-24	Generate summary from a valid file.	1) Student logs in successfully. 2) Student opens the document upload page. 3) Student selects a valid file (PDF/DOCX/PPTX) that contains readable content. 4) Student clicks on "Generate Summary".	The system generate summary and displayed to the student .

Table 29 Test case specification for Export Summary as pdf

Test Case Scenario:		Case 8: Export Summary as pdf	
Test case id	Test case title	Test steps	Expected result
TC-25	Export summary to PDF.	1) Login as Student. 2) Uploads a valid file. 3) Generates summary successfully. 4) Clicks Export as PDF.	The system Export Summary to PDF and download link displayed to student.

TC-26	Attempt export without summary.	1) Login as student. 2) Uploads a valid file. 3) skip summary generation. 4) Click Export as PDF.	Error: "please generate a summary before exporting"; no PDF created.
-------	---------------------------------	--	--

Table 30 Test case specification fo Create Course

Test Case Scenario:		Case 9: Create Course	
Test case id	Test case title	Test steps	Expected result
TC-27	Create Course with Valid Data.	1) The student selects Create Course. 2) The system displays the course creation form. 3) The student enters valid course information. 4) The student submits the form.	The course is created successfully and added to the course list.
TC-28	Create Course with Missing Required Fields.	1) The student selects "Create Course. 2) The student leaves one or more required fields empty. 3) The student submits the form.	The system displays an error message indicating missing or invalid data.

Table 31 Test case specification for Edit Course Info

Test Case Scenario:		Case 10: Edit Course Info	
Test case id	Test case title	Test steps	Expected result
TC-29	Edit Course with Valid Information.	1) The student selects an existing course. 2) The student chooses the edit option. 3) The student updates the course information with valid data. 4) The student submits the changes.	The course information is successfully updated and saved.
TC-30	Edit Course with Invalid Data.	1) The student selects a course to edit. 2) The student enters invalid or incomplete course data. 3) The student submits the changes.	The system rejects the update and displays an error message.

Table 32 Test case specification for Delete Course

Test Case Scenario:		Case 11: Delete Course	
Test case id	Test case title	Test steps	Expected result
TC-31	Delete Course with Confirmation.	1) The student selects the course to delete. 2) The system displays a deletion confirmation message. 3) The student confirms the deletion.	The course and all related data are permanently deleted.

Table 33 Test case specification for View Lecture

Test Case Scenario:		Case 12: View Lecture	
Test case id	Test case title	Test steps	Expected result
TC-32	View Existing Lecture.	1) The student selects a course. 2) The system displays the list of available lectures. 3) The student selects a lecture.	The selected lecture content is displayed successfully.
TC-33	View Lecture When No Lectures Exist.	1) The student selects a course.	The system displays a message indicating that no lectures are available.

Table 34 Test case specification for Edit lecture info

Test Case Scenario:		Case 13: Edit lecture info	
Test case id	Test case title	Test steps	Expected result
TC-34	Edit lecture name successfully.	1) Login as student. 2) Navigate to the lectures list. 3) Select a lecture. 4) Click Edit . 5) Change the lecture name. 6) Click Save .	Lecture name is updated successfully and displayed with the new name.

TC-35	Edit lecture name with empty value.	1) Login as student. 2) Navigate to the lectures list. 3) Select a lecture. 4) Click Edit . 5) Clear the lecture name field. 6) Click Save .	System shows validation error and lecture name is not updated.
TC-36	Edit lecture name with special characters.	1) Login as student. 2) Navigate to the lectures list. 3) Select a lecture. 4) Click Edit . 5) Enter a name with special characters (e.g. @#lecture!) 6) Click Save .	System shows validation error and lecture name is not updated.

Table 35 Test case specification for Delete lecture

Test Case Scenario:		Case 14: Delete lecture	
Test case id	Test case title	Test steps	Expected result
TC-37	Delete lecture successfully.	1) Login as student. 2) Navigate to the lectures list. 3) Select a lecture. 4) Click Delete . 5) Confirm deletion.	Lecture is deleted successfully and removed from the lectures list.

Table 36 Test case specification for Search lecture content

Test Case Scenario:		Case 15: Search lecture content	
Test case id	Test case title	Test steps	Expected result
TC-38	Search lecture content successfully.	1) Login as student. 2) Navigate to the lectures list. 3) Open a lecture. 4) Enter a valid keyword in the search field. 5) Click Search .	Matching content inside the lecture is highlighted or displayed.
TC-39	Search with keyword not found in lecture.	1) Login as student. 2) Navigate to the lectures list. 3) Open a lecture. 4) Enter a keyword not present in the lecture. 5) Click Search .	System displays "No results found in this lecture".
TC-40	Search with empty input inside lecture.	1) Login as student. 2) Navigate to the lectures list. 3) Open a lecture. 4) Leave search field empty. 5) Click Search .	System shows validation message asking for a keyword.

Table 37 Test case specification for Upload Lecture

Test Case Scenario:		Case 16: Upload Lecture	
Test case id	Test case title	Test steps	Expected result
TC-41	Upload Lecture with Supported File.	1) The student selects the upload lecture option. 2) The student chooses a supported lecture file. 3) The student submits the file.	The lecture file is uploaded successfully and sent for parsing.
TC-42	Upload Lecture with Unsupported File Type.	1) The student selects an unsupported file format. 2) The student submits the file.	The system rejects the file and displays an error message.

Table 38 Test case specification for Parse Lecture File

Test Case Scenario:		Case 17: Parse Lecture File	
Test case id	Test case title	Test steps	Expected result
TC-43	Parse Lecture File (PDF).	1) The system detects the uploaded lecture file. 2) The system identifies the file type as PDF. 3) The system extracts text content from the PDF file.	The lecture content is successfully extracted as text.
TC-44	Parse Lecture File (DOCX).	1) The system detects the uploaded lecture file. 2) The system identifies the file type as DOCX. 3) The system extracts text content from the DOCX file.	The lecture content is successfully extracted as text.
TC-45	Parse Lecture File (PPTX).	1) The system detects the uploaded lecture file. 2) The system identifies the file type as PPTX. 3) The system extracts text content from presentation slides.	The lecture content is successfully extracted as text.
TC-46	Parse Lecture File with Unsupported Format.	1) The system detects the uploaded file. 2) The system identifies an unsupported file format.	The system rejects the file and displays an error message.

4.3.8 Initial RTM (Requirements Traceability Matrix):

Table 39 Initial RTM

REQ-ID	Use cases	analysis	System design	Detailed design	coding	Test cases
RE-FR-SU-01	UC-01	analysis				TC-01
						TC-02
RE-FR-LI-02	UC-02	analysis				TC-03
						TC-04
RE-FR-MA-03	UC-03	analysis				TC-05
						TC-06
						TC-07
RE-FR-GQ-04	UC-04	analysis				TC-08
						TC-09
						TC-10
						TC-11
						TC-12
						TC-13
						TC-14
RE-FR-AEA-05	UC-04	analysis				TC-15
						TC-16
						TC-17
RE-FR-U&PL-06	UC-05	analysis				TC-18
						TC-43
						TC-44
						TC-45
RE-FR-ELI-07	UC-06	analysis				TC-46
						TC-34

						TC-35
						TC-36
RE-FR-SLC-08	UC-07	analysis				TC-38
						TC-39
						TC-40
RE-FR-DL-09	UC-08	analysis				TC-37
RE-FR-CC-10	UC-09	analysis				TC-27
						TC-28
RE-FR-DC-11	UC-10	analysis				TC-31
RE-FR-ECI-12	UC-11	analysis				TC-29
						TC-30
RE-FR-VL-13	UC-12	analysis				TC-32
						TC-33
RE-FR-AUF-14	UC-13	analysis				TC-19
						TC-20
						TC-21
						TC-22
						TC-23
RE-FR-GSUAI-15	UC-14	analysis				TC-24
RE-FR-ESPDF-16	UC-15	analysis				TC-25
						TC-26

Chapter 5 System Design

5.1 Introduction

This chapter offers a comprehensive overview of the design phase steps, encompassing system design and architecture determination, as well as detailed design and modeling. It underscores the importance of these steps in creating a robust system for collecting, analyzing, and utilizing user feedback to continuously enhance the software system.

5.2 System Architecture

In this section we will discuss system design where system design involves defining the overall structure and architecture of the software, ensuring that all components work seamlessly together. This phase establishes the blueprint for efficient data flow, interaction, and integration within the system.

The first constraints to take into consideration when building an architecture for a software system are the non-functional requirements (quality attributes), so first we will start with the non-functional requirements of our system

1. **(Usability – User-Friendly Interface):**
The system shall provide simple and intuitive user interfaces for both Students and Admins. A new user should be able to understand the main system functions (course creation, lecture upload, summary viewing, and quiz usage) within **15 minutes** of initial use without external assistance.
2. **(Security – Authentication and Data Protection):**
The system shall ensure the security and privacy of all user data by implementing JWT-based authentication, role-based access control, and secure password policies (minimum 8 characters including at least one number). All data transmission must occur over secure HTTPS connections.
3. **(Performance – Responsiveness and AI Processing):**
The system shall respond to standard user operations (navigation, search, viewing summaries) within **10 seconds**. AI-based operations such as lecture summarization and quiz generation shall be processed asynchronously and completed within an acceptable time frame, with progress status provided to the user.
4. **(Scalability and Extensibility):**
The system shall be designed with a modular and scalable architecture that allows future expansion, including upgrading AI models, adding new features (search, export, multimedia support), and handling increased numbers of users and lecture files without major system redesign.

5.3 System decomposition

5.3.1 Component diagram

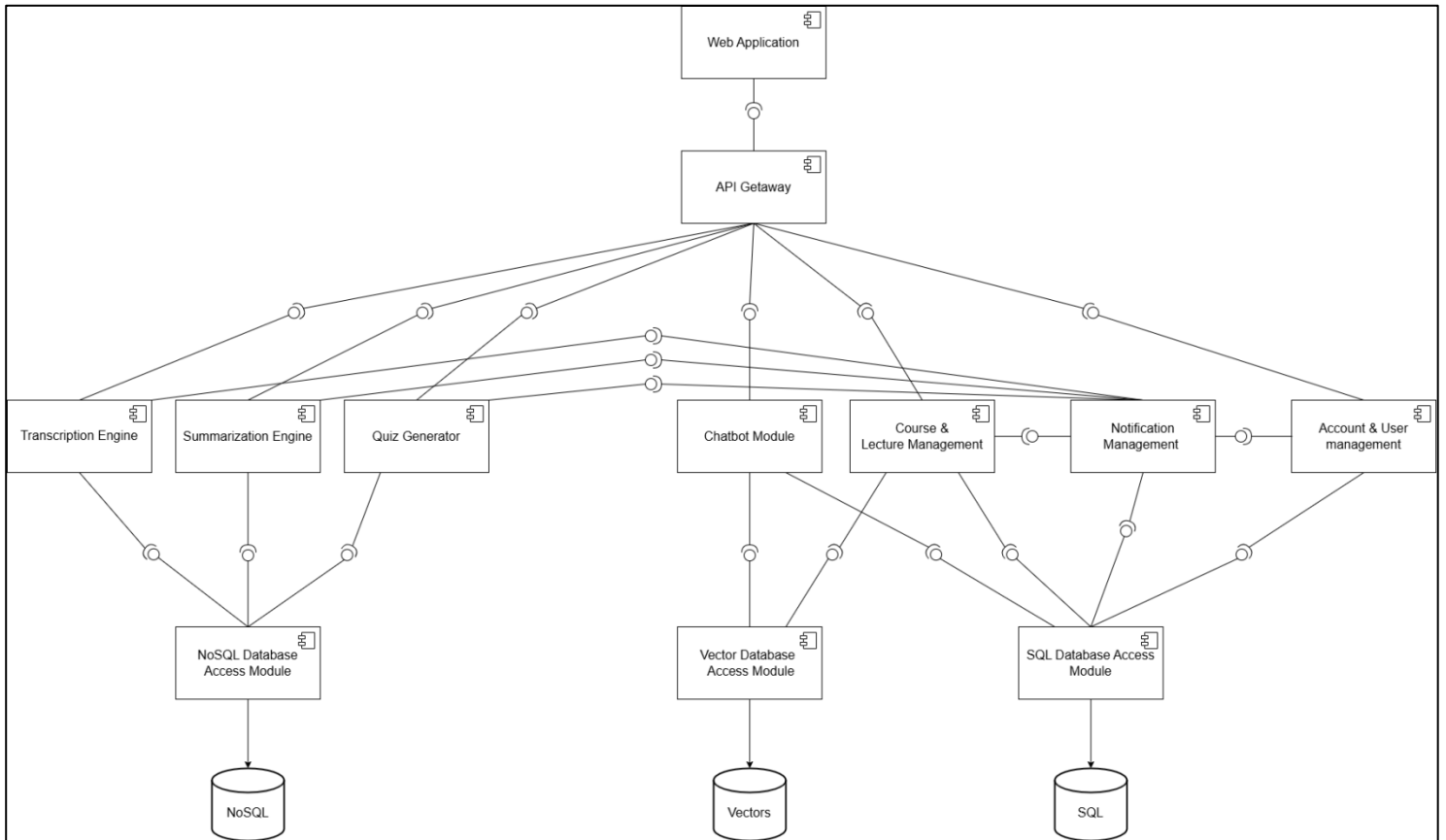


Figure 23 Component diagram

5.3.1.1 Web Application

The Web Application represents the frontend interface used by students and administrators. It allows users to interact with the system features such as authentication, course management, lecture upload, summarization, quiz generation, chatbot interaction, transcription, and notifications.

5.3.1.2 API Gateway

The API Gateway acts as the central entry point between the frontend and backend services. It routes requests to the correct microservice, verifies authentication tokens, forwards user-related headers, and helps centralize communication between system components.

5.3.1.3 Account & User Management

This component is responsible for user registration, login, logout, profile management, and authentication-related operations. It manages user data and provides secure access control using JWT-based authentication.

5.3.1.4 Course & Lecture Management

This component manages courses and lecture materials. It allows students to create, update, delete, and view courses and lectures. It also handles lecture file uploads and provides lecture data to other AI services for processing.

5.3.1.5 Summarization Engine

The Summarization Engine processes extracted lecture text and generates structured summaries using AI models. It helps students review lecture content quickly by producing concise and organized summaries.

5.3.1.6 Quiz Generator

The Quiz Generator creates questions from lecture content using AI. It supports generating quizzes from uploaded materials and helps students test their understanding through automatically generated questions and answers.

5.3.1.7 Chatbot Module

The Chatbot Module provides an AI-powered question-answering interface. It uses lecture content and retrieved context to answer student questions, explain concepts, and support interactive learning.

5.3.1.8 Transcription Engine

The Transcription Engine converts audio lecture content into text using speech-to-text technology. The generated text can then be used by other services such as summarization, quiz generation, and chatbot support.

5.3.1.9 Notification Management

The Notification Management component is responsible for sending system notifications to users. It informs students about important events such as completed summaries, completed transcriptions, quiz status, or other system updates.

5.3.1.10 SQL Database Access Module

This component manages communication with the relational SQL database. It is used by services that store structured data such as users, courses, lectures, notifications, and system records.

5.3.1.11 NoSQL Database Access Module

This component manages communication with the NoSQL database. It stores flexible AI-related artifacts such as extracted text, generated summaries, quiz outputs, and logs.

5.3.1.12 Vector Database Access Module

This component manages communication with the vector database. It stores embeddings generated from lecture chunks and retrieves semantically relevant content for chatbot and retrieval-based AI features.

5.3.1.13 SQL Database

The SQL database stores structured relational data such as user accounts, course records, lecture metadata, and notification records.

5.3.1.14 NoSQL Database

The NoSQL database stores unstructured or semi-structured data such as AI outputs, generated summaries, extracted lecture text, and quiz generation results.

5.3.1.15 Vector Database

The Vector database stores text embeddings and supports semantic search. It allows the chatbot to retrieve the most relevant lecture chunks based on the student's questi

5.4 System Architecture

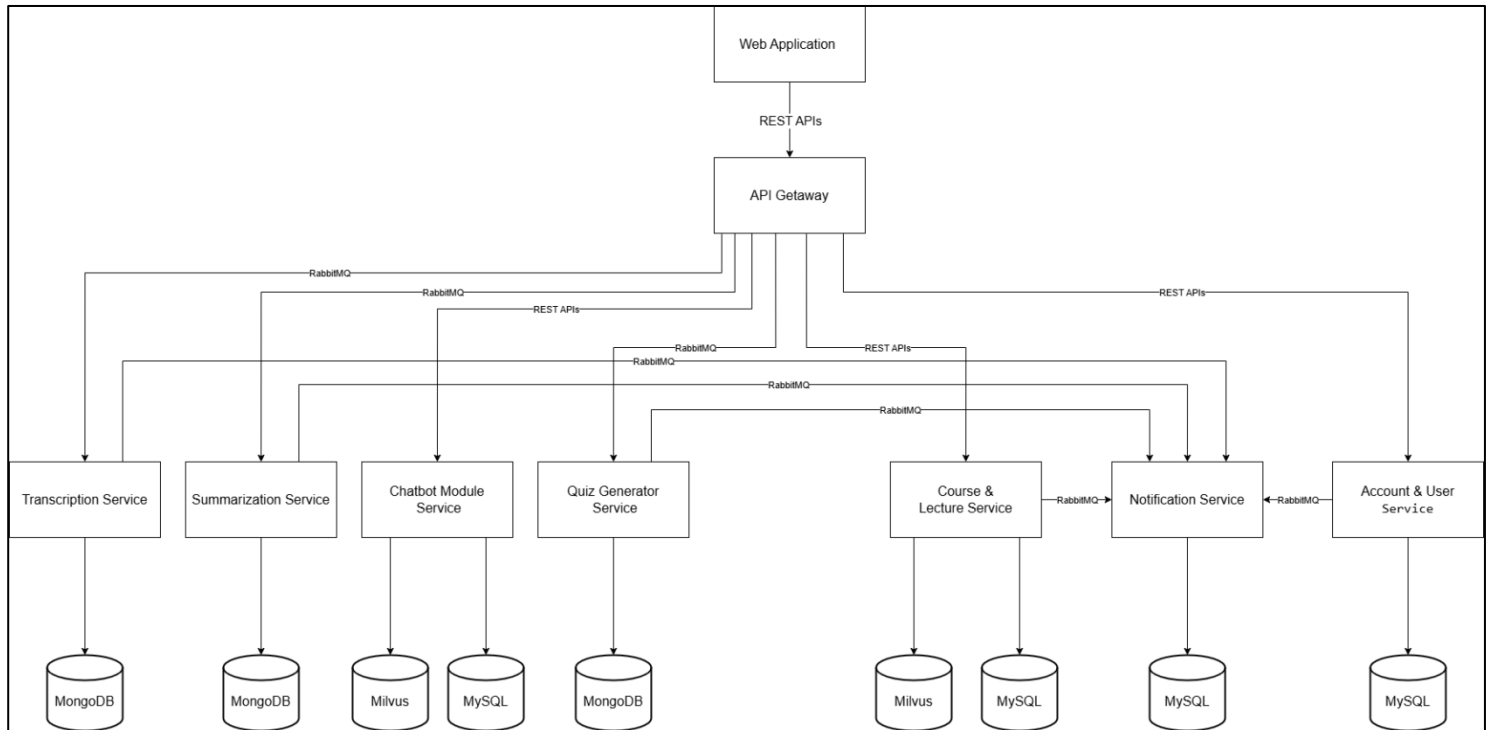


Figure 24 System architecture

Moein follows a microservices-based architecture in which the system is divided into independent services, each responsible for a specific business or AI function. The frontend Web Application communicates with the backend through REST APIs using the API Gateway as the main entry point.

The API Gateway receives requests from the Web Application, validates authentication, and routes each request to the appropriate backend service. This design centralizes request handling and makes the system easier to secure, maintain, and extend.

The backend is composed of several services: Account & User Service, Course & Lecture Service, Notification Service, Chatbot Module Service, Quiz Generator Service, Summarization Service, and Transcription Service. Each service manages its own responsibilities and communicates with other services when needed through REST APIs or RabbitMQ.

RabbitMQ is used as a message broker to support asynchronous communication between services. This is important for long-running operations such as lecture summarization, transcription, quiz generation, and notification creation, because these tasks can run in the background without blocking the user interface.

The system uses multiple databases depending on the type of data. MySQL is used for structured relational data such as users, courses, lectures, and notifications. MongoDB is used for flexible AI-related outputs such as summaries, transcription results, and generated quizzes. Milvus is used as a vector database to store embeddings and support semantic retrieval for chatbot and AI-based search features.

Overall, this architecture improves scalability, maintainability, and separation of concerns. Each service can be developed, tested, and improved independently, while the API Gateway and RabbitMQ provide organized communication between the system components.

5.5 Used design patterns

5.5.1 Facade Design Pattern

5.5.1.1 Pattern Overview

The **Facade Pattern** is a structural design pattern that provides a simplified and unified interface to a complex subsystem. It hides the internal complexity of the system and exposes only the necessary functionality to the client.

5.5.1.2 Application of the Facade Pattern in the System

In this system, the **API Gateway** implements the **Facade Pattern**.

Instead of allowing the Web Application to communicate directly with multiple backend microservices, the API Gateway acts as a single, unified interface. The client sends all requests to the API Gateway, which then internally forwards each request to the appropriate microservice such as Account & User Management, Course & Lecture Management, Summarization Engine, or Quiz Generator.

The client is therefore unaware of:

- The number of backend services.
- Their internal structure.
- Their communication mechanisms.

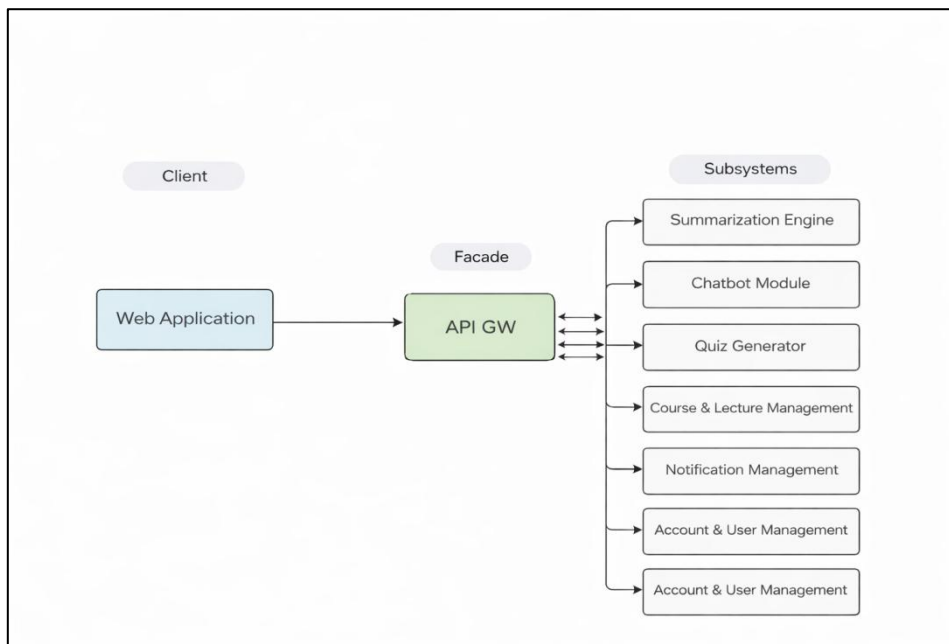


Figure 25 Facade pattern applied using the API Gateway.

5.5.2 Template Method Design Pattern

5.5.2.1 Pattern Overview

The **Template Method Pattern** is a behavioral design pattern that defines the skeleton of an algorithm in a base class while allowing subclasses to implement specific steps of the algorithm without changing its overall structure.

5.5.2.2 Application of the Template Method Pattern in the System

In this system, the **Template Method pattern** is applied in the **file parsing process** for lecture files.

Lecture files may be provided in different formats such as PDF, DOCX, or PDF. Although the overall parsing workflow remains the same, the **file parsing and text extraction step differs based on the file format**. A common parsing structure is defined, while format-specific parsing logic is implemented in specialized parsers.

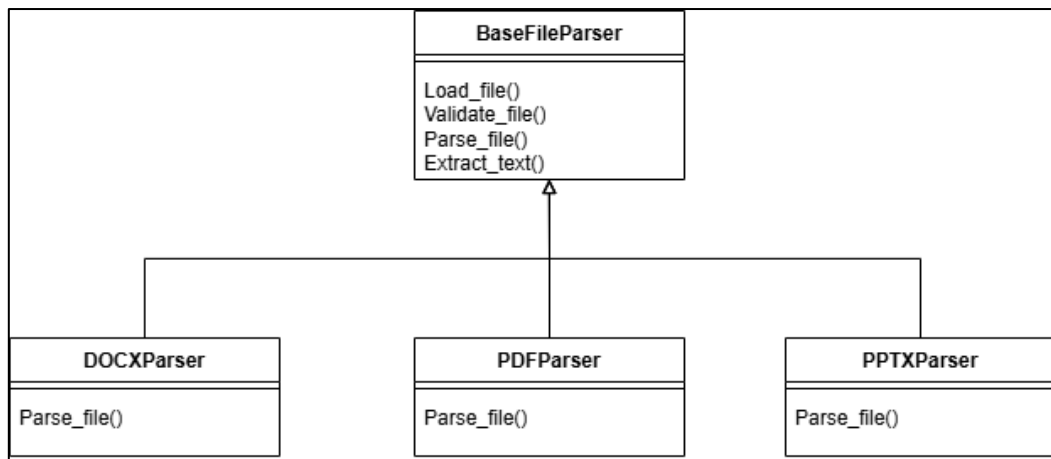


Figure 26 Template Method pattern for lecture file parsing.

5.6 Database Design

5.6.1 NoSQL Database – MongoDB: For data related to user reviews and AI algorithm results, we employ MongoDB. This NoSQL database is well-suited for handling large volumes of unstructured data, providing the flexibility and scalability needed to manage diverse and dynamic datasets efficiently.

Review document structure:

```

_id: ObjectId('6950e1e92af1b86d85f0e8a7')
lecture_id: "8253cdc1-67a3-40e9-8d74-3627e9a66dae"
created_at: 2025-12-28T07:53:13.528+00:00
status: "READY"
summary_text: "**Lecture 06: Pull Requests**
              *Eng. Raghad al-Hossny*

              A **pull requ..."
updated_at: 2025-12-28T07:53:13.528+00:00
  
```

Figure27 MongoDB document structure

5.6.2 SQL Database – MySQL: In our system, MySQL is utilized for storing operational data related to accounts, projects, and requirements. This data is highly structured, making MySQL an ideal choice due to its efficiency in handling defined schemas and transactional integrity.

MySQL- database modeling:

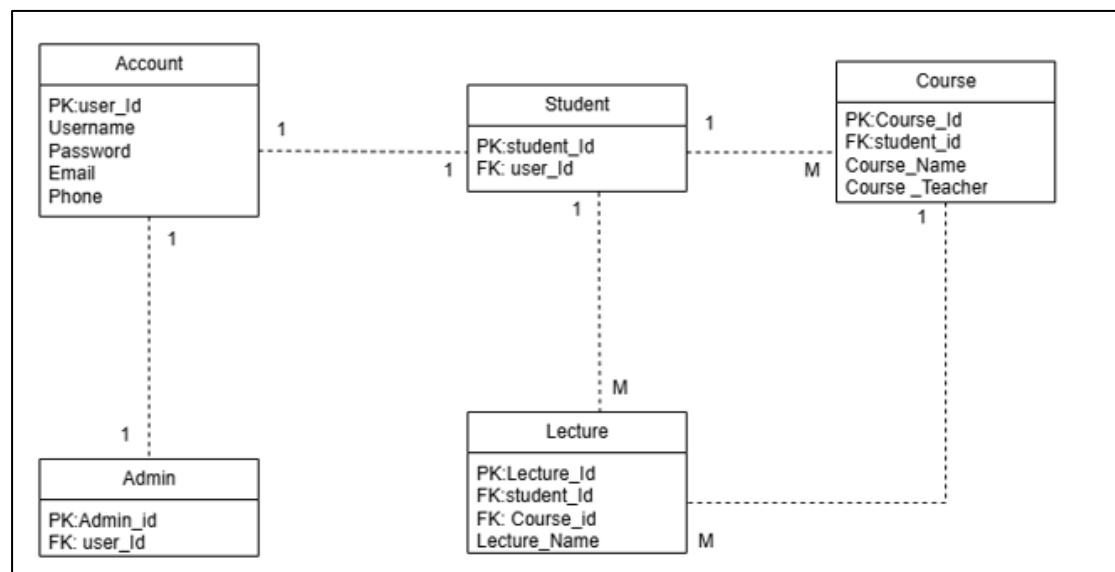


Figure 28 MySQL database modeling

5.7 Updating the RTM

Table 40 Updated RTM

REQ-ID	Use cases	analysis	System design	Detailed design	coding	Test cases
RE-FR-SU-01	UC-01	analysis	System design	Detailed design		TC-01
						TC-02
RE-FR-LI-02	UC-02	analysis	System design	Detailed design		TC-03
						TC-04
RE-FR-MA-03	UC-03	analysis	System design	Detailed design		TC-05
						TC-06
						TC-07
RE-FR-GQ-04	UC-04	analysis	System design	Detailed design		TC-08
						TC-09
						TC-10
						TC-11
						TC-12
						TC-13
						TC-14
RE-FR-AEA-05	UC-04	analysis	System design	Detailed design		TC-15
						TC-16
						TC-17
RE-FR-U&PL-06	UC-05	analysis	System design	Detailed design		TC-18
						TC-43
						TC-44
						TC-45
						TC-46

RE-FR-ELI-07	UC-06	analysis	System design	Detailed design		TC-34 TC-35 TC-36
RE-FR-SLC-08	UC-07	analysis	System design	Detailed design		TC-38 TC-39 TC-40
RE-FR-DL-09	UC-08	analysis	System design	Detailed design		TC-37
RE-FR-CC-10	UC-09	analysis	System design	Detailed design		TC-27 TC-28
RE-FR-DC-11	UC-10	analysis	System design	Detailed design		TC-31
RE-FR-ECI-12	UC-11	analysis	System design	Detailed design		TC-29 TC-30
RE-FR-VL-13	UC-12	analysis	System design	Detailed design		TC-32 TC-33
RE-FR-AUF-14	UC-13	analysis	System design	Detailed design		TC-19 TC-20 TC-21 TC-22 TC-23
RE-FR-GSUA1-15	UC-14	analysis	System design	Detailed design		TC-24
RE-FR-ESPFD-16	UC-15	analysis	System design	Detailed design		TC-25 TC-26

Chapter 6 Account & User Service

6.1 Account & User Service Overview

The **Account & User Service** is a core microservice responsible for managing user accounts and identity-related operations within the system. It handles user registration, authentication support, and user profile management. By encapsulating these responsibilities within a dedicated service, the system achieves improved modularity, security, and scalability.

In the adopted microservices architecture, the Account & User Service serves as the central authority for user and account data. Other services rely on it to validate user identity and permissions, rather than storing sensitive user information locally. This design reduces duplication, enhances security, and enforces consistent identity management across the system.

Separating account and user management into an independent service also allows authentication mechanisms and user-related logic to evolve without affecting other system components.

6.2 Purpose of the Account & User Service

The purpose of the Account & User Service is to provide a secure, reliable, and centralized mechanism for managing user accounts and identities. It ensures that only authenticated and authorized users can access system resources while protecting sensitive data such as passwords and personal information.

The service is designed to:

- Centralize user and account management.
- Enforce authentication and authorization policies.
- Support other microservices with identity verification.
- Maintain integrity and consistency of user data.

6.3 Responsibilities of the Account & User Service

The Account & User Service is responsible for the following key functionalities:

- **User Registration**
Allows new users to create accounts by submitting required information such as username, email address, and password.
- **Authentication Support**
Validates user credentials during login and supports token-based authentication mechanisms used across the system.
- **User Profile Management**
Enables users to view and update their personal information while enforcing access control rules.
- **Password Management**
Handles password creation, updates, and resets using secure hashing techniques.

6.4 Functional Requirements

The Account & User Service must satisfy the following functional requirements:

- The system shall allow users to register new accounts.
- The system shall validate user credentials during login.
- The system shall allow users to view and update their profile information.
- The system shall securely manage user passwords.
- The system shall prevent duplicate accounts using the same email or username.
- The system shall allow authorized actions such as account deactivation or deletion.

6.5 API Endpoints

The Account & User Service exposes a set of RESTful API endpoints that allow clients and other microservices to interact with user account and identity data. These endpoints are designed following standard HTTP conventions and return meaningful status codes.

Table 41 Main API Endpoints

HTTP Method	Endpoint	Description
POST	/api/signup/	Creates a new user account and associated student profile.
POST	/api/login/	Authenticates a user and generates a JWT token for accessing protected services.
PUT	/api/account/edit/	Allows an authenticated user to update their account information.
DELETE	/api/account/delete/	Deletes the authenticated user account and removes all related courses through the Course Service.
GET	/api/account/me/	Retrieves information about the currently authenticated user and their student profile.

Each endpoint validates input data, enforces authorization rules, and returns appropriate responses based on the operation outcome.

6.6 Security Considerations

Security is a critical aspect of the Account & User Service due to the sensitive nature of user data.

The service implements the following security measures:

- **Password Hashing:**
User passwords are hashed using strong cryptographic algorithms before storage.
- **Token-Based Authentication:**
Authentication tokens are issued upon successful login and used to authorize subsequent requests.
- **Input Validation:**
User inputs are validated to prevent injection attacks and malformed requests.

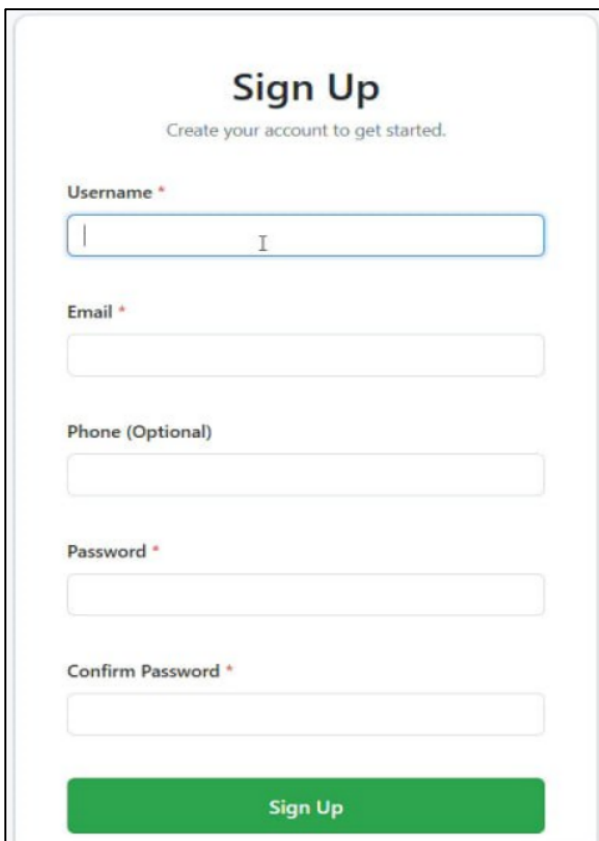
6.7 Technology Stack

The Account & User Service is implemented using the following technologies:

- Backend Framework: (Django).
- Backend Framework: (React).
- Database: (MySQL).
- Authentication: JWT-based authentication.
- Communication Protocol: RESTful APIs.

This technology stack ensures scalability, security, and ease of maintenance.

6.8 Service screenshots



Sign Up
Create your account to get started.

Username *

Email *

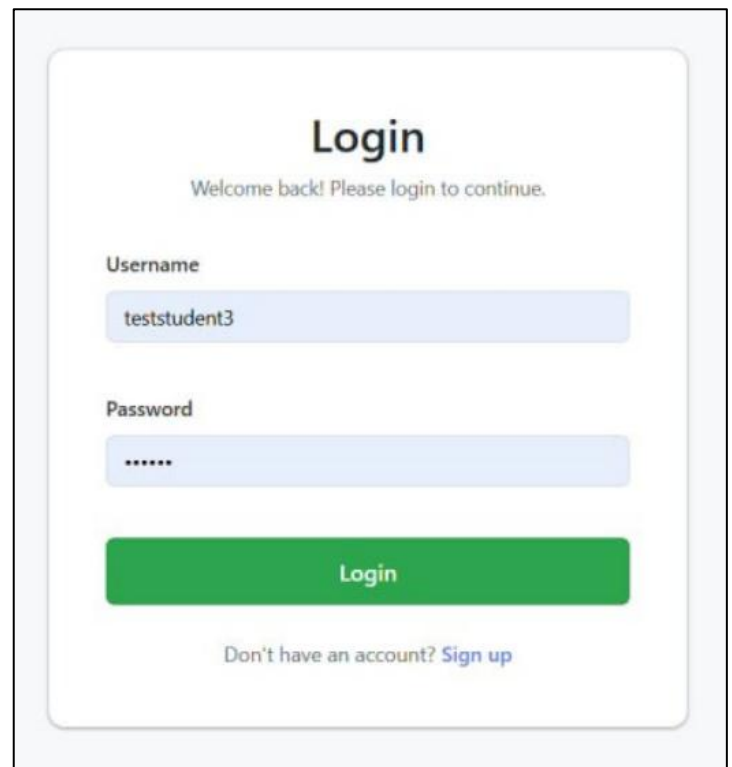
Phone (Optional)

Password *

Confirm Password *

Sign Up

Figure29 Signup screenshot



Login
Welcome back! Please login to continue.

Username

Password

Login

Don't have an account? [Sign up](#)

Figure30 Login screenshot

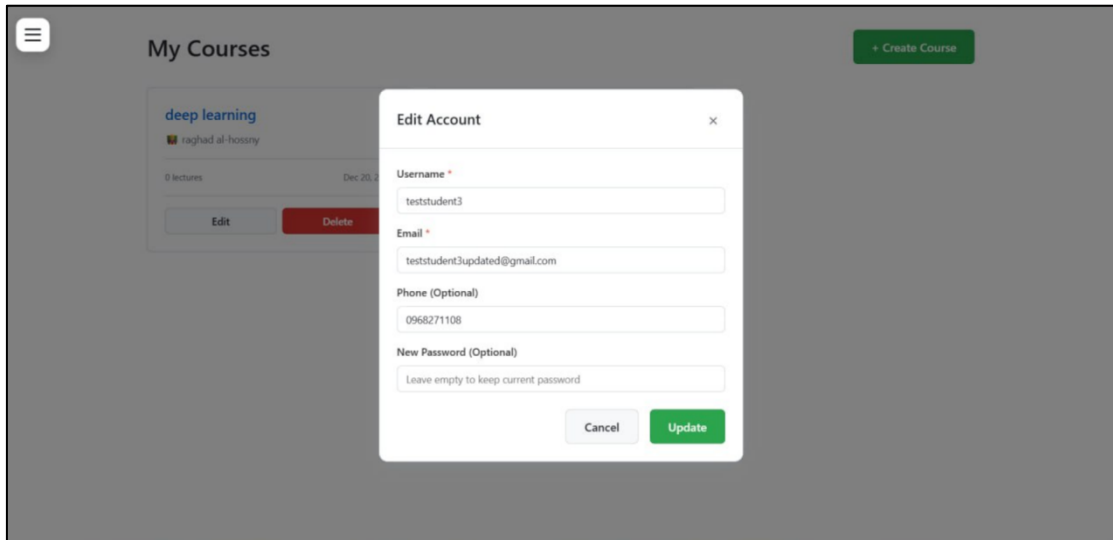


Figure31 Edit account screenshot

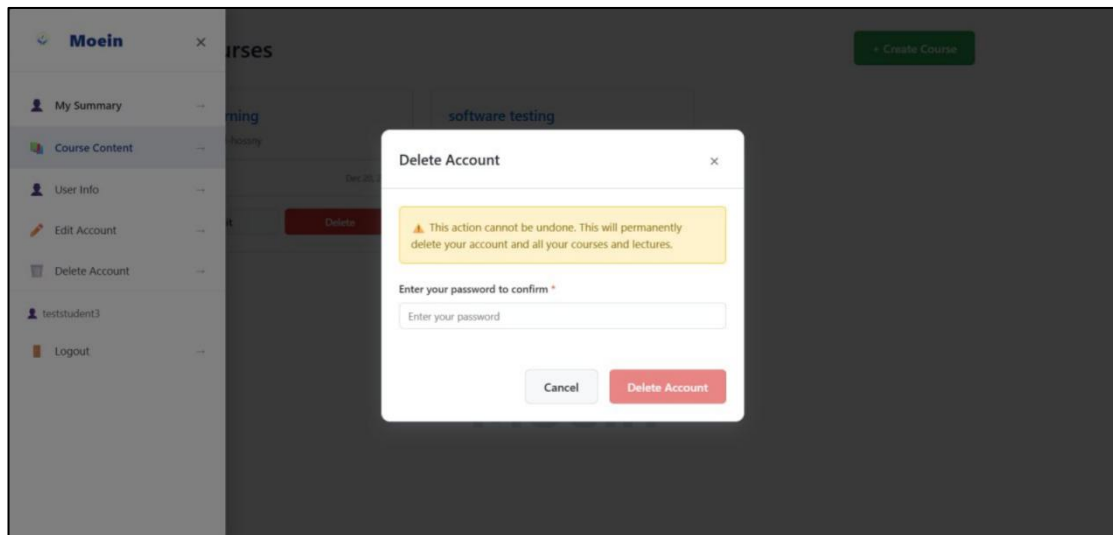


Figure32 Delete account screenshot

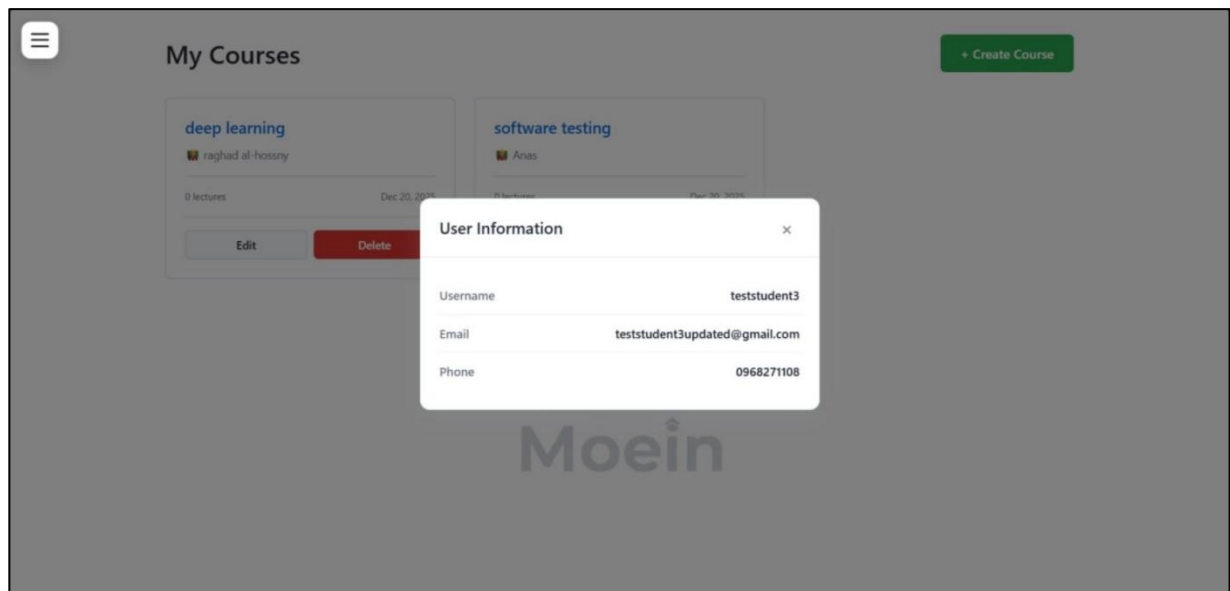


Figure33 user info screenshot

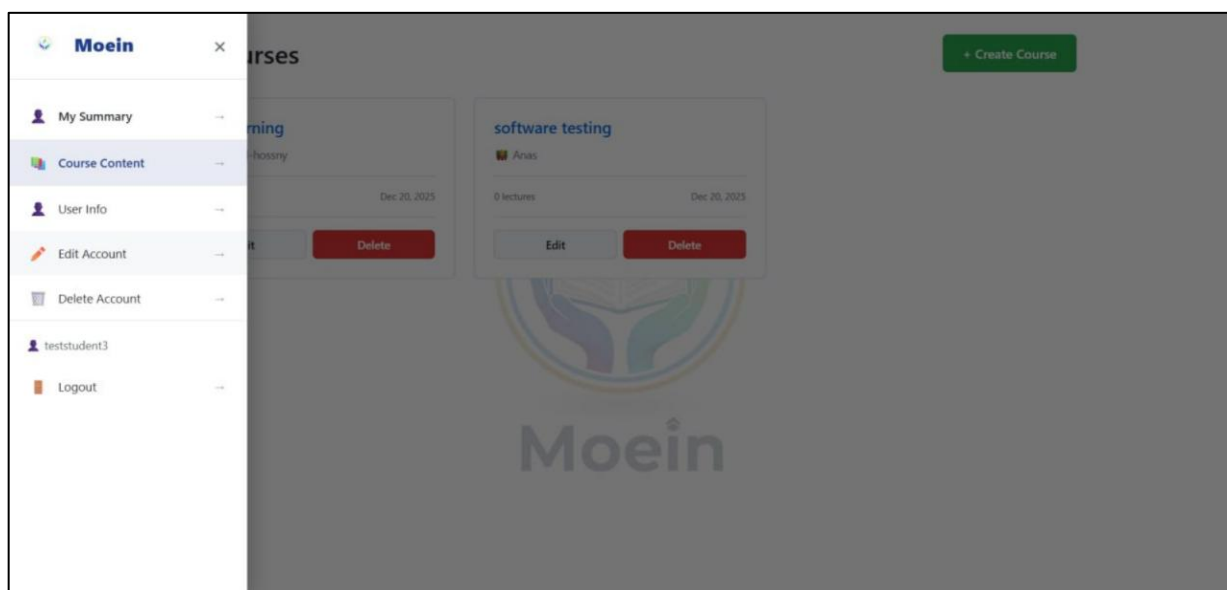


Figure34 side bar screenshot

6.9 Conclusion

The Account & User Service plays a vital role in managing user identities and ensuring secure access to system resources. By separating account and user management into a dedicated microservice, the system benefits from improved modularity, enhanced security, and better scalability.

The service provides a reliable foundation for authentication and authorization while supporting seamless integration with other microservices in the system architecture.

Chapter 7 Course & Lecture Service

7.1 Course & Lecture Service Overview

The **Course & Lecture Service** is a core microservice responsible for managing courses and their associated lectures within the system. It handles the creation, modification, organization, and deletion of courses and lectures, as well as managing lecture-related content. By encapsulating course and lecture management into a dedicated service, the system achieves better modularity, scalability, and maintainability.

Within the adopted microservices architecture, the Course & Lecture Service acts as the authoritative source for course structures and lecture data. Other services interact with it to retrieve course information, access lecture content, or perform course-related operations without directly managing course data themselves. This approach reduces tight coupling and ensures consistency across the system.

Separating course and lecture functionality into an independent service allows the system to evolve its educational content management features without impacting authentication, user management, or other unrelated services.

7.2 Purpose of the Course & Lecture Service

The purpose of the Course & Lecture Service is to provide a centralized, reliable, and scalable mechanism for managing educational content in the form of courses and lectures. It ensures that courses and lectures are properly organized, accessible to authorized users, and consistently maintained throughout the system lifecycle.

The service is designed to:

- Centralize course and lecture management.
- Organize lectures under their respective courses.
- Support content-related operations for other microservices.
- Maintain integrity and consistency of course and lecture data.

7.3 Responsibilities of the Course & Lecture Service

The Course & Lecture Service is responsible for the following key functionalities:

- **Course Management**
Allows users to create, update, view, and delete courses, including managing course metadata such as title and teacher name.
- **Lecture Management**
Enables the creation, editing, and deletion of lectures within a specific course.
- **Lecture Content Handling**
Manages lecture content such as uploaded files or learning materials associated with each lecture.
- **Course–Lecture Association**
Ensures that each lecture is correctly linked to its corresponding course.
- **Access Control Enforcement**
Ensures that only authorized users can modify or access course and lecture resources.

7.4 Functional Requirements

The Course & Lecture Service must satisfy the following functional requirements:

- The system shall allow users to create new courses.
- The system shall allow users to update or delete existing courses.
- The system shall allow users to add lectures to a specific course.
- The system shall allow users to update or delete lectures.
- The system shall allow users to retrieve courses and their associated lectures.
- The system shall prevent unauthorized access to course and lecture resources.

7.5 API Endpoints

The Course & Lecture Service exposes a set of RESTful API endpoints that allow clients and other microservices to manage courses and lectures. These endpoints follow standard HTTP conventions and return appropriate status codes.

Table 42 Main API Endpoints

HTTP Method	Endpoint	Description
POST	/api/courses/create/	Create a new course for the authenticated student.
GET	/api/courses/	Get all courses for the authenticated student.
GET	/api/courses/<course_id>/	Get a specific course owned by the student.
GET	/api/courses/<course_id>/lectures/	Get all lectures for a specific course with file URLs.
PUT	/api/courses/<course_id>/	Update a course's name or teacher (student-owned).
DELETE	/api/courses/<course_id>/	Delete a course and all its lectures/files.
POST	/api/courses/<course_id>/lectures/upload/	Upload a lecture file to a course and trigger AI summarization.
PUT	/api/courses/<course_id>/lectures/<lecture_id>/update-name/	Update the name of a lecture in a course.
DELETE	/api/courses/<course_id>/lectures/<lecture_id>/delete/	Delete a lecture and its physical file.
DELETE	/api/admin/delete_student_courses/<student_id>/	Delete all courses for a student (admin/internal).
GET	/api/lectures/<lecture_id>/	Get a single lecture with file URL and course info; polls for AI summary readiness.
GET	/api/lectures/<lecture_id>/summary/	Get lecture summary from AI summarization service.

Each endpoint validates input data, enforces authorization rules, and returns appropriate responses based on the operation outcome.

7.6 Security Considerations

Security is an important aspect of the Course & Lecture Service to ensure that educational content is protected from unauthorized access or modification.

The service implements the following security measures:

- **Token-Based Authorization:**
Access to course and lecture endpoints is restricted to authenticated users through token validation.
- **Access Control:**
Users are allowed to modify or delete only their own courses and lectures.
- **Input Validation:**
All incoming data is validated to prevent malformed requests and data inconsistency.

7.7 Technology Stack

The Course & Lecture Service is implemented using the following technologies:

- **Backend Framework:** (Django).
- **Backend Framework:** (React).
- **Database:** MySQL.
- **Authentication:** JWT-based authentication (validated via Account & User Service).
- **Communication Protocol:** RESTful APIs.

This technology stack ensures efficient content management, scalability, and seamless integration with other services.

7.8 Service screenshots

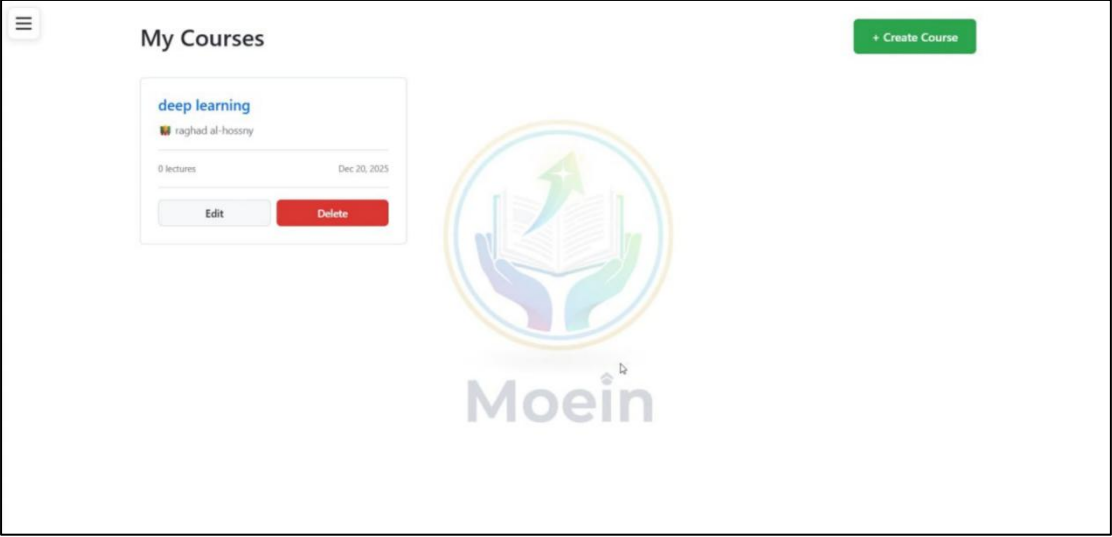


Figure35 Courses list screenshot

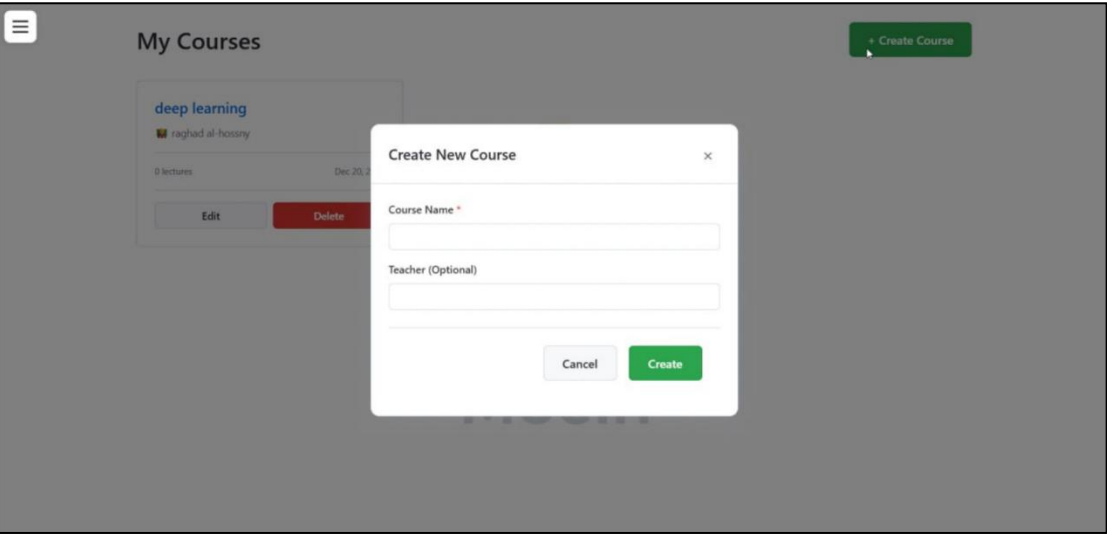


Figure36 Create course screenshot

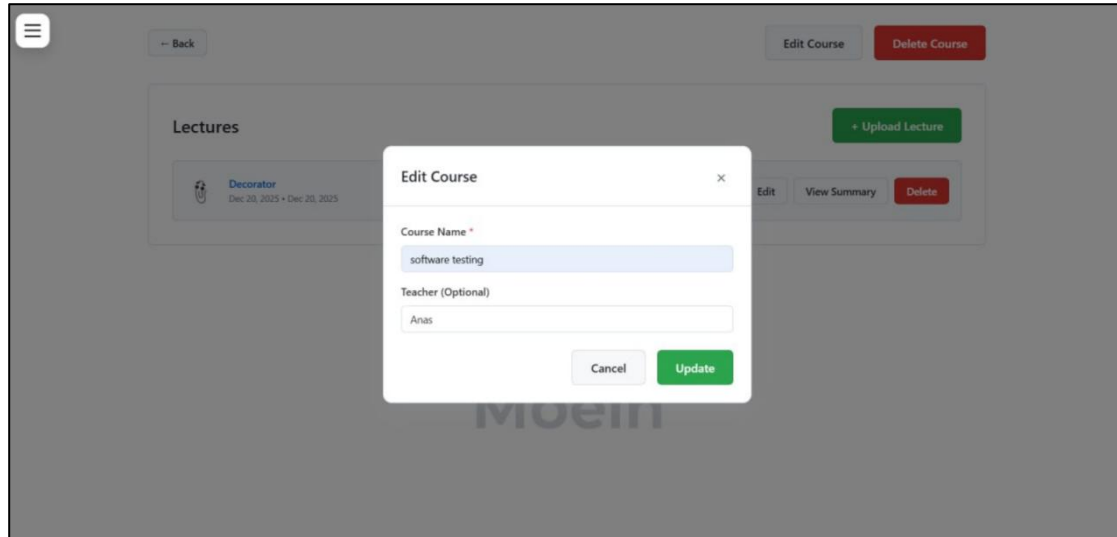


Figure37 Edit course info screenshot

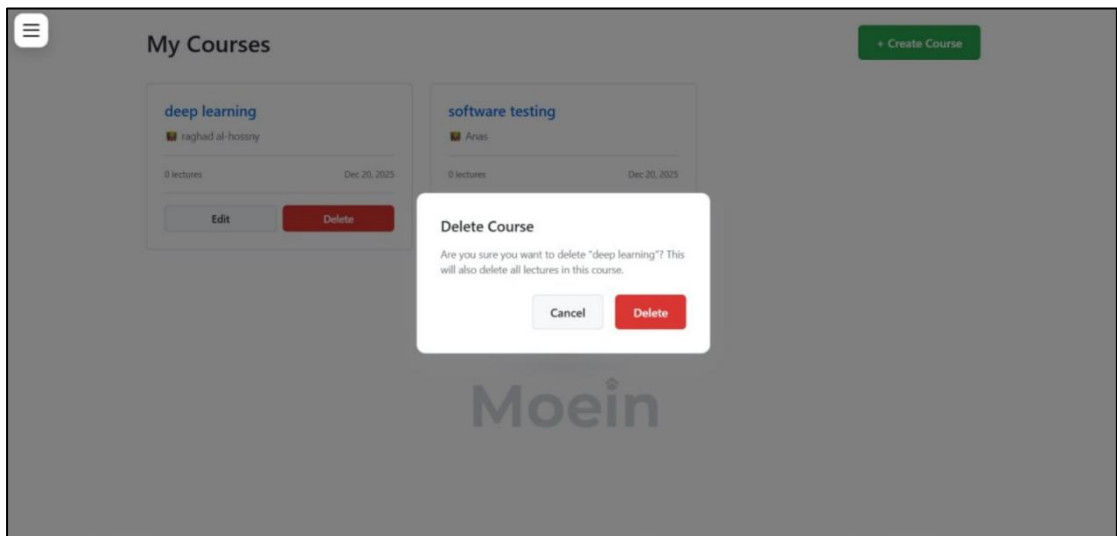


Figure38 Delete course screenshot

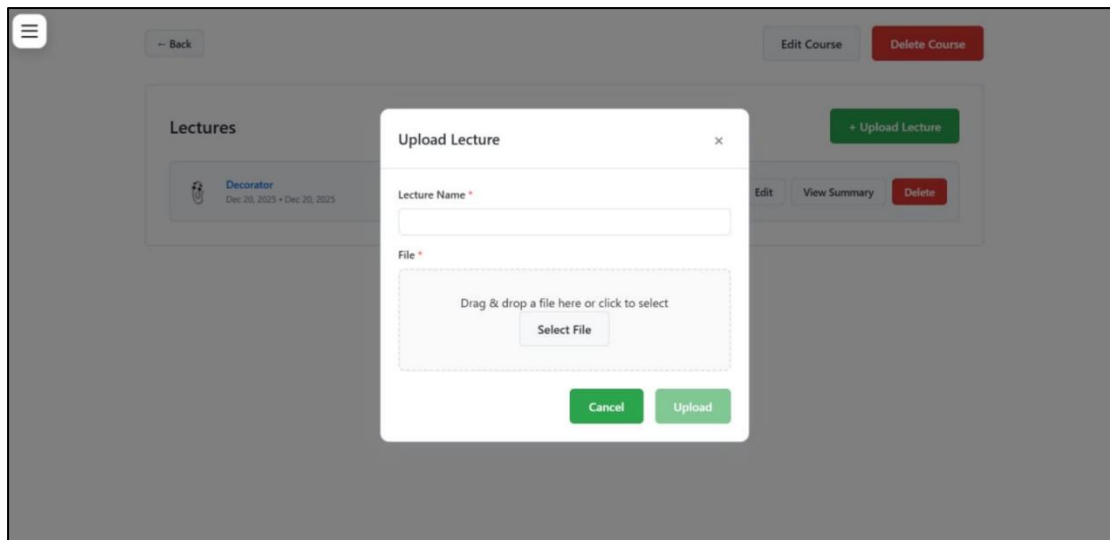


Figure39 upload lecture screenshot

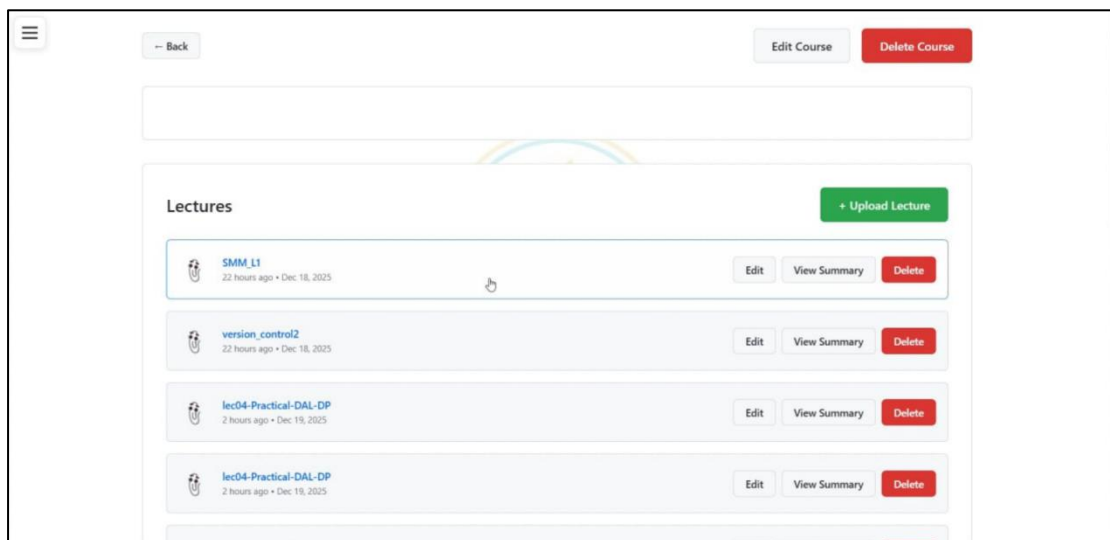


Figure40 Lectures list screenshot

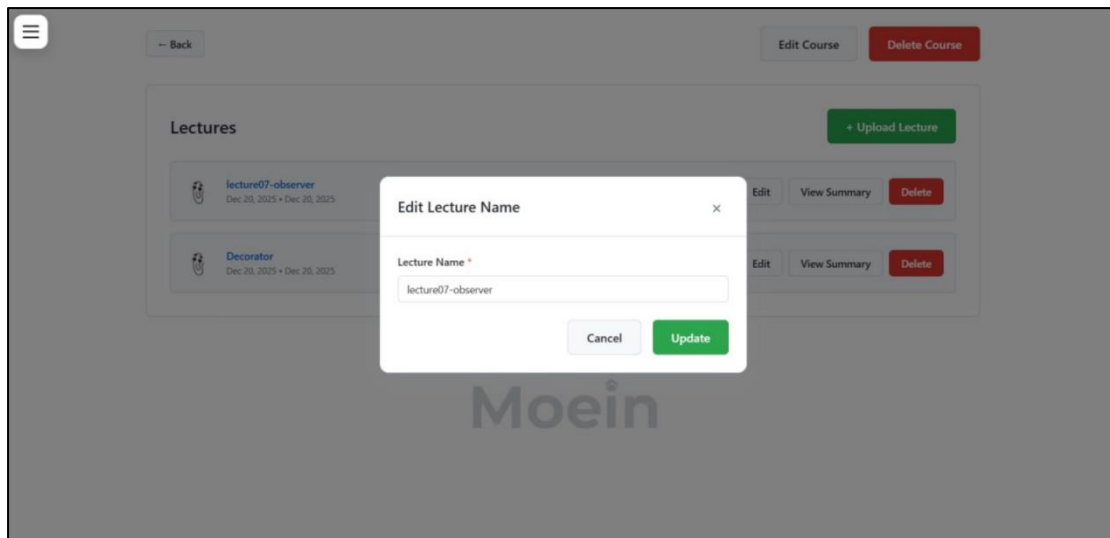


Figure41 Edit lecture name screenshot

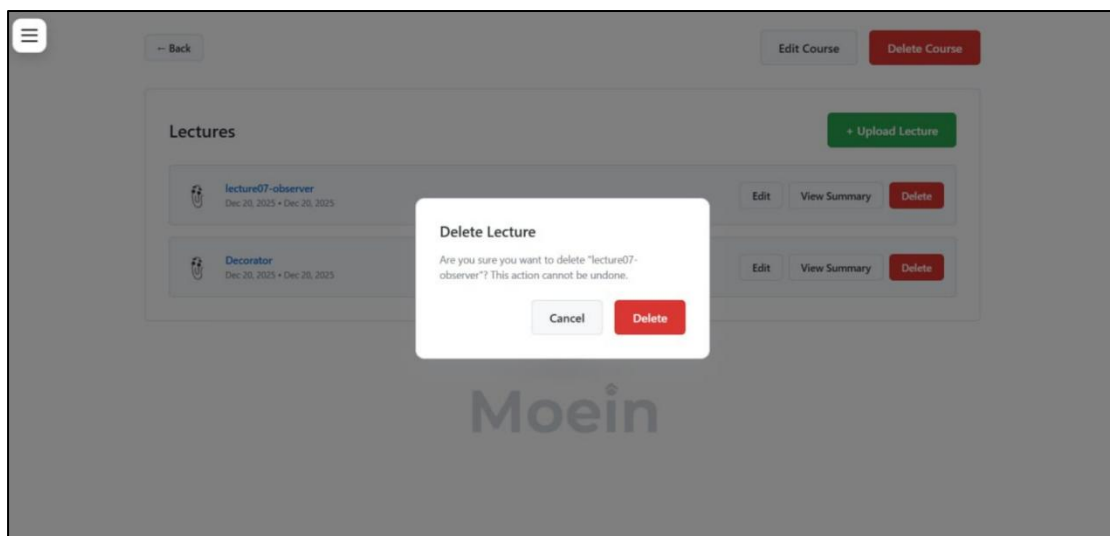


Figure42 Delete lecture screenshot

7.9 Conclusion

The Course & Lecture Service plays a central role in managing the educational content of the system. By separating course and lecture functionality into a dedicated microservice, the system benefits from improved modularity, easier maintenance, and better scalability.

The service provides a structured and reliable foundation for organizing courses and lectures while supporting smooth interaction with other microservices in the overall system architecture.

Chapter 8 AI Service

8.1 Introduction

Artificial Intelligence (AI) is increasingly transforming the education sector by automating content processing, enhancing learning efficiency, and supporting students in managing large volumes of academic material. University lectures often contain dense and lengthy information, making it difficult for students to manually extract key concepts and essential points in a limited time.

The AI Processing Service in the proposed Student Course Management and AI Summarization System addresses this challenge by automatically generating structured summaries of uploaded lecture materials. The service leverages AI-based content analysis and summarization techniques to extract main topics, key points, and important concepts from lecture content.

Implemented as an independent microservice, the AI Processing Service operates asynchronously and is triggered after a lecture is uploaded. This design ensures scalability and prevents blocking other system components. By automating the summarization process, the service significantly reduces students' study time and improves content comprehension, thereby enhancing the overall learning experience.

- **Key Functionalities of AI Service:**

The AI service processes uploaded lecture content through a set of core components designed to automatically generate meaningful academic summaries. These components work together to transform raw lecture material into structured and easy-to-understand outputs.

- **Content Processing and Text Extraction:**
The service analyzes uploaded lecture files or recordings and extracts textual content in a machine-readable format suitable for further analysis.
- **Summarization:**
The AI service generates concise and well-structured summaries by identifying the main topics and important sections within the lecture content. This enables students to quickly review essential information without revisiting the entire lecture.
- **Key Concepts Extraction:**
The service identifies essential concepts, definitions, and important points discussed during the lecture, helping students focus on critical academic material and improve knowledge retention.

In this chapter, the key tools and technologies utilized in the AI Processing Service are presented. In addition, relevant literature related to AI-based content summarization is reviewed, followed by a detailed discussion of the design and testing phases of the AI Processing Service within the proposed system.

8.2 Basic Concepts and literature review

8.2.1 Introduction:

This section provides an overview of the key concepts, technologies, and tools that are essential for the AI Processing Service. It also includes a review of relevant literature on similar models. By examining these foundational elements, the chapter highlights the significance of the proposed service and demonstrates how it enhances existing solutions in lecture summarization and content processing.

8.2.2 Basic concepts:

- AI pre-trained models:

AI pre-trained models have revolutionized the way artificial intelligence is developed and deployed across various industries. These models are trained on massive datasets, enabling them to understand patterns, recognize structures, and generate intelligent responses without the need for training from scratch. Instead of spending significant time, computational resources, and large amounts of labeled data to develop models from the ground up, pre-trained models offer a ready-to-use foundation that can be fine-tuned for specific tasks. Leveraging pre-trained models improves efficiency, accuracy, and scalability while significantly reducing the cost of AI development. This approach allows developers to focus on customization and optimization rather than starting with raw data.

- AI summarization:

AI-Based Summarization in this system leverages the Qwen/Qwen3-30B-A3B-Instruct-2507 model to automatically generate concise and meaningful summaries of lecture content. This advanced large language model is specifically designed for instruction-following tasks, making it well-suited for educational applications where structured, coherent summaries are required.

8.2.3 Literature review:

This section provides a comprehensive analysis of pre-trained AI models and existing text-to-text solutions, focusing on their applicability to the core functionalities of the AI Processing Service. Various summarization models are examined, with an evaluation of their strengths, limitations, and alignment with the specific requirements of the lecture summarization system. The analysis highlights the rationale behind

selecting Qwen/Qwen3-30B-A3B-Instruct-2507 for generating structured and coherent text summaries, ensuring high-quality output tailored for educational use.

- text to text models:

For text-to-text component, several models from different families will be compared.

openai/gpt-oss-120b:groq:

This is a large-scale language model with approximately 120 billion parameters. It is designed for general-purpose tasks, including text generation, reasoning, and complex instruction following. The “groq” variant is optimized for high-performance hardware to speed up inference. It is highly flexible and powerful but requires substantial computational resources, making it more suitable for large-scale industrial applications rather than light academic projects.

google/gemma-2-2b-it:nebius:

This is a lightweight language model with around 2 billion parameters. It is part of the Gemma/Gemini series and is primarily designed for instruction-following tasks. Its small size makes it fast and efficient on limited hardware, but it may struggle with understanding long or complex academic texts. It is more suitable for simple text generation or short-form tasks rather than processing lengthy lecture notes.

deepseek-ai/DeepSeek-R1:novita:

DeepSeek-R1 is a language model designed with a focus on reasoning and analytical tasks. It excels at complex problem-solving, logical inference, and structured analysis. While it performs well on tasks requiring deep understanding and reasoning, it may be slower and less efficient when handling large volumes of text for summarization purposes. It is best suited for projects that require critical thinking or technical analysis rather than casual text summarization.

Qwen/Qwen3-Coder-480B-A35B-Instruct:novita

This is an extremely large language model with around 480 billion parameters, specifically optimized for coding and agentic tasks. It excels in code generation, debugging, and reasoning in technical workflows. While powerful, it is not primarily designed for summarizing academic texts and may be excessive (“overkill”) for projects focused on lecture summarization. Its computational requirements are very high, making it suitable mainly for specialized industrial or research use.

zai-org/GLM-4.5:novita

GLM-4.5 is a very large Mixture-of-Experts (MoE) language model, with approximately 355 billion parameters in total, and around 32 billion active parameters used per inference. It is designed for general-purpose reasoning, text generation, and complex tasks requiring deep understanding. It supports very long contexts, making it suitable for processing extensive documents. However, it requires substantial computational resources, so it may not be practical for small academic projects.

Qwen/Qwen3-4B-Thinking-2507:nscale

Qwen3-4B-Thinking is a lightweight model with approximately 4 billion parameters, designed with enhanced reasoning capabilities. It is suitable for tasks that require logical thinking or basic analysis. Its small size makes it fast and efficient, but it may lack the depth needed for summarizing long and complex academic lectures. It is better suited for short texts or preliminary summarization.

Qwen/Qwen2.5-7B-Instruct-1M:featherless-a

This is a medium-sized general-purpose instruct model with approximately 7 billion parameters. It is designed to follow instructions for text generation and simple analytical tasks. While it is more capable than smaller models, it may still struggle with long academic texts and complex lecture materials. It is suitable for moderate summarization tasks but may not capture all intricate details in lengthy documents.

Qwen/Qwen2.5-Coder-32B-Instruct:nscale

Qwen2.5-Coder-32B is a large model with approximately 32 billion parameters, specifically optimized for coding tasks. It excels at code generation, debugging, and technical reasoning. While it can process text, it is not primarily designed for summarizing academic lectures, and using it for this purpose may be inefficient. It is best suited for programming-related projects rather than general text summarization.

Qwen/Qwen3-30B-A3B-Instruct-2507:nebius

Qwen3-30B-A3B-Instruct-2507 is a large Mixture-of-Experts (MoE) model with a total of approximately 30 billion parameters and around 3.3 billion active parameters per inference. It is specifically optimized for general-purpose instruction-following tasks, text summarization, and reasoning. It supports long contexts, making it highly suitable for processing and summarizing lengthy academic lectures. It offers a balanced combination of performance and computational efficiency, making it the most practical choice for lecture summarization projects.

Table 43 Model Comparison for Lecture Summarization

Model Name	Size	Model Type	Strength	Weaknesses	Suitability for Lecture Summarization Project
google/gemma-2-2b-it	2B	Instruct (Lightweight)	Fast, low resource usage	Weak understanding of long texts	Not suitable
Qwen/Qwen3-4B-Thinking-2507	4B	Reasoning / Thinking	Good logical reasoning, lightweight	Limited summarization quality	Limited
Qwen/Qwen2.5-7B-Instruct-1M	7B	General Instruct	Stable, moderate performance	Not sufficient for long academic lectures	Acceptable
Qwen/Qwen2.5-Coder-32B-Instruct	32B	Coder (Programming)	Strong in code and technical reasoning	Not designed for academic text summarization	Not suitable
deepseek-ai/DeepSeek-R1	Large	Reasoning-focused	Deep analytical and reasoning abilities	Slower, not optimized for summarization	Acceptable with limitations
openai/gpt-oss-120b	120B	General / Agentic	Very strong overall understanding	High cost, impractical for this task	Overkill
zai-org/GLM-4.5	355B	General / Reasoning	Excellent accuracy, deep comprehension	Extremely resource-intensive	Overkill

Qwen/Qwen3-Coder-480B-A35B	480B	Coder	Outstanding for coding and agent workflows	Not suitable for text summarization	Not suitable
Qwen/Qwen3-30B-A3B-Instruct-2507	30B	General Instruct	Excellent balance, strong summarization, long context	Requires moderate resources	Best choice

8.3 Services Design and implementation

8.3.1 Overview

This section describes the design and implementation of the AI-related services in Moein. These services are responsible for processing lecture materials, generating summaries and quizzes, supporting chatbot-based question answering, transcribing audio files, and notifying students about completed background tasks.

The AI services are implemented as independent microservices and communicate with the rest of the system through REST APIs and asynchronous messaging using RabbitMQ. This design allows long-running AI operations to be executed in the background without blocking the frontend user experience.

8.3.2 Summarization Service

8.3.2.1 Purpose

The Summarization Service generates structured summaries from uploaded lecture content. After a lecture file is uploaded and its text is extracted by the Course & Lecture Service, the extracted text is sent to the Summarization Service for AI-based processing.

8.3.2.2 Responsibilities

The Summarization Service is responsible for:

- Receiving extracted lecture text.
- Storing lecture processing requests.
- Running AI summarization using the selected LLM.
- Saving generated summaries.
- Returning summary status and result to the frontend.
- Supporting asynchronous processing for long-running tasks.

8.3.2.3 Service Flow

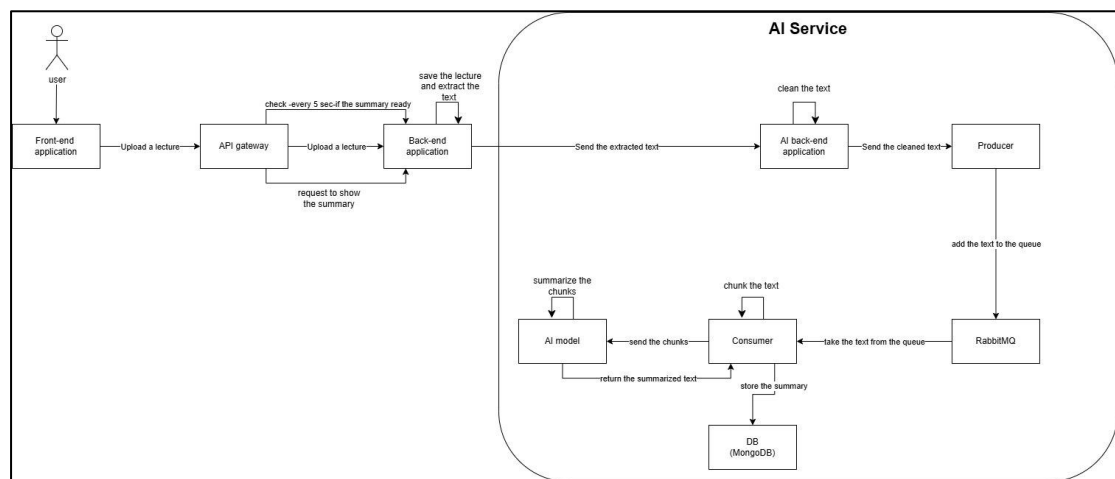


Figure 43 Summarization Service Flow

8.3.2.4 API Endpoints

Table 44 Summarization Service API Endpoints

Endpoint	HTTP Method	Description
/lecture-text/	POST	Submits a lecture text for processing. The text is cleaned and sent to the processing queue.
summary/status/<str:lecture_id>/	GET	Retrieves the current processing status of a lecture summary, indicating whether the summary is ready.
summary/<str:lecture_id>/	GET	Retrieves the final processed summary for a given lecture. Returns an error if the summary is not yet ready.

8.3.2.5 Technologies Used

- Django REST Framework
- Hugging Face / Qwen model
- MongoDB depending on stored data
- RabbitMQ for asynchronous processing

8.3.2.6. Service screenshots

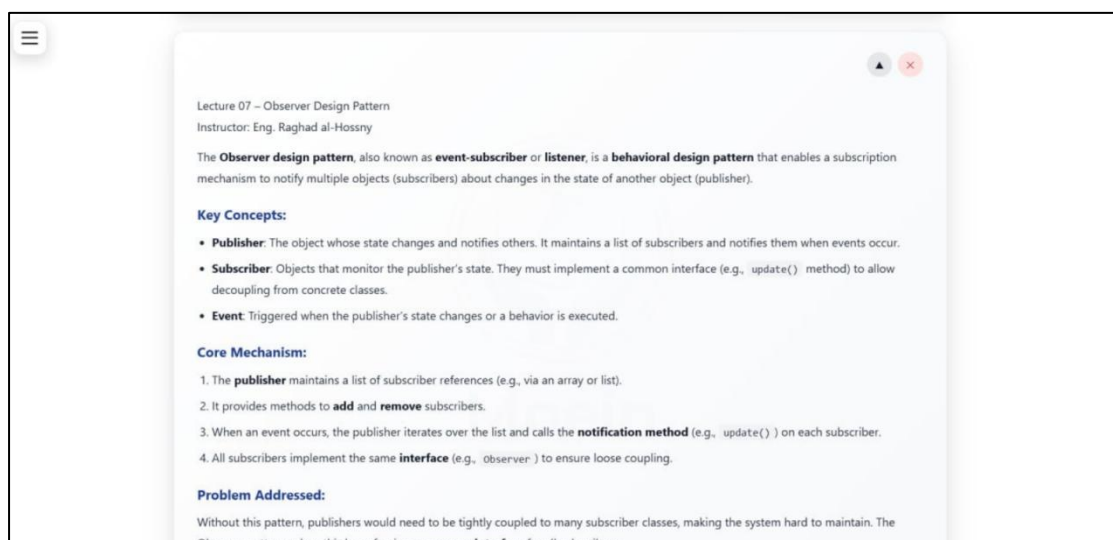


Figure44 Generated summary screenshot

8.3.3 Chatbot Module Service

8.3.3.1 Purpose

The Chatbot Module Service provides an AI-powered assistant that answers student questions based on uploaded lecture materials. It uses Retrieval-Augmented Generation to retrieve relevant lecture chunks and generate context-aware answers.

8.3.3.2 Responsibilities

The Chatbot Service is responsible for:

- Creating and managing chat sessions.
- Storing user messages and assistant responses.
- Chunking lecture text during ingestion.
- Generating embeddings using the embedding model.
- Storing vectors in Milvus.
- Retrieving relevant chunks based on student questions.
- Sending retrieved context to the LLM.
- Returning final answers to the student.

8.3.3.3 Service Flow

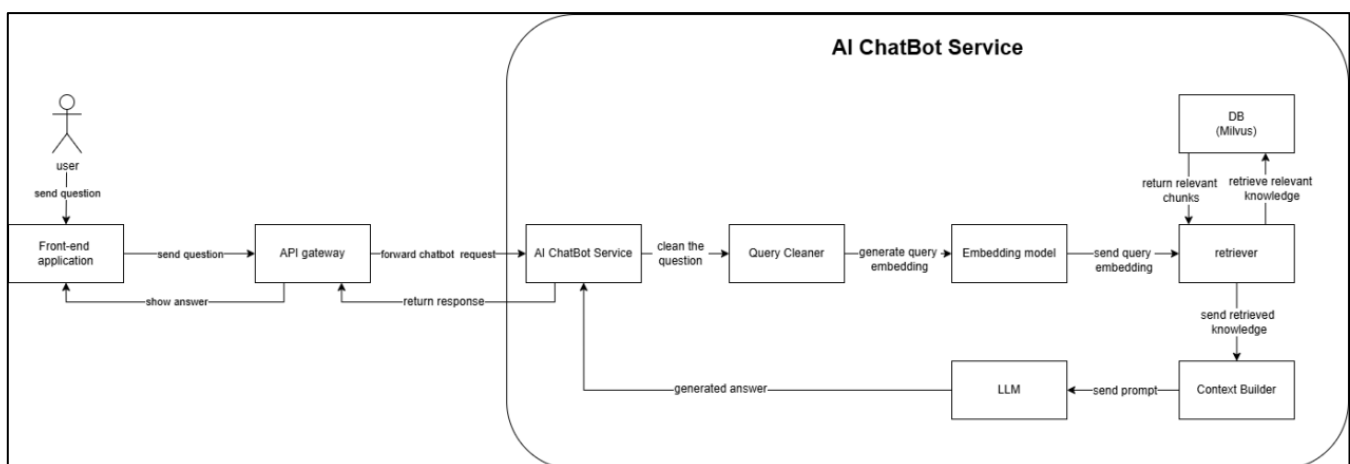


Figure 45 Chatbot Module Service Flow

8.3.3.4 API Endpoints

Table 45 Chatbot Module Service API Endpoints

Endpoint	HTTP Method	Description
/api/chat/lectures/ingest/	POST	Receives lecture text and stores chunks in the vector database.
/api/chat/sessions/create/	POST	Creates a new chat session.
/api/chat/sessions/	GET	Retrieves the user's chat sessions.
/api/chat/sessions/<session_id>/messages/	GET	Retrieves messages of a chat session.
/api/chat/sessions/<session_id>/messages/send/	POST	Sends a user question and returns the AI response.

8.3.3.5 Technologies Used

- Django REST Framework
- Qwen 7B LLM
- BGE-M3 embedding model
- Milvus vector database
- MySQL for chat sessions/messages
- RabbitMQ if asynchronous ingestion is used

8.3.4 Quiz Generator Service

8.3.4.1 Purpose

The Quiz Generator Service automatically generates quizzes from lecture content. It helps students evaluate their understanding by producing questions and checking submitted answers.

8.3.4.2 Responsibilities

The Quiz Generator Service is responsible for:

- Receiving lecture content or fetching lecture files.
- Extracting or using existing lecture text.
- Generating quiz questions using an LLM.
- Supporting MCQ, True/False, or mixed question modes.
- Saving generated quizzes.
- Receiving student submissions.
- Calculating score and returning feedback.

8.3.4.3 Service Flow

8.3.4.4 API Endpoints

Table 46 Quiz Generator Service API Endpoints

Endpoint	HTTP Method	Description
/quiz/generate/file/	POST	Generates a quiz from an uploaded file.
/quiz/generate/existing/	POST	Generates a quiz from an existing lecture.
/quiz/generate/scope/	POST	Generates a quiz from an existing lecture.
/quiz/<quiz_id>/status/	GET	Retrieves quiz generation status.
/quiz/<quiz_id>/	GET	Retrieves quiz details.
/quiz/<quiz_id>/submit/	POST	Submits answers and returns evaluation result.

8.3.4.5 Technologies Used

- Django REST Framework
- Qwen 7B LLM
- MongoDB for raw LLM logs and generated question data
- RabbitMQ for background quiz generation

8.3.5 Transcription Service

8.3.5.1 Purpose

The Transcription Service converts audio lecture files into text using speech-to-text technology. The generated text can then be used by other AI services such as summarization, chatbot, and quiz generation.

8.3.5.2 Responsibilities

The Transcription Service is responsible for:

- Receiving uploaded audio files.
- Creating transcription jobs.
- Processing audio using a speech-to-text model.
- Saving transcription results.
- Returning job status to the frontend.
- Allowing students to view the final transcript.

8.3.5.3 Service Flow

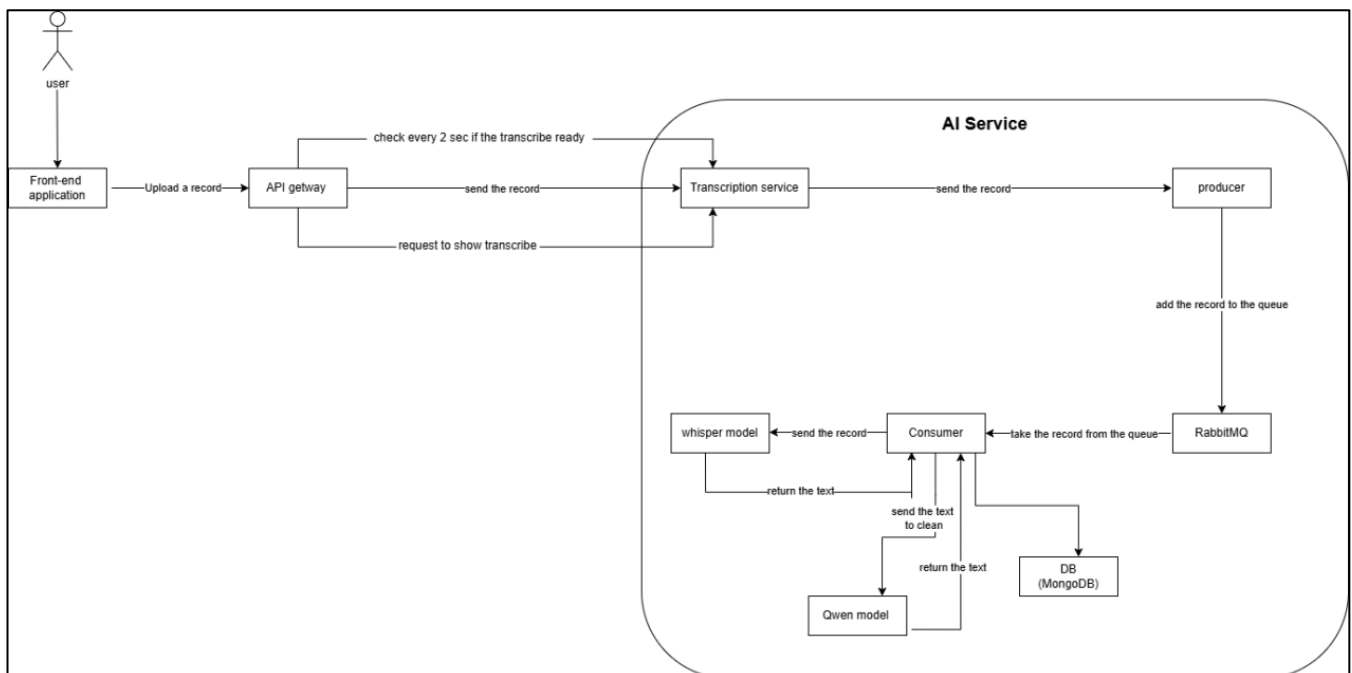


Figure 46 Transcription Service Flow

8.3.5.4 API Endpoints

Table 47 Transcription Service API Endpoints

Endpoint	HTTP Method	Description
/stt/upload/	POST	Uploads an audio file.
/stt/stt-status/<job_id>/	GET	Retrieves transcription job status.

8.3.5.5 Technologies Used

- Django REST Framework
- Faster Whisper / Whisper Medium
- MongoDB for transcription records
- RabbitMQ for asynchronous processing

8.3.6 Notification Service

8.3.6.1 Purpose

The Notification Service is responsible for delivering system messages to users. It supports both authentication-related notifications, such as OTP verification, and AI workflow notifications, such as completed summaries, quizzes, and transcriptions.

8.3.6.2 Responsibilities

The Notification Service is responsible for:

- Sending OTP codes during account verification.
- Creating notifications for completed AI tasks.
- Storing user notifications.
- Returning unread and read notifications to the frontend.
- Allowing users to mark notifications as read.
- Supporting communication between services through REST APIs or RabbitMQ.

8.3.6.3 Notification Types

The service supports different notification types, including:

Notification Type	Description
OTP Verification	Sends a one-time password to verify user registration or authentication.
Summary Completed	Notifies the student when lecture summarization is finished.
Quiz Completed	Notifies the student when quiz generation is finished.
Transcription Completed	Notifies the student when speech-to-text processing is finished.
Processing Failed	Notifies the student if an AI task fails.

8.3.6.4 API Endpoints

Endpoint	HTTP Method	Description
/api/notifications/user-notifications/	GET	Retrieves user notifications.
/api/notifications/mark-as-read/	POST	Marks a notification as read.
api/signup/	POST	Request a OTP code to complete the account creation process
api/notifications/send-otp/	POST	Verify the OTP code

8.3.6.5 Service flow

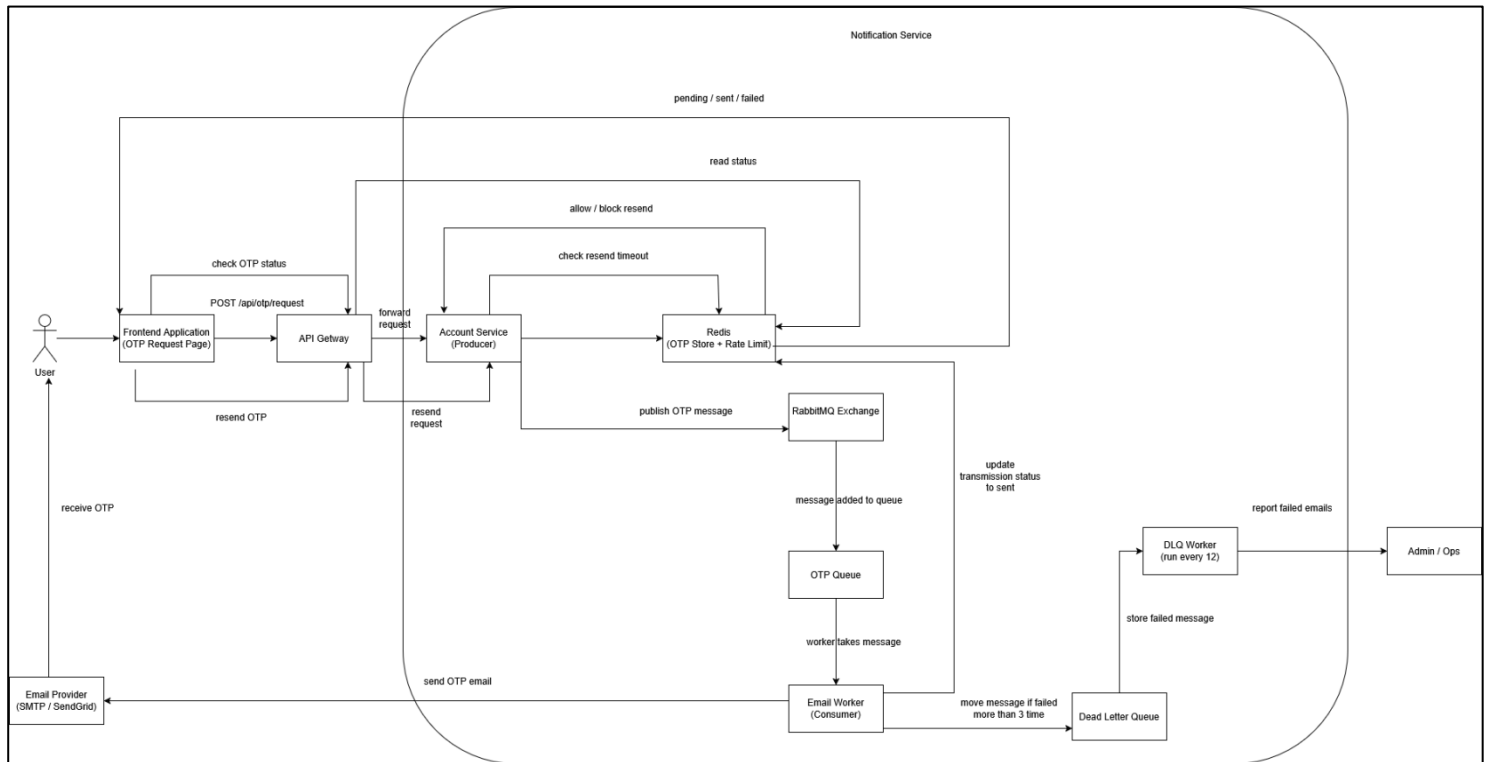


Figure 47 Notification service flow

Chapter 9 Conclusion

9.1 Introduction

As a result of our project, we have developed a software system designed to support software companies that follow the Agile methodology. This system streamlines the management of frequent and recurring meetings, ensuring that critical meeting artifacts are efficiently captured, organized, and utilized to enhance planning and decision-making for subsequent project phases. In this chapter we will conclude it all and put it all together, showing the final architecture, lessons learnt and the future possible enhancements.

9.2 Lessons Learnt

During the development of Moein, the team gained practical experience in building a complete AI-powered educational system using modern software engineering practices. One of the main lessons learned was the importance of clear system decomposition. Dividing the system into independent microservices made the project easier to organize, develop, test, and extend.

The team also learned that AI-based features require careful workflow design. Tasks such as summarization, quiz generation, transcription, and chatbot retrieval can take time to process, so asynchronous execution and status tracking are essential to keep the user interface responsive.

Another important lesson was the value of reliable integration between services. Since the system depends on communication between the frontend, API Gateway, backend services, databases, RabbitMQ, and AI models, proper API design, authentication headers, error handling, and logging were necessary for debugging and stability.

The project also highlighted the importance of data management. Different types of data require different storage solutions, such as MySQL for structured records, MongoDB for flexible AI outputs, and Milvus for vector embeddings used in semantic retrieval.

Finally, the team learned that building an AI educational assistant is not only about generating outputs, but also about making those outputs useful, accessible, and understandable for students. This helped the team focus on usability, organized workflows, and features that support real learning needs.

9.3 Future Recommendations

Future versions of Moein can be improved by enhancing the accuracy and reliability of the AI services, especially summarization, quiz generation, transcription, and chatbot responses. This can be achieved by testing stronger models, improving prompts, adding evaluation methods, and supporting more Arabic academic content.

The system can also be extended with more personalized learning features, such as adaptive study plans, progress tracking, weak-topic detection, and recommendations based on student performance in quizzes and chatbot interactions.

Another recommended improvement is to strengthen the retrieval-based chatbot by adding hybrid search, reranking, and better citation support. This would allow the chatbot to provide more accurate answers linked directly to the original lecture content.

From a technical perspective, future work should focus on improving deployment, monitoring, and scalability. Containerizing all services, adding centralized logging, monitoring service health, and optimizing background workers would make the system more production-ready.

Finally, Moein can be expanded to support more learning formats, such as video lectures, OCR for scanned PDFs, collaborative course spaces, and exporting summaries, quizzes, and transcripts in different formats.

9.4 Appendices

- [GitHub repository](#)

9.5 References

- <https://docs.djangoproject.com/en/5.0/>
- <https://docs.python.org/3/>
- <https://www.mongodb.com/docs/>
- <https://www.rabbitmq.com/docs>
- https://www.google.com/search?q=react+documentation&oq=react+documentation&gs_lcrp=EgZjaHJvbWUyBggAEEUYOTIGCAEQRRg7MgYIAhBFGDsyBggDEEUYO9IBCDQzMTdqMG_o3qAIAAsAIA&sourceid=chrome&ie=UTF-8